

Program: Write a program to print the type (category) uppercase, lowercase, Numeric character, Special character by using conditional operator only.

```
#include<stdio.h>

int main()
{
    char ch;
    printf("Enter The character : ");
    scanf("%c",&ch);
    ((ch>='A')&&(ch<='Z'))?printf("Uppercase Character\n"):
    ((ch>='a')&&(ch<='z'))?printf("Lowercase Charcater\n"):
    ((ch>='0')&&(ch<='9'))?printf("Digit\n"):
    printf("Special Character\n");
}
```

Program: Write a program to input integer data, test bit value of given position. (Using Conditional Operator)

```
#include<stdio.h>

int main()
{
    int data;
    int bitPosn;
    printf("Enter The Data : ");
    scanf("%d",&data);
    printf("Enter The Bit Posn : ");
    scanf("%d",&bitPosn);
    ((data>>bitPosn)&1)?printf("Set Bit\n"):printf("Clear Bit\n");
}
```

Program: Write a program to input integer data, test bit value of given position.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int data;
```

```
    int bitPosn;
```

```
    printf("Enter The Data : ");
```

```
    scanf("%d",&data);
```

```
INPUT: printf("Enter The BitPosn (0 to %lu) : ",sizeof(int)*8-1);
```

```
    scanf("%d",&bitPosn);
```

```
    if((bitPosn<0)|| (bitPosn>=sizeof(int)*8))
```

```
    {
```

```
        printf("Error : Invalid bit Supply\n");
```

```
        goto INPUT;
```

```
    }
```

```
    printf("Bit : %d in data : %d is : %d\n",bitPosn,data,(data>>bitPosn)&1);
```

```
}
```

Program: Write a program to print the binary of given input data.

```
#include<stdio.h>

int main()
{
    int data;
    int bitPosn;
    printf("Enter The data : ");
    scanf("%d",&data);
    printf("Enter bit Equat. of : %d\n",data);
    bitPosn=31;
TEST:  printf("%d",(data>>bitPosn)&1);
        if(bitPosn%8==0)
            printf(" ");
        bitPosn--;
        if(bitPosn<=0)
            goto TEST;
        printf("\n");
}
```

Program: Write a program to print the type (category) uppercase, lowercase, Numeric character, Special character by using library function.

```
#include<stdio.h>
```

```
#include<ctype.h>
```

```
int main()
```

```
{
```

```
    char ch;
```

```
    printf("Enter The Character : ");
```

```
    scanf("%c",&ch);
```

```
    isupper(ch)?printf("Uppercase Character\n");
```

```
    islower(ch)?printf("Lowercase Charcater\n");
```

```
    isdigit(ch)?printf("Digit Character\n");
```

```
    printf("Special Character\n");
```

```
}
```

Program: Write a program to print the type (category) uppercase, lowercase, Numeric character, Special character by using if statement.

```
#include<stdio.h>

int main()
{
    char ch;

    printf("Enter The Character : ");
    scanf("%c",&ch);

    if((ch>='A')&&(ch<='Z'))
        printf("Uppercase Character\n");
    if((ch>='a')&&(ch<='z'))
        printf("Lowercase Character\n");
    if((ch>='0')&&(ch<='9'))
        printf("Digit\n");

    if(!(((ch>='a')&&(ch<='z'))||((ch>='A')&&(ch<='Z'))||((ch>='0')&&(ch<='9'))))
        printf("Special Character\n");
}
```

Program: Write a program to print the type (category) uppercase, lowercase, Numeric character, Special character by using else if statement.

```
#include<stdio.h>

int main()
{
    char ch;
    printf("Enter The Character : ");
    scanf("%c",&ch);
    if((ch>='a')&&(ch<='z'))
        printf("Lowercase Character\n");
    else if((ch>='A')&&(ch<='Z'))
        printf("Uppercase Character\n");
    else if((ch>='0')&&(ch<='9'))
        printf("Digit\n");
    else
        printf("Special Character\n");
}
```

Program: Write a program to scan 3 integers and find the relation between them. Find which is higher, all three, two of them or only one using only conditional operator.

```
#include<stdio.h>

int main()
{
    int a;
    int b;
    int c;
    printf("Enter The Integer a : ");
    scanf("%d",&a);
    printf("Enter The Integer b : ");
    scanf("%d",&b);
    printf("Enter The Integer c : ");
    scanf("%d",&c);

    ((a>b)&&(a>c))?printf("a is max\n"):
    ((b>a)&&(b>c))?printf("b is max\n"):
    ((c>a)&&(c>b))?printf("c is max\n"):
    ((a==b)&&(a>c))?printf("a and b max\n"):
    ((b==c)&&(b>c))?printf("b and c max\n"):
    ((c==a)&&(c>b))?printf("c and a max\n"):
    printf("a, b, and c max\n");
}
```

Program: Write a program to convert lowercase to uppercase and vice versa.

Method 1:

```
#include<stdio.h>

int main()
{
    char ch;
    printf("Enter The Character : ");
    scanf("%c",&ch);
    if((ch>='A')&&(ch<='Z'))
        ch+=32;
    else
        ch-=32;
    printf("Conveted Character is : %c\n",ch);
}
```


Method 2:

```
#include<stdio.h>
#include<ctype.h>
int main()
{
    char ch;
    printf("Enter The Character : ");
    scanf("%c",&ch);
    if(isalpha)
    {
        ch^=32;
        printf("Converted Case is : %c\n",ch);
    }
    else
    {
        printf("Error : Invalid Input\n");
    }
}
```

Program: Write a program to scan 3 integers and find the complete relation between them using conditional operator only.

```
#include<stdio.h>

int main()
{
    int a;
    int b;
    int c;
    printf("Enter The Integer a : ");
    scanf("%d",&a);
    printf("Enter The Integer b : ");
    scanf("%d",&b);
    printf("Enter The Integer c : ");
    scanf("%d",&c);

    ((a==b)&&(a>c))?printf("a=b both are high\n"):
    ((a==b)&&(a<c))?printf("a=b and c is high\n"):
    ((b==c)&&(b>a))?printf("b=c and both high\n"):
    ((b==c)&&(b<a))?printf("b=c and a high\n"):
    ((c==a)&&(c>b))?printf("c=a and c high\n"):
    ((c==a)&&(c<b))?printf("c=a and b is high\n"):
    ((a>b)&&(a>c))?printf("a is high\n"):
    ((b>c)&&(b>a))?printf("b is high\n"):
    ((c>a)&&(c>b))?printf("c is high\n"):
    printf("a=b=c\n");
}
```

Program: Write a program to input integer data, test bit value of given position.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int data;
```

```
    int bit;
```

```
    printf("Enter The data : ");
```

```
    scanf("%d",&data);
```

```
INPUT: printf("Enter The Bit Posn : ");
```

```
    scanf("%d",&bit);
```

```
    if((bit>=0)&&(bit<=31))
```

```
    {
```

```
        if((data>>bit)&1)
```

```
            printf("Bit is Set\n");
```

```
        else
```

```
            printf("Bit is Clear\n");
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("Invalid Bit: Enter Bit Again\n");
```

```
        goto INPUT;
```

```
    }
```

```
}
```

Program: Write a program to print the binary of an integer value entered.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int data;
```

```
    int bit;
```

```
    printf("Enter The data : ");
```

```
    scanf("%d",&data);
```

```
    bit=31;
```

```
TEST:  if((data>>bit)&1)
```

```
        printf("1");
```

```
    else
```

```
        printf("0");
```

```
    bit--;
```

```
    if(bit>=0)
```

```
        goto TEST;
```

```
    printf("\n");
```

```
}
```

Program: Write a program to calculate percentage scored by a student, whose 5 subject marks are entered by user. Find the gradation. (Using method – if, else if ladder, nested if)

Method 1: Using if Statement –

```
#include<stdio.h>

int main()
{
    float a,b,c,d,e;
    float p;
    printf("Enter subject 1 marks : ");
    scanf("%f",&a);
    printf("Enter subject 2 marks : ");
    scanf("%f",&b);
    printf("Enter subject 3 marks : ");
    scanf("%f",&c);
    printf("Enter subject 4 marks : ");
    scanf("%f",&d);
    printf("Enter subject 5 marks : ");
    scanf("%f",&e);

    p=((a+b+c+d+e)/5);
    printf("Scored Percentage : %f\n",p);
    printf("Earned Grage : ");

    if((p>=0)&&(p<=40))
        printf("Fail\n");
    if((p>=40)&&(p<=50))
        printf("Third Class\n");
```

```
    if((p>=50)&&(p<=60))
        printf("Second Class\n");
    if((p>=60)&&(p<=75))
        printf("First Class\n");
    if((p>=75)&&(p<=100))
        printf("Destionction\n");
}
```

Using else if statement –

```
#include<stdio.h>

int main()
{
    float a,b,c,d,e;
    float p;
    printf("Enter subject 1 marks : ");
    scanf("%f",&a);
    printf("Enter subject 1 marks : ");
    scanf("%f",&b);
    printf("Enter subject 1 marks : ");
    scanf("%f",&c);
    printf("Enter subject 1 marks : ");
    scanf("%f",&d);
    printf("Enter subject 1 marks : ");
    scanf("%f",&e);
    p=((a+b+c+d+e)/5);
    printf("Scored Percentage : %f\n",p);
    printf("Earned Grade : ");

    if((p>=0)&&(p<=40))
        printf("Fail\n");
    else if((p>=40)&&(p<=50))
        printf("Third Class\n");
    else if((p>=50)&&(p<=60))
        printf("Second Class\n");
    else if((p>=60)&&(p<=75))
```

```
        printf("First Class\n");  
    else  
        printf("Distinction\n");  
}
```


Nested if else –

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    float a,b,c,d,e,f;
```

```
    float p;
```

```
    printf("Enter subject 1 marks : ");
```

```
    scanf("%f",&a);
```

```
    printf("Enter subject 2 marks : ");
```

```
    scanf("%f",&b);
```

```
    printf("Enter subject 3 marks : ");
```

```
    scanf("%f",&c);
```

```
    printf("Enter subject 4 marks : ");
```

```
    scanf("%f",&d);
```

```
    printf("Enter subject 5 marks : ");
```

```
    scanf("%f",&e);
```

```
    p=((a+b+c+d+e)/5);
```

```
    printf("Scored Percentage is : %f\n",p);
```

```
    printf("Earned Grade : ");
```

```
if(p>40)
{
    if(p>50)
    {
        if(p>60)
        {
            if(p>75)
                printf("Destinction\n");
            else
                printf("1st Class\n");
        }
        else
            printf("2nd Class\n");
    }
    else
        printf("3rd class\n");
}
else
    printf("Fail\n");
}
```

Program: Write a program to print all the character and their respective ASCII equivalent (Without using loop statement)

```
#include<stdio.h>

int main()
{
    int data=0;
    printf("character\tASCII\n");
LOOP:  printf("%c\t%d\n",data,data);
        data++;
        if(data<128)
            goto LOOP;
}
```

Program: Write a program to set particular bit in given data.

```
#include<stdio.h>

int main()
{
    int data;
    int bit;
    printf("Enter The data : ");
    scanf("%d",&data);
    printf("Enter The Bit : ");
    scanf("%d",&bit);
    data=data|(1<<bit);
    printf("Result after Set The Bit : %d\n",data);
}
```

Program: Write a program to print character, its ASCII value and its binary value. (Without using looping statement)

```
#include<stdio.h>

int main()
{
    int data=0;
    int bit;
    printf("Character\tASCII\tBinary\n");
LOOP:  printf("%c\t%d\t",data,data);
    bit=7;
BIN:   if((data>>bit)&1)
        printf("1");
    else
        printf("0");
    bit--;
    if(bit>=0)
        goto BIN;
    printf("\n");
    data++;
    if(data<128)
        goto LOOP;
}
```

Program: Write a program to print character, its ASCII value and its binary value.

```
#include<stdio.h>

int main()
{
    int data;
    int bit;
    printf("Symbol\tASCII\tBinary\n");
    data=0;
    while(data<=127)
    {
        printf("%c\t%d\t",data,data);
        bit=7;
        while(bit>=0)
        {
            printf("%d",(data>>bit)&1);
            bit--;
        }
        printf("\n");
        data++;
    }
}
```

Program: Write a program to clear a particular bit in given data.

```
#include<stdio.h>

int main()
{
    int data;
    int bit;
    printf("Enter The Data : ");
    scanf("%d",&data);
    printf("Enter The Bit : ");
    scanf("%d",&bit);
    data=data&~(1<<bit);
    printf("Result after Bit Clear : %d\n",data);
}
```

Program: Write a program to complement a particular bit in given data.

```
#include<stdio.h>

int main()
{
    int data;
    int bit;
    printf("Enter The data : ");
    scanf("%d",&data);
    printf("Enter The bit : ");
    scanf("%d",&bit);
    data=data^(1<<bit);
    printf("Result after complement : %d\n",data);
}
```

Program: Pattern Printing

1)

```
#include<stdio.h>

int main()
{
    int a;
    int b;
    for(a=1;a<=5;a++,printf("\n"))
        for(b=1;b<=a;b++)
            printf("* ");
}
```

*

* *

* * *

* * * *

* * * * *

2)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int a;
```

```
    int b;
```

```
    for(a=1;a<=5;a++,printf("\n"))
```

```
    {
```

```
        for(b=1;b<=5-a;b++)
```

```
            printf(" ");
```

```
        for(b=1;b<=a;b++)
```

```
            printf("* ");
```

```
    }
```

```
}
```

```
    *
```

```
    * *
```

```
    * * *
```

```
    * * * *
```

```
    * * * * *
```


3)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int i,j;
```

```
    for(i=1;i<=5;i++,printf("\n"))
```

```
        for(j=1;j<=5;j++)
```

```
            (j<i)?printf(" "):printf("* ");
```

```
}
```

```
* * * * *
```

```
* * * *
```

```
* * *
```

```
* *
```

```
*
```

4)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int i,j;
```

```
    for(i=1;i<=5;i++,printf("\n"))
```

```
        for(j=1;j<=5;j++)
```

```
            (j<i)?printf(" "):printf("* ");
```

```
}
```

```
* * * * *
```

```
* * * *
```

```
* * *
```

```
* *
```

```
*
```

5)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int i,j,flag=1;
```

```
    for(i=1;i<=5;i++,printf("\n"))
```

```
    {
```

```
        if(i%2==0)
```

```
            flag=0;
```

```
        else
```

```
            flag=1;
```

```
        for(j=1;j<=i;j++)
```

```
        {
```

```
            printf("%d ",flag);
```

```
            flag=!flag;
```

```
        }
```

```
    }
```

```
}
```

1

0 1

1 0 1

0 1 0 1

1 0 1 0 1

6)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int i,j,flag=65;
```

```
    for(i=1;i<=5;i++,printf("\n"))
```

```
    {
```

```
        for(j=5,flag=65;j>=i;j--)
```

```
        {
```

```
            printf("%c ",flag);
```

```
            flag++;
```

```
        }
```

```
    }
```

```
}
```

A B C D E

A B C D

A B C

A B

A

7)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int i,j;
```

```
    for(i=1;i<=5;i++,printf("\n"))
```

```
        for(j=5;j>=1;j--)
```

```
            (j>i)?printf(" "):printf("* ");
```

```
    for(i=1;i<=4;i++,printf("\n"))
```

```
        for(j=1;j<=5;j++)
```

```
            (j<=i)?printf(" "):printf("* ");
```

```
}
```

```
    *
```

```
    * *
```

```
    * * *
```

```
    * * * *
```

```
    * * * * *
```

```
    * * * * *
```

```
    * * *
```

```
    * *
```

```
    *
```

8)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int i,j,k;
```

```
    for(i=1;i<=5;i++,printf("\n"))
```

```
        for(j=5;j>=1;j--)
```

```
            (j>i)?printf(" "):printf("* ");
```

```
    for(i=1;i<=4;i++,printf("\n"))
```

```
        for(j=1;j<=5;j++)
```

```
            (j<=i)?printf(" "):printf("* ");
```

```
}
```

```
    *
```

```
    * *
```

```
    * * *
```

```
    * * * *
```

```
* * * * *
```

```
* * * *
```

```
* * *
```

```
* *
```

```
*
```

9)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int i,j;
```

```
    for(i=1;i<=5;i++,printf("\n"))
```

```
        for(j=1;j<=i;j++)
```

```
            printf("* ");
```

```
    for(i=1;i<=4;i++,printf("\n"))
```

```
        for(j=5;j>i;j--)
```

```
            printf("* ");
```

```
}
```

```
*  
  
* *  
  
* * *  
  
* * * *  
  
* * * * *  
  
* * * * *  
  
* * *  
  
* *  
  
*
```

10)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int i,j;
```

```
    for(i=1;i<=5;i++,printf("\n"))
```

```
        for(j=1;j<=5;j++)
```

```
            (j<i)?printf(" "):printf("* ");
```

```
    for(i=1;i<=4;i++,printf("\n"))
```

```
        for(j=5;j>=1;j--)
```

```
            (j>i+1)?printf(" "):printf("* ");
```

```
}
```

```
* * * * *
```

```
* * * *
```

```
* * *
```

```
* *
```

```
*
```

```
* *
```

```
* * *
```

```
* * * *
```

```
* * * * *
```


11)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int i,j;
```

```
    for(i=1;i<=5;i++,printf("\n"))
```

```
        for(j=1;j<=5;j++)
```

```
            (j<i)?printf(" "):printf("* ");
```

```
    for(i=1;i<=4;i++,printf("\n"))
```

```
        for(j=5;j>=1;j--)
```

```
            (j>i+1)?printf(" "):printf("* ");
```

```
}
```

```
* * * * *
```

```
* * * *
```

```
* * *
```

```
* *
```

```
*
```

```
* *
```

```
* * *
```

```
* * * *
```

```
* * * * *
```

12)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int i,j;
```

```
    for(i=1;i<=5;i++,printf("\n"))
```

```
        for(j=5;j>=i;j--)
```

```
            printf("* ");
```

```
    for(i=1;i<=4;i++,printf("\n"))
```

```
        for(j=1;j<=i+1;j++)
```

```
            printf("* ");
```

```
}
```

```
* * * * *
```

```
* * * *
```

```
* * *
```

```
* *
```

```
*
```

```
* *
```

```
* * *
```

```
* * * *
```

```
* * * * *
```

13)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int a,b,c,n,f=1;
```

```
    char ch=65;
```

```
    printf("Enter The Number N : ");
```

```
    scanf("%d",&n);
```

```
    for(a=1;a<=n;a++,printf("\n"))
```

```
    {
```

```
        f=1;
```

```
        for(b=1;b<=n;b++)
```

```
        {
```

```
            if(b<=(n-a))
```

```
                printf(" ");
```

```
            else if(f==1)
```

```
            {
```

```
                printf("%c",ch);
```

```
                f=0;
```

```
            }
```

```
            else if(b==n)
```

```
            {
```

```
                printf("*%c",ch);
```

```
            }
```

```
        else
```

```
        {
```

```
            printf("***");
```

```
        }  
    }  
    ch++;  
}  
}
```

Output:

Enter The Number N : 5

A

B*B

C***C

D*****D

E*****E

14)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int a,b,B,n;
```

```
    int f;
```

```
    char ch;
```

```
    printf("Enter The N : ");
```

```
    scanf("%d",&n);
```

```
    for(a=0;a<n;a++,printf("\n"))
```

```
    {
```

```
        f=1;
```

```
        ch=65;
```

```
        for(b=-(n-1);b<n;b++)
```

```
        {
```

```
            if(b<0)
```

```
                B=-b;
```

```
            else
```

```
                B=b;
```

```
            if(B<=a)
```

```
            {
```

```
                if(f==1)
```

```
                {
```

```
                    printf("%c",ch++);
```

```
                    f=0;
```

```
                }
```

```
        else
        {
            printf("*");
            f=1;
        }
    }
    else
    {
        printf(" ");
    }
}
}
```

Output:

Enter The N : 5

A

A*B

A*B*C

A*B*C*D

A*B*C*D*E

15)

```
#include<stdio.h>

int main()
{
    int a,b,n;
    printf("Enter The N : ");
    scanf("%d",&n);
    for(a=n;a>0;a--,printf("\n"))
        for(b=1;b<=a;b++)
        {
            if(a%2==0)
                printf("*");
            else
                printf("%d",a);
        }
}
```

Output:

Enter The N : 5

55555

333

**

1

Program: Write a program to reverse bits of a given integer number.

```
#include<stdio.h>

int main()
{
    int data;
    int bit;
    int i,j;
    printf("Enter The data : ");
    scanf("%d",&data);
    for(bit=31;bit>=0;bit--)
        printf("%d", (data>>bit)&1);
    printf("\n");
    for(i=31,j=0;i>j;i--,j++)
    {
        if(((data>>i)&1) != ((data>>j)&1))
        {
            data=data^(1<<i);
            data=data^(1<<j);
        }
    }
    for(bit=31;bit>=0;bit--)
        printf("%d", (data>>bit)&1);
    printf("\n");
}
```


Program: Write a program to count the digits of an integer number.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int data;
```

```
    int count=0;
```

```
    printf("Enter The data : ");
```

```
    scanf("%d",&data);
```

```
    if(data==0)
```

```
        printf("Count : 1\n");
```

```
    for(int i=data;i;i=i/10)
```

```
        count++;
```

```
    printf("Count : %d\n",count);
```

```
}
```

Program: Write a program to calculate sum of digit in a given integer.

```
#include<stdio.h>

int main()
{
    int data;
    int sum=0;

    printf("Enter The data : ");
    scanf("%d",&data);
    for(int i=data;i>0;i=i/10)
        sum+=i%10;
    printf("Sum is : %d\n",sum);
}
```

Program: Write a program to reverse digits of given integers.

```
#include<stdio.h>

int main()
{
    int data;
    int sum=0;

    printf("Enter The data : ");
    scanf("%d",&data);
    for(int i=data;i>0;i=i/10)
        sum=sum*10+i%10;
    printf("Reverse Number is : %d\n",sum);
}
```

Program: Write a program to find the factorial of given integer.

```
#include<stdio.h>

int main()
{
    int data;
    int fact=1;
    printf("Enter The Data : ");
    scanf("%d",&data);
    for(int i=data;i;i--)
        fact*=i;
    printf("Factorial is : %d\n",fact);
}
```

Program: Write a program to print Fibonacci series up to given max value.

```
#include<stdio.h>

int main()
{
    int max;
    int a=0,b=1;
    int c;
    printf("Enter The Max Number : ");
    scanf("%d",&max);
    printf("0, 1, ");
    for(c=a+b;(c=a+b)<=max;a=b,b=c)
        printf("%d, ",c);
    printf("\n");
}
```

Program: Write a program to input two integer x and y, calculate x^y .

```
#include<stdio.h>

int main()
{
    int x;
    int y;
    int result=1;
    printf("Enter The X value : ");
    scanf("%d",&x);
    printf("Enter The Y Value : ");
    scanf("%d",&y);
    for(int i=y;i-->0)
        result=result*x;
    printf("%d^%d is %d\n",x,y,result);
}
```

Program: Write a program check if x is power of y or not.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int x,y;
```

```
    int flag;
```

```
    int count=0,temp;
```

```
    printf("Enter value for x : ");
```

```
    scanf("%d",&x);
```

```
    printf("Enter value for y : ");
```

```
    scanf("%d",&y);
```

```
    if(y==x)
```

```
    {
```

```
        flag=1;
```

```
        goto END;
```

```
    }
```

```
    for(temp=x;temp>1;temp=temp/y)
```

```
    {
```

```
        if(temp%y==0)
```

```
        {
```

```
            count++;
```

```
            continue;
```

```
        }
```

```
    else
```

```
    {
```

```
        flag=1;
```

```
        break;
    }
}
END:  if(flag==1)
{
    if(x==y)
        printf("%d is 1 power of %d\n",x,y);
    else
        printf("%d is not power of %d\n",x,y);
}
else
    printf("%d is %d power of %d\n",x,count,y);
}
```

Program: Write a program to check inter number is prime or not.

```
#include<stdio.h>
#include<math.h>
int main()
{
    int data;
    int i;
    int s;
    printf("Enter The Data : ");
    scanf("%d",&data);
    s=sqrt(data);
    for(i=2;i<=s;i++)
    {
        if((data%i)==0)
            break;
    }
    if(i==s+1)
        printf("Entered Number is Prime Number\n");
    else
        printf("Entered Number is Not Prime Number\n");
}
```

Program: Write a program to print the prime number in given range.

```
#include<stdio.h>
#include<math.h>
int main()
{
    int min,max;
    int num;
    int i,s;
    printf("Enter The Min Number : ");
    scanf("%d",&min);
    printf("Enter The Max Number : ");
    scanf("%d",&max);
    for(num=min;num<=max;num++)
    {
        s=sqrt(num);
        for(i=2;i<=s;i++)
        {
            if((num%i)==0)
                break;
        }
        if(i==s+1)
            printf("%d, ",num);
    }
    printf("\n");
}
```



```
#include<stdio.h>
#include<math.h>
int testPrime(int);
int main()
{
    int data;
    printf("Enter The Data : ");
    scanf("%d",&data);
    if(testPrime(data))
        printf("%d is Prime\n",data);
    else
        printf("%d is not-Prime\n",data);
}
int testPrime(int data)
{
    int i,s;
    s=sqrt(data);
    for(i=2;i<=s;i++)
    {
        if(data%i==0)
            break;
    }
    if(i==(s+1))
        return 1;
    else
        return 0;
}
```

Program: Write a program to print multiplication table in given range.

```
#include<stdio.h>

int main()
{
    int min,max;
    int temp,i,j;
    int sum=0;

    printf("Enter The Min Number : ");
    scanf("%d",&min);
    printf("Enter The Max Number : ");
    scanf("%d",&max);

    for(temp=max*10;temp;temp=temp/10)
        sum++;

    for(i=min;i<=max;i++)
    {
        printf("%*d : ",sum-1,i);
        for(j=1;j<=10;j++)
            printf("%*d",sum,i*j);
        printf("\n");
    }
}
```

Program: Write a program to check enter number is prime or not.

```
#include<stdio.h>
```

```
#include<math.h>
```

```
int testPrime(int);
```

```
int main()
```

```
{
```

```
    int data;
```

```
    printf("Enter The Data : ");
```

```
    scanf("%d",&data);
```

```
    if(testPrime(data))
```

```
        printf("Entered Number is Prime\n");
```

```
    else
```

```
        printf("Enter Number is Not Prime\n");
```

```
}
```

```
int testPrime(int data)  
{  
    int squire;  
    int i;  
    squire=sqrt(data);  
    for(i=2;i<=squire;i++)  
    {  
        if(data%i==0)  
            break;  
    }  
    if(i==(squire+1))  
        return 1;  
    else  
        return 0;  
}
```

Program: Write a program to check whether entered number is palindrome or not.

```
#include<stdio.h>

int main()
{
    int data;
    printf("Enter The Data : ");
    scanf("%d",&data);
    if(testRev(data)==data)
        printf("Palindrome\n");
    else
        printf("Not Palindrome\n");
}

int testRev(int data)
{
    int rev=0;
    while(data)
    {
        rev=rev*10+data%10;
        data/=10;
    }
    return rev;
}
```

Program: Write a program list the numbers between min and max, where sum of digits is 9.

```
#include<stdio.h>
```

```
int sumDigits(int);
```

```
int main()
```

```
{
```

```
    int min,max;
```

```
    int data;
```

```
    printf("Enter The Min Digit : ");
```

```
    scanf("%d",&min);
```

```
    printf("Enter The Max Digit : ");
```

```
    scanf("%d",&max);
```

```
    for(data=min;data<=max;data++)
```

```
        if(sumDigits(data)==9)
```

```
            printf("%d, ",data);
```

```
    printf("\n");
```

```
}
```

```
int sumDigits(int data)
```

```
{
```

```
    int sum=0;
```

```
    while(data)
```

```
    {
```

```
        sum=sum+data%10;
```

```
        data/=10;
```

```
    }
```

```
    return sum;
```

```
}
```

Program: Write a program to make arrangement to list the numbers the sum of digits, reduce to single digit is 9.

```
#include<stdio.h>

int main()
{
    int min,max;
    int data;
    int sum=0;
    int temp;
    printf("Enter The Min Digit : ");
    scanf("%d",&min);
    printf("Enter The Mix Digit : ");
    scanf("%d",&max);
    for(data=min;data<=max;data++)
    {
        temp=data;
        while(temp)
        {
            sum=sum+temp%10;
            temp/=10;
        }
        if(sum>9)
        {
            temp=sum;
            sum=0;
            while(temp)
            {
                sum=sum+temp%10;
```

```
        temp/=10;
    }
}
if(sum==9)
    printf("%d, ",data);
}
printf("\n");
}
```


Program: Write a program to make arrangement to list the numbers if digits are in sorted order in ascending order.

```
#include<stdio.h>

int testDigits(int);
int testAsc(int);

int main()
{
    int min,max;
    int data;
    printf("Enter The Min Value : ");
    scanf("%d",&min);
    printf("Enter The Max Value : ");
    scanf("%d",&max);
    for(data=min;data<=max;data++)
        if(testDigits(data)==9)
            if(testAsc(data))
                printf("%d, ",data);
}

int testDigits(int data)
{
    int sum=0;
    while(data)
    {
        sum=sum+data%10;
        data/=10;
    }
    return sum;
}
```

```
int testAsc(int data)
{
    int curDig;
    int preDig=9;
    while(data)
    {
        curDig=data%10;
        if(curDig>preDig)
            return 0;
        preDig=curDig;
        data/=10;
    }
    if(data==0)
        return 1;
    else
        return 0;
}
```

Program: Write a program to print the N numbers starting from S.

```
#include<stdio.h>
#include<math.h>
int testPrime(int);
int main()
{
    int num,strt,data;
    int cnt=0;
    printf("Enter Numbers of prime Number you want : ");
    scanf("%d",&num);
    printf("Enter Starting Number : ");
    scanf("%d",&strt);

    for(data=strt;cnt<num;data++)
    {
        if(testPrime(data))
        {
            printf("%d, ",data);
            cnt++;
        }
    }
    printf("\n");
}
```

```
int testPrime(int data)
```

```
{
```

```
    int s,i;
```

```
    s=sqrt(data);
```

```
    for(i=2;i<=s;i++)
```

```
    {
```

```
        if(data%i==0)
```

```
            break;
```

```
    }
```

```
    if(i==(s+1))
```

```
        return 1;
```

```
    else
```

```
        return 0;
```

```
}
```

Program: Write a program to find the factorial of a number by recursive method.

```
#include<stdio.h>
```

```
int fact(int);
```

```
int main()
```

```
{
```

```
    int data;
```

```
    printf("Enter The Data : ");
```

```
    scanf("%d",&data);
```

```
    printf("Factorial Of %d is %d\n",data,fact(data));
```

```
}
```

```
int fact(int data)
```

```
{
```

```
    if(data==0)
```

```
        return(1);
```

```
    else
```

```
        return(data*fact(data-1));
```

```
}
```

Program: Write a program to generate Fibonacci series using recursion.

```
#include<stdio.h>
```

```
int fib(int);
```

```
int main()
```

```
{
```

```
    int data,i;
```

```
    printf("Enter The Data : ");
```

```
    scanf("%d",&data);
```

```
    for(i=0;i<data;i++)
```

```
        printf("%d, ",fib(i));
```

```
    printf("\n");
```

```
}
```

```
int fib(int data)
```

```
{
```

```
    if(data==0 || data==1)
```

```
        return(1);
```

```
    else
```

```
        return(fib(data-1)+fib(data-2));
```

```
}
```

Program: Write a program to find the sum of the digits of any number.

```
#include<stdio.h>
```

```
int sumofDigits(int);
```

```
int main()
```

```
{
```

```
    int data;
```

```
    printf("Enter The data : ");
```

```
    scanf("%d",&data);
```

```
    printf("Sum of Digit of %d is %d\n",data,sumofDigits(data));
```

```
}
```

```
int sumofDigits(int data)
```

```
{
```

```
    int sum=0;
```

```
    int rem;
```

```
    while(data>0)
```

```
    {
```

```
        rem=data%10;
```

```
        sum=sum+rem;
```

```
        data/=10;
```

```
    }
```

```
    return sum;
```

```
}
```

Program: Write a program that return the sum of square of all odd numbers from 1 to 25.

```
#include<stdio.h>

int main()
{
    int num;
    int s=0;
    for(num=1;num<=25;num++)
    {
        if(num%2!=0)
            s+=num*num;
    }
    printf("%d",s);
}
```


Program: Write a program to find out the factorial of a number.

```
#include<stdio.h>
```

```
int factData(int);
```

```
int main()
```

```
{
```

```
    int data;
```

```
    printf("Enter The data : ");
```

```
    scanf("%d",&data);
```

```
    if(data<0)
```

```
        printf("No Factorial For Negative Number\n");
```

```
    else
```

```
        printf("Factorial Of %d is %d\n",data,factData(data));
```

```
}
```

```
int factData(int data)
```

```
{
```

```
    int i,fact=1;
```

```
    if(data==0)
```

```
        return 1;
```

```
    for(i=data;i>1;i--)
```

```
        fact=fact*i;
```

```
    return fact;
```

```
}
```

Program: Write a program to find whether a number is even or odd.

```
#include<stdio.h>
```

```
int findOddEven(int);
```

```
int main()
```

```
{
```

```
    int data;
```

```
    printf("Enter The Data : ");
```

```
    scanf("%d",&data);
```

```
    findOddEven(data);
```

```
}
```

```
int findOddEven(int data)
```

```
{
```

```
    if(data%2==0)
```

```
        printf("%d is Even\n",data);
```

```
    else
```

```
        printf("%d is Odd\n",data);
```

```
}
```

Program: Write a program to find the sum and average of 10 positive integers.

```
#include<stdio.h>

int main()
{
    int i=1,n,sum=0;
    float avg;
    printf("Enter The 10 Positive Number\n");
    while(i<=10)
    {
        printf("Enter The Number %d : ",i);
        scanf("%d",&n);
        if(n<0)
        {
            printf("Enter Only Positive Number \n");
            continue;
        }
        sum=sum+n;
        i++;
    }
    avg=sum/10.0;
    printf("Sum : %d\nAvg : %f\n",sum,avg);
}
```

Program: Write a program to find the sum of digits of number until the sum is reduce to 1 digit.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int num,dig,sum=0;
```

```
    printf("Enter The Number : ");
```

```
    scanf("%d",&num);
```

```
    do
```

```
    {
```

```
        for(sum=0;num!=0;num/=10)
```

```
        {
```

```
            dig=num%10;
```

```
            sum=sum+dig;
```

```
        }
```

```
        printf("%d\t",sum);
```

```
        num=sum;
```

```
    }while(num/10!=0);
```

```
    printf("\n");
```

```
}
```

Program: Write a program to print the number from 1 to 10 using while loop.

```
#include<stdio.h>

int main()
{
    int i=1;
    while(i<=10)
    {
        printf("%d\t",i);
        i=i+1;
    }
    printf("\n");
}
```

Program: Write a program to check weather a year is leap or not.

```
#include<stdio.h>

int main()
{
    int year;
    printf("Enter The Year : ");
    scanf("%d",&year);
    if(year%4==0&&year%100!=0||year%400==0)
        printf("Lear Year\n");
    else
        printf("Not Leap Year\n");
}
```

Program: Write a program to print the sum of digits using recursive method.

```
#include<stdio.h>
```

```
int sumDigits(int);
```

```
int main()
```

```
{
```

```
    int data;
```

```
    printf("Enter The Data : ");
```

```
    scanf("%d",&data);
```

```
    printf("Sum of Digit in %d is %d\n",data,sumDigits(data));
```

```
}
```

```
int sumDigits(int data)
```

```
{
```

```
    if(data/10==0)
```

```
        return data;
```

```
    return (data%10+sumDigits(data/10));
```

```
}
```

Program: Write a program to reverse the number using recursive function.

```
#include<stdio.h>
```

```
int revDigits(int,int);
```

```
int digData(int);
```

```
int pwr(int);
```

```
int main()
```

```
{
```

```
    int data;
```

```
    printf("Enter The Data : ");
```

```
    scanf("%d",&data);
```

```
    printf("Reverse Data : %d\n",revDigits(data,digData(data)));
```

```
}
```

```
int revDigits(int data,int n)
```

```
{
```

```
    if(data)
```

```
        return (pwr(n)*(data%10)+revDigits(data/10,--n));
```

```
}
```

```
int pwr(int n)
```

```
{
```

```
    if(n>1)
```

```
        return (10*pwr(--n));
```

```
}
```

Program: Write a program to find the highest data in 5 integers.

```
#include<stdio.h>

int main()
{
    int arr[5];
    int i;
    int max,size;
    size=sizeof(arr)/sizeof(arr[0]);
    for(i=0;i<size;i++)
    {
        printf("Enter The Data %d : ",i);
        scanf("%d",&arr[i]);
    }
    max=arr[0];
    for(i=1;i<size;i++)
        if(arr[i]>max);
    printf("Max Number is : %d\n",max);
}
```


Program: Write a program to input 10 integers and find the index of highest and lowest data.

```
#include<stdio.h>

int main()
{
    int arr[10];
    int i,n,max,min,max_index=0,min_index=0;
    n=sizeof(arr)/sizeof(arr[0]);
    printf("Enter The 10 Integers : ");
    for(i=0;i<n;i++)
        scanf("%d",&arr[i]);
    max=min=arr[0];
    for(i=0;i<n;i++)
    {
        if(arr[i]>max)
        {
            max=arr[i];
            max_index=i;
        }
        if(arr[i]<min)
        {
            min=arr[i];
            min_index=i;
        }
    }
    printf("Max Number : %d at Index : %d\n",max,max_index);
    printf("Min Number : %d at Index : %d\n",min,min_index);
}
```

Program: Write a program to initialize an array of 10 characters. Count numbers of alphabets, characters and special characters.

```
#include<stdio.h>

int main()
{
    char arr[10]={'a','A','1','&','r','R','%','6','q','7'};
    int i,n;
    int alpha=0,num=0,special=0;
    n=sizeof(arr)/sizeof(arr[0]);
    for(i=0;i<n;i++);
    printf("%c\n",arr[i]);
    for(i=0;i<n;i++)
    {
        if(((arr[i]>='a')&&(arr[i]<='z'))||((arr[i]>='A')&&(arr[i]<='Z'))))
            alpha++;
        if((arr[i]>='0')&&(arr[i]<='9'))
            num++;
        else
            special++;
    }
    printf("Alphabates : %d\n",alpha);
    printf("Numbers   : %d\n",num);
    printf("Special Characters : %d\n",special);
}
```

Program: Write a program to use of srand() function.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int arr[10];
```

```
    int i;
```

```
    srand(getpid());
```

```
    for(i=0;i<10;i++)
```

```
        arr[i]=rand()%101;
```

```
    for(i=0;i<10;i++)
```

```
        printf("%d,",arr[i]);
```

```
    printf("\n");
```

```
}
```

Program: Write a program to find higher number from array and all its indexes.

```
#include<stdio.h>

int main()
{
    int arr[10],i,max=0;
    srand(getpid());
    for(i=0;i<10;i++)
    {
        arr[i]=rand()%101;
        printf("%d ",arr[i]);
        if(arr[i]>max)
            max=arr[i];
    }
    printf("\nMax : %d\n",max);
    printf("Indexes are : ");
    for(i=0;i<10;i++)
        if(arr[i]==max)
            printf("%d",i);
    printf("\n");
}
```

Program: Write a program to input 25 characters and count numbers of vowels.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int i,cnt=0;
```

```
    char arr[25];
```

```
    srand(getpid());
```

```
    for(i=0;i<25;i++)
```

```
    {
```

```
        arr[i]=rand()%26+97;
```

```
        printf("%c ",arr[i]);
```

```
        if((arr[i]=='a')||(arr[i]=='e')||(arr[i]=='i')||(arr[i]=='o')||(arr[i]=='u'))
```

```
            cnt++;
```

```
    }
```

```
    printf("\nNumbers of vowels : %d\n",cnt);
```

```
}
```

Program: Write a program to input 10 integers in array and find minimum and maximum number and all its indexes.

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
void input(int*,int);
void print(int*,int);
void find_max_min(int*,int);
void find_indexes(int*,int,int);
int main()
{
    srand(getpid());
    int arr[10]={0},n;
    n=sizeof(arr)/sizeof(arr[0]);
    input(arr,n);
    print(arr,n);
    find_max_min(arr,n);
}
void input(int arr[],int n)
{
    int i;
    for(i=0;i<n;i++)
        arr[i]=rand()%100;
}
```

void print(int arr[],int n)

```
{  
    int i;  
    for(i=0;i<n;i++)  
        printf("%d ",arr[i]);  
}
```

void find_max_min(int arr[],int n)

```
{  
    int i,max=arr[0],min=arr[0];  
    for(i=0;i<n;i++)  
    {  
        if(arr[i]>max)  
            max=arr[i];  
        if(arr[i]<min)  
            min=arr[i];  
    }  
    printf("\nMax : %d\n",max);  
    printf("Index : ");  
    find_indexes(arr,n,max);  
    printf("\nMin : %d\n",min);  
    printf("Index : ");  
    find_indexes(arr,n,min);  
    printf("\n");  
}
```

```
void find_indexes(int arr[],int n,int num)
```

```
{  
    int i;  
    for(i=0;i<n;i++)  
        if(arr[i]==num);  
    printf("%d ",i);  
}
```

Program: Write a program to assign 10 unique random alphabets to array

```
#include<stdio.h>  
#include<unistd.h>  
#include<stdlib.h>  
int test_unique(char*,int);  
int main()  
{  
    srand(getpid());  
    char str[10];  
    int i;  
    for(i=0;i<10;i++)  
    {  
        str[i]=rand()%26+65;  
        if(!test_unique(str,i))  
            i--;  
    }  
    for(i=0;i<10;i++)  
        printf("%c ",str[i]);  
}
```



```

int test_unique(char *p,int i)
{
    int j;
    for(j=0;j<i;j++)
    {
        if(p[i]==p[j])
            return 0;
    }
    return 1;
}

```

Program: Write a program to get random 25 number of any character data and count number of alphabets, numeric data and special characters.

```

#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
int input_data(char*,int);
void print_data(char*,int);
void cnt_data(char*,int);
int main()
{
    srand(getpid());
    char arr[25];
    input_data(arr,25);
    print_data(arr,25);
    cnt_data(arr,25);
}

```

```
int input_data(char arr[],int n)
```

```
{  
    int i;  
    for(i=0;i<n;i++)  
        arr[i]=rand()%128;  
}
```

```
void print_data(char arr[],int n)
```

```
{  
    int i;  
    for(i=0;i<n;i++)  
        printf("arr[%d] : %c\n",i,arr[i]);  
}
```

Program: User define strlen_finction.

```
size_t Ustrlen(const char *p)
```

```
{  
    size_t count=0;  
    int i;  
    for(i=0;p[i];i++)  
        count++;  
    return count;  
}
```

Program: Write a program to implement your own strstr() function.

```
#include<stdio.h>

char *user_strstr(char*,char*);

int main()
{
    char str_1[100];
    char str_2[100];
    char *ptr;
    printf("Enter The String - 1 : ");
    scanf("%s",str_1);
    printf("Enter The String - 2 : ");
    scanf("%s",str_2);
    ptr=user_strstr(str_1,str_2);
    if(ptr)
    {
        printf("Found at Index : %ld\n",ptr-str_1);
    }
    else
        printf("Not Found\n");
}
```

```
char *user_strrstr(char str_1[],char str_2[])
```

```
{
```

```
    int i,j;
```

```
    char *ptr=NULL;
```

```
    for(i=0;str_1[i];i++,j=1)
```

```
    {
```

```
        if(str_1[i]==str_2[0])
```

```
        {
```

```
            for(j=1;str_2[j];j++)
```

```
            {
```

```
                if(str_1[i+j]!=str_2[j])
```

```
                    break;
```

```
            }
```

```
            if(str_2[j]=='\0')
```

```
                ptr=str_1+i;
```

```
        }
```

```
    }
```

```
    if(ptr)
```

```
        return ptr;
```

```
    else
```

```
        return NULL;
```

```
}
```

Program: Write program to reverse the entered string.

```
#include<stdio.h>
```

```
#include<string.h>
```

```
void reverse_string(char*);
```

```
int main()
```

```
{
```

```
    char str[100];
```

```
    printf("Enter The String : ");
```

```
    scanf("%s",str);
```

```
    reverse_string(str);
```

```
    printf("Reverse String : %s\n",str);
```

```
}
```

```
void reverse_string(char *ptr)
```

```
{
```

```
    int n=strlen(ptr);
```

```
    char *q,temp;
```

```
    for(q=ptr+n-1;ptr<q;ptr++,q--)
```

```
    {
```

```
        temp=*ptr;
```

```
        *ptr=*q;
```

```
        *q=temp;
```

```
    }
```

```
}
```

ii)

```
#include<stdio.h>

int main()
{
    char str[100];;
    char ch;
    int i;
    int cnt = 0;
    printf("Enter The String : ");
    gets(str);
    for(i=0;str[i];i++)
        cnt=i;
    printf("String Count is : %d\n",cnt);
    for(i=0;i<=cnt/2;i++)
    {
        ch=str[i];
        str[i]=str[cnt-i-1];
        str[cnt-i-1]=ch;
    }
    printf("Reverse String : %s\n",str);
}
```

Program: Write a program to compare two strings.

```
#include<stdio.h>
#include<string.h>
int main()
{
    char str_1[100];
    char str_2[100];
    printf("Enter The String - 1 : ");
    scanf("%s",str_1);
    printf("Enter The String - 2 : ");
    scanf("%s",str_2);
    if(strcmp(str_1,str_2)==0)
        printf("Equal\n");
    else
        printf("Not Equal\n");
}
```

Program: Write user define memset function.

```
void memset(void *str,int c,size_t n)
{
    while(n--)
    {
        *(char *)s=c;
        ((char *)s)++;
    }
    return s;
}
```

Program: Write user define strcmp function.

```
int user_strcpy(const char *str_1,const char *str_2)
{
    while((*str_1)&&(str_2))
    {
        if(*str_1!=*str_2)
            break;
        str_1++;
        str_2++;
    }
    if(*str_1==*str_1)
        return 0;
    else
    {
        if(*str_1>*str_2)
            return 1;
        else
            return -1;
    }
}
```


Program: Write a program to hide string 1 in string 2.

```
#include<stdio.h>
#include<string.h>
int main()
{
    char str_1[100];
    char str_2[100];
    char *ptr;
    printf("Enter The String - 1 : ");
    scanf("%[^\n]s",str_1);
    printf("Enter The String - 2 : ");
    scanf("%s",str_2);
    ptr=str_1;
    while(ptr=strstr(ptr,str_2))
    {
        memset(ptr,'*',strlen(str_2));
        ptr++;
    }
    printf("Str_1 : %s\n",str_1);
}
```

```
#include<stdio.h>
#include<string.h>
int main()
{
    char str_1[100];
    char str_2[100];
    char *ptr;
    int lenght;
    printf("Enter The String - 1 : ");
    scanf("%[^\n]s",str_1);
    printf("Enter The String - 2 : ");
    scanf("%s",str_2);
    while(ptr=strstr(str_1,str_2))
    {
        lenght=strlen(str_2);
        for(int i=0;i<lenght;i++)
            str_1[(ptr-str_1)+i]='*';
    }
    printf("String is : %s\n",str_1);
}
```

Program: Write a program to remove extra consecutive blank spaces.

```
#include<stdio.h>
#include<string.h>
int main()
{
    char str_1[100];
    char *ptr,*qtr;
    char ch=32;
    printf("Enter The String : ");
    scanf("%^[^\n]s",str_1);
    ptr=str_1;
    while(ptr=strchr(ptr,ch))
    {
        qtr=++ptr;
        while(*ptr==ch)
            ptr++;
        memmove(qtr,ptr,strlen(ptr)+1);
    }
    printf("String : %s\n",str_1);
}
```

Program: Write a program to reverse string 2 In string 1.

```
#include<stdio.h>
#include<string.h>
int main()
{
    char str_1[100];
    char str_2[100];
    char *ptr;
    char ch;
    int i,length;
    printf("Enter The String - 1 : ");
    gets(str_1);
    printf("Enter The String - 2 : ");
    gets(str_2);
    while(ptr=strstr(str_1,str_2))
    {
        length=strlen(str_2)-1;
        for(i=0;i<length;i++,length--)
        {
            ch=*(ptr+1);
            *(ptr+i)=*(ptr+length);
            *(ptr+length)=ch;
        }
    }
    printf("String is : %s\n",str_1);
}
```

Program: Write a program to reverse string 2 in string 1.

```
#include<stdio.h>
#include<string.h>
int main()
{
    char str_1[100];
    char str_2[100];
    char *ptr,ch;
    int i,j;
    printf("Enter The String - 1 : ");
    gets(str_1);
    printf("Enter The String - 2 : ");
    gets(str_2);
    while(ptr=strstr(str_1,str_2))
    {
        j=strlen(str_2)-1;
        for(i=0;i<j;i++,j--)
        {
            ch=*(ptr+i);
            *(ptr+i)=*(ptr+j);
            *(ptr+j)=ch;
        }
    }
    printf("String - 1 : %s\n",str_1);
}
```

Program: Write a program to remove occurrence of string 2 from string 1.

```
#include<string.h>
#include<stdio.h>
int main()
{
    char str_1[100];
    char str_2[100];
    char *ptr;
    printf("Enter The String - 1 : ");
    gets(str_1);
    printf("Enter The String - 2 : ");
    gets(str_2);
    while(ptr=strstr(str_1,str_2))
    {
        memmove(ptr,ptr+strlen(str_2),strlen(ptr+strlen(str_2))+1);
        printf("String is : %s\n",str_1);
    }
}
```

Program: User define strstr function.

```
char user_strstr(const char *str_1,const char *str_2)
{
    int i,j;
    for(i=0;str_1[i];i++)
    {
        if(str_1[i]==str_2[0])
        {
            for(j=1;str_2[j];j++)
            {
                if(str_1[i+j]!=str_2[j])
                    break;
                if(str_2[j]=='\0')
                    return str_1+i;
            }
        }
    }
    return NULL;
}
```

```
#include<stdio.h>
#include<string.h>
int main()
{
    char str_1[100];
    char str_2[100];
    char *ptr;
    printf("Enter The String - 1 : ");
    gets(str_1);
    printf("Enter The String - 2 : ");
    gets(str_2);
    int i;
    ptr=strstr(str_1,str_2);
    if(ptr)
        printf("Found at : %u\n",ptr-str_1);
    else
        printf("Not Found\n");
}
```



```
#include<stdio.h>
#include<string.h>
int main()
{
    char str_1[100];
    char str_2[100];
    char *ptr;
    int i;
    int cnt=0;
    printf("Enter The String - 1 : ");
    gets(str_1);
    printf("Enter The String - 2 : ");
    gets(str_2);
    ptr=str_1;
    while(ptr=strstr(ptr,str_2))
    {
        cnt++;
        ptr++;
    }
    printf("String 2 Occurs %d time in String 1\n",cnt);
}
```

Program: Write a program to understand the work of strlen() function.

```
#include<stdio.h>
#include<string.h>
int main()
{
    char str[20];
    int length;
    printf("Enter The String : ");
    scanf("%[^\\n]s",str);
    length=strlen(str);
    printf("Length of String is : %d\\n",length);
}
```

Program: Write the user defined strlen() function.

```
int user_strlen(char str[])
{
    int i=0;
    while(str[i]!='\\0')
        i++;
    return i;
}
```

Program: Write a program to input a char and search the occurrences.

```
#include<stdio.h>
#include<string.h>
int main()
{
    char str[100];
    char ch;
    int i,cnt=0;
    printf("Enter The String : ");
    gets(str);
    printf("Enter The Charcater to be search : ");
    scanf("%c",&ch);
    for(i=0;str[i];i++)
    {
        if(str[i]==ch)
            cnt++;
    }
    printf("Character %c occurs %d count\n",ch,cnt);
}
```

Program: Write a program to understand strcat() function.

```
#include<stdio.h>
#include<string.h>
int main()
{
    char str_1[20];
    char str_2[20];
    printf("Enter The String - 1 : ");
    scanf("%s",str_1);
    printf("Enter The String - 2 : ");
    scanf("%s",str_2);
    strcat(str_1,str_2);
    printf("Strcat Result : %s\n",str_1);
}
```

ii)

```
#include<stdio.h>
#include<string.h>
int main()
{
    char str_1[20]="Subhash";
    char str_2[20]="Chandra";
    strcat(strcat(str_1,str_2),"Bose");
    printf("Result : %s\n",str_1);
}
```

Program: Write a program to convert all lowercase character to uppercase.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    char str[100];
```

```
    int i;
```

```
    printf("Enter The String : ");
```

```
    gets(str);
```

```
    for(i=0;str[i];i++)
```

```
    {
```

```
        if((str[i]>='a')&&(str[i]<='z'))
```

```
            str[i]-=32;
```

```
    }
```

```
    printf("String : %s\n",str);
```

```
}
```

Program: Write a program to copying one string into another, using member by member.

```
#include<stdio.h>

int main()
{
    char s1[100];
    char s2[100];
    int i;
    printf("Enter String For S1 : ");
    gets(s1);
    for(i=0;s1[i];i++)

        s2[i]=s1[i];
        s2[i]=0;
        printf("S2 : %s\n",s2);

}
```

Program: Write a program to copy string using built in function strcpy.

```
#include<stdio.h>
#include<string.h>
int main()
{
    char str_1[100];
    char str_2[100];
    printf("Enter The String - 1 : ");
    gets(str_1);
    strcpy(str_2,str_1);
    printf("String - 2 is : %s\n",str_2);
}
```

Program: Write a program for user defined string copy strcpy().

```
char *user_strcpy(char *dest,const char *src)
{
    int i;
    for(i=0;src[i];i++)
        dest[i]=src[i];
    dest[i]=0;
    return dest;
}
```

Program: Write user define strncpy() function.

```
char *user_strncpy(char *dest,const char *src,size_t n)
{
    int i;
    for(i=0;(i<n && src[i]);i++)
        dest[i]=src[i];
    for(;i<n;i++)
        dest[i]='\0';
    return dest;
}
```

Program: Write user define memmove() function.

```
void *memmove(void *dest,const void *src,size_t n)
{
    char temp[n+1];
    int i;
    for(i=0;i<n;i++)
        temp[i]=((char *)src)[i];
    for(i=0;i<n;i++)
        ((char *)dest)[i]=temp[i];
    return dest;
}
```


Program: Write user define strcat() function.

```
char *user_strcat(char *dest,const char *src)
{
    int len=strlen(dest);
    int i;
    for(i=0;src[i];i++)
        dest[len+i]=src[i];
    dest[len+i]='\0';
    return dest;
}
```

Program: Write user define strncat() function.

```
char *user_strncat(char *dest,const char *src,size_t n)
{
    int length=strlen(dest);
    int i;
    for(i=0;i<n && src[i];i++)
        dest[len+i]=src[i];
    dest[len+i]='\0';
    return dest;
}
```

Program: Write a program to input three string stores in str_1, str_2, str_3. Copy all three strings into string 4, one after another separated by space.

```
#include<stdio.h>

#include<string.h>

int main()
{
    char str_1[100];
    char str_2[100];
    char str_3[100];
    char str_4[100];
    printf("Enter String - 1 : ");
    gets(str_1);
    printf("Enter String - 2 : ");
    gets(str_2);
    printf("Enter String - 3 : ");
    gets(str_3);
    strcpy(str_4,str_1);
    strcat(str_4," ");
    strcat(str_4,str_2);
    strcat(str_4," ");
    strcat(str_4,str_3);
    printf("String - 4 : %s\n",str_4);
}
```

Program: Write a program to input string 1 and string 2. Insert string 2 in given position in string 1.

```
#include<stdio.h>

#include<string.h>

int main()
{
    char str_1[100];
    char str_2[100];
    char temp[100];
    int posn;
    printf("Enter The Stings : ");
    gets(str_1);
    printf("Enter The String : ");
    gets(str_2);
    printf("Enter The Posn : ");
    scanf("%d",&posn);
    strcpy(temp,str_1+posn);
    strcpy(str_1+posn,str_2);
    strcat(str_1,temp);
    printf("Modified Str - 1 : %s\n",str_1);
}
```

Program: Write a program to input string 1 and string 2. Insert string 2 in given position in string 1.

```
#include<stdio.h>

#include<string.h>

int main()
{
    char str_1[100];
    char str_2[100];
    int posn;
    printf("Enter The String - 1 : ");
    gets(str_1);
    printf("Enter The String - 2 : ");
    gets(str_2);
    printf("Enter The Posn : ");
    scanf("%d",&posn);
    memmove(str_1+posn+strlen(str_2),str_1+posn,strlen(str_1+posn)+1);
    memmove(str_1+posn,str_2,strlen(str_2));
    printf("Modified String : %s\n",str_1);
}
```

Program: Write a program to input a string and remove all non-alphabets.

```
#include<stdio.h>
#include<string.h>
int main()
{
    char str_1[100];
    char str_2[100];
    int i,len;
    printf("Enter The String - 1 : ");
    gets(str_1);
    for(i=0;str_1[i];i++)
    {

if(((str_1[i]>='a')&&(str_1[i]<='z'))||((str_1[i]>='A')&&(str_1[i]>='A')&&(str_1
[i]<='Z'))
        continue;
    else
    {
        memmove(str_1+i,str_1+i+1,strlen(str_1+i));
        i--;
    }
    }
    printf("Modified String : %s\n",str_1);
}
```

Program: Write a program to input a string and a string character and remove all occurrences of the character from the string.

```
#include<string.h>
#include<stdio.h>
int main()
{
    char str[100];
    int i;
    char ch;
    printf("Enter The String : ");
    gets(str);
    printf("Enter The Character : ");
    scanf("%s",&ch);
    for(i=0;str[i];i++)
    {
        if(str[i]==ch)
        {
            memmove(str+i,str+i+1,strlen(str+i+1)+1);
            i--;
        }
    }
    printf("Modified String : %s\n",str);
}
```

Program: Write a program to print the character position in entered string.

```
#include<stdio.h>
#include<string.h>
int main()
{
    char str[100];
    printf("Enter The String : ");
    gets(str);
    char ch;
    printf("Enter The Character : ");
    scanf("%c",&ch);
    if(strchr(str,ch)==NULL)
        printf("Not Found\n");
    else
        printf("Found at index : %u\n",strchr(str,ch)-str);
}
```

Program: Write a program to print the character position in entered string.

```
#include<stdio.h>
#include<string.h>
int main()
{
    char str[100];
    char ch;
    char *ptr;
    printf("Enter The String : ");
    gets(str);
    printf("Enter The Characater : ");
    scanf("%c",&ch);
    ptr=strchr(str,ch);
    if(ptr==NULL)
        printf("Not Found\n");
    else
        printf("Found at index : %u\n",ptr-str);
}
```


Program: Write user defined strchr() Function.

```
char *user_strchr(const char *str,int ch)
{
    int i;
    for(i=0;str[i];i++)
    {
        if(str[i]==ch)
            return str+i;
    }
    return NULL;
}
```

Program: Write a program to remove extra repetitions of all characters in a given string.

```
#include<stdio.h>
#include<string.h>
int main()
{
    char str[100];
    char *ptr;
    printf("Enter The String : ");
    gets(str);
    int i;
    for(i=0;str[i];i++)
    {
        while(ptr=strchr(str+i+1,str[i]))
            memmove(ptr,ptr+1,strlen(ptr+1)+1);
    }
    printf("Modified String : %s\n",str);
}
```

Program: Write a program to input a string and find out how many characters are repeated.

```
#include<stdio.h>
#include<string.h>
int main()
{
    char str[100];
    char *ptr;
    char temp[100];
    int cnt=0;
    printf("Enter The String : ");
    gets(str);
    strcpy(temp,str);
    for(int i=0;temp[i];i++)
    {
        if(strchr(temp+i+1,temp[i]))
        {
            cnt++;
            while(ptr=strchr(temp+i+1,temp[i]))
                memmove(ptr,ptr+1,strlen(ptr)+1);
        }
    }
    printf("No. of Characters Repeating : %d\n",cnt);
}
```

Program: Write user defined strrchr() Function.

```
char *user_strrchr(const char *str,int ch)
{
    int i;
    int len=-1;
    for(i=0;str[i];i++)
    {
        if(str[i]==ch)
            len=i;
    }
    if(len>=0)
        return str+1;
    else
        return NULL;
}
```

Program: Write a program to find the min and max numbers in an array.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int i=0,n,min;
```

```
    int arr[5];
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        puts("Enter the numbers: ");
```

```
        scanf("%d",&arr[i]);
```

```
    }
```

```
    puts("Elements of Array : ");
```

```
    for(int j=0;j<5;j++)
```

```
        printf("%d",arr[j]);
```

```
    n=arr[0];
```

```
    for(int j=1;j<5;j++)
```

```
    {
```

```
        if(n<arr[j])
```

```
            n = arr[j];
```

```
    }
```

```
    min=arr[0];
```

```
    int temp;
```

```
    for(int j=1;j<5;j++)
```

```
    {
```

```
        if(min>arr[j])
```

```
            min = arr[j];
```

```
    }
```

```
printf("Largest no is: %d\n",n);  
printf("Smallest No is: %d\n",min);  
}
```

Program: Write a program to insert insertion sort.

```
#include<stdio.h>

void insert_sort(int *p);

int main()
{
    int i=0,n;
    int arr[5];
    for(i=0;i<5;i++)
    {
        puts("Enter the numbers:");
        scanf("%d",&arr[i]);
    }
    puts("Before Sort");
    for(int j=0;j<5;j++)
        printf("%d",arr[j]);
    insert_sort(arr);
}

void insert_sort(int *p)
{
    int temp;
    int j;
    for(int i=1;i<5;i++)
    {
        temp=p[i];
        j=i-1;
        while(j>=0)
        {
```

```
        if(temp<p[j])
        {
            p[j+1]=p[j];
            j--;
        }
        else
            break;
    }
    p[j+1]=temp;
}
puts("After Soting:");
for(int x=0;x<5;x++)
    printf("%d, ",p[x]);
printf("\n");
}
```


Program: Write a program to find the highest value from an array using 1 loop.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int i=0,n;
```

```
    int arr[5];
```

```
    for(i=0;i<5;i++) //This loop is only taken for getting array assigned if we  
    don't want this for loop we can use arr[]={5,7,9,5,1};
```

```
    {
```

```
        printf("Enter The Numbers:");
```

```
        scanf("%d",&arr[i]);
```

```
    }
```

```
    n=arr[0];
```

```
    for(int j=1;j<5;j++)
```

```
    {
```

```
        if(n<arr[j])
```

```
            n = arr[j];
```

```
    }
```

```
    printf("Largest No is: %d\n",n);
```

```
}
```

Program: Write a Program to print binary equivalent of float using union.

```
#include<stdio.h>

union t
{
    float f;
    struct st
    {
        unsigned int mentisa:23; //Bit field of 23 bit for mantissa part.
        unsigned int exponent:8; //Bit Field of 8 bit for exponent part.
        unsigned int sign:1; //Bit filed of 1 bit for signed part
    }row;
}u;

int main()
{
    union t v;
    puts("Enter Floating Real Value:");
    scanf("%f",&v.f); //Getting Real Value Here
    printf("Binary Form is: ");
    printf("%d ",v.row.sign); //1st Printing Signed bit of Real value
    /* For Exponent part */
    for(int i=8;i>=0;i--) //loop for printing binary value till 8 bit
    {
        if((v.row.exponent >> i) & 1) //if exponent of real value (in binary)
is true then print 1
            printf("1");
        else
            printf("0");
    }
}
```

```
/* For Mentisa Part */  
for(int i=23;i>=0;i--)  
{  
    if((v.row.mentisa >> i) & 1)  
        printf("1");  
    else  
        printf("0");  
    if(i%8==0)  
    {  
        printf(" ");  
    }  
}  
printf("\n");  
}
```

Program: To find GCD of two numbers.

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    int a, b, small, i;
    a = atoi(argv[1]);
    b = atoi(argv[2]);
    if (a > b)
        small = b;
    else
        small = a;
    for (i = small; i >= 1; i--)
    {
        if ((a % i) == 0 && (b % i) == 0)
        {
            printf("%d", i);
            break;
        }
    }
    return 0;
}
```

Program: Write a program to find the lcm of two numbers.

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    int a, b, large;
    a = atoi(argv[1]);
    b = atoi(argv[2]);
    if (a > b)
        large = a;
    else
        large = b;
    while (1)
    {
        if ((large % a) == 0 && (large % b) == 0)
        {
            printf("%d", large);
            break;
        }
        large++;
    }
    return 0;
}
```

Program. Write a program to find the area of a circle ($\text{area}=3.14*r*r$), when diameter is given.

```
#include <stdio.h>

#define PI 3.14

int main(int argc,char *argv[])
{
    float dia,radius,area=0;
    dia=atoi(argv[1]);
    radius=0.5*dia;
    area=PI*radius*radius;
    printf("%.2f",area);
    return 0;
}
```

Program: Write a program to check whether given number is strong number or not.

```
#include<stdio.h>
```

```
int fact_data(int);
```

```
int main(int argc, char *argv[])
```

```
{  
    int num,d,n,res=0,i,count=0,x;  
    n=atoi(argv[1]);  
    num=n;  
    x=num;  
    while(n!=0)  
    {  
        n=n/10;  
        count++;  
    }  
    for(i=0;i<count;i++)  
    {  
        if(x>0)  
        {  
            d=x%10;  
            res=res+fact_data(d);  
            x=x/10;  
        }  
    }  
    if(res==num)  
    {  
        printf("Strong Number\n");  
    }  
}
```

```
        else printf("Not Strong Number\n");  
        return 0;  
    }
```

```
int fact_data(int x)  
{  
    if(x==0)  
        return 1;  
    else  
        return x*fact_data(x-1);  
}
```


Program: Write a program to print the CPU is little endian or big endian.

```
#include<stdio.h>
```

```
union tag
```

```
{
```

```
    int i;
```

```
    char ch;
```

```
};
```

```
int main()
```

```
{
```

```
    union tag var={1};
```

```
    if(var.ch)
```

```
        printf("CPU is byte Order is Little Endian\n");
```

```
    else
```

```
        printf("CPU is byte Order is Big Endian\n");
```

```
}
```

Program: Write a program to find the 2nd highest digit in a supplied integer

Note: Use a single loop.

```
#include<stdio.h>
```

```
int main ()
```

```
{
```

```
    int data,rem,largest=0,Sec_large=0;
```

```
    printf("Enter the Number :");
```

```
    scanf("%d",&data);
```

```
    while(data>0)
```

```
    {
```

```
        rem=data%10;
```

```
        if(largest<rem)
```

```
        {
```

```
            Sec_large=largest;
```

```
            largest=rem;
```

```
        }
```

```
    else if(rem>=Sec_large)
```

```
        Sec_large=rem;
```

```
    data=data/10;
```

```
    }
```

```
    printf("The Second Largest Digit is %d\n", Sec_large);
```

```
    return 0;
```

```
}
```

Program: Write a Program to calculate sum-of-Odd-digits in an integer.

```
#include<stdio.h>
```

```
int sumofodddigits(int);
```

```
int main()
```

```
{
```

```
    int data;
```

```
    printf("Enter a Number: ");
```

```
    scanf("%d",&data);
```

```
    printf("Sum of Odd Digits: %d\n",sumofodddigits(data));
```

```
}
```

```
int sumofodddigits(int data)
```

```
{
```

```
    int r,sum=0;
```

```
    while(data>0)
```

```
    {
```

```
        r=data%10;
```

```
        data=data/10;
```

```
        if (r%2==1)
```

```
            sum=sum+r;
```

```
    }
```

```
        return sum;
```

```
}
```

Program: Write a program to sort the array of strings.

```
#include<stdio.h>
#include<string.h>
int main()
{
    char arr[5][10]={"White","Red","Green","Yellow","Blue"};
    char temp[10];
    int i;
    printf("Before Sorting : ");
    printf("\n");
    for(i=0;i<5;i++)
        printf("%s\t",arr[i]);
    printf("\n");
    for(i=0;i<5;i++)
        for(int j=i+1;j<5;j++)
            if(strcmp(arr[i],arr[j])>0)
            {
                strcpy(temp,arr[i]);
                strcpy(arr[i],arr[j]);
                strcpy(arr[j],temp);
            }
    printf("After Sorting : \n");
    for(i=0;i<5;i++)
        printf("%s\t",arr[i]);
}
```

Program: Write a program to add the elements of the array.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int arr[10];
```

```
    int i;
```

```
    int sum=0;
```

```
    for(i=0;i<10;i++)
```

```
    {
```

```
        printf("Enter The Value for arr[%d] : ",i);
```

```
        scanf("%d",&arr[i]);
```

```
        sum+=arr[i];
```

```
    }
```

```
    printf("Sum of array is : %d\n",sum);
```

```
}
```

Program: Write a program to find the maximum and minimum numbers in an array.

```
#include<stdio.h>

int main()
{
    int i,j,arr[10]={2,4,5,2,1,8,9,11,6,3};
    int min,max;
    min=max=arr[0];
    for(i=0;i<10;i++)
    {
        if(arr[i]<min)
            min=arr[i];
        if(arr[i]>max)
            max=arr[i];
    }
    printf("Min : %d\n",min);
    printf("Max : %d\n",max);
}
```

Program: Write a program to reverse the element of array.

```
#include<stdio.h>

int main()
{
    int i,j,temp;
    int arr[10]={1,2,3,4,5,6,7,8,9,10};
    for(i=0,j=9;i<j;i++,j--)
    {
        temp=arr[i];
        arr[i]=arr[j];
        arr[j]=temp;
    }
    printf("Revese Array : ");
    for(i=0;i<10;i++)
        printf("%d,",arr[i]);
    printf("\n");
}
```

Program: Write a program to open and close the file.

```
#include<stdio.h>

int main()
{
    FILE *fptr;
    char file_name[]="file.txt";
    if((fptr=fopen(file_name,"r"))==NULL)
    {
        printf("Cannot Open The File :%s\n",file_name);
    }
    else
    {
        printf("File Open SucessFully\n");
        printf("Ready To Close The File\n");
        fclose(fptr);
    }
}
```


Program: Write a program to Read and Write one character at a time.

```
#include<stdio.h>

int main()
{
    FILE *fptr;
    char ch;

    /*Writing text into file*/
    fptr=fopen("text.txt","w");
    printf("Enter The Text : ");
    /*Print Ctrl+d to stop Reading Character*/
    while((ch=getchar())!=EOF)
        fputs(ch,fptr);
    fclose(fptr);

    /*Reading text from file and printing on to screen*/
    if((fptr=fopen("text.txt","r"))==NULL)
        printf("This FILE doesn't exist\n");
    else
    {
        printf("Data in file : ");
        while((ch=fgetc(fptr))!=EOF)
            printf("%c",ch);
    }
    fclose(fptr);
}
```

Program: Write a program to read and write one line a time.

```
#include<stdio.h>

int main()
{
    FILE *fptr;
    char wbuff[80];
    char rbuff[80];

    /*Writing into the file*/
    fptr=fopen("file.txt","w");
    printf("Enter The Text : ");

    /*Press Ctrl+d to stop reading character*/
    while(gets(wbuff)!=NULL)
        fputs(wbuff,fptr);
    fclose(fptr);

    /*Reading text from file and printing on to screen*/
    if((fptr=fopen("file.txt","r"))==NULL)
        printf("This File Dosen't Exist\n");
    else
    {
        printf("Data in the File : \n");
        while(fgets(rbuff,80,fptr)!=NULL)
            printf("%s",rbuff);
    }
    fclose(fptr);
}
```

Program: Write a program to understand the use of fprintf() and fscanf().

```
#include<stdio.h>

struct student
{
    char name[20];
    float percentage;
}stu1,stu2;

int main()
{
    FILE *fptr;
    int i,n;
    /*Writing The Data In The File using fscanf()*/
    fptr=fopen("student_data.dat","w");
    printf("Enter Number Of Records : ");
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
        printf("Enter Name : ");
        scanf("%s",stu1.name);
        printf("Enter The Percentage : ");
        scanf("%f",&(stu1.percentage));
        fprintf(fptr,"%s%f",stu1.name,stu1.percentage);
    }
    fclose(fptr);
}
```

```
/*Reading The Data From File Using fscanf()*/  
fptr=fopen("student.dat","r");  
printf("Name\nPercentage\n");  
while(fscanf(fptr,"%s%f",stu2.name,&(stu2.percentage))!=EOF)  
    printf("%s\t%f\n",stu2.name,stu2.percentage);  
fclose(fptr);  
}
```

Program: Write a program to read and write one block of character at a time.

```
#include<stdio.h>

struct student
{
    char name[20];
    int roll;
    float percentage;
}stu1,stu2;

int main()
{
    FILE *fptr;
    int i,n;
    /*Writing Data into File Using Fwrite*/
    fptr=fopen("student.dat","wb");
    printf("Enter Numbers of Record : ");
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
        printf("Enter The Name : ");
        scanf("%s",stu1.name);
        printf("Enter The Roll Number : ");
        scanf("%d",&(stu1.roll));
        printf("Enter The Percentage : ");
        scanf("%f",&(stu1.percentage));
        fwrite(&stu1,sizeof(stu1),1,fptr);
    }
    fclose(fptr);
}
```

```
/*Reading Data From File Using fread()*/
    fptr=fopen("student.dat","rb");
    printf("Name:\tRoll Number:\tPercentage:\n");
    while(fread(&stu2,sizeof(stu2),1,fptr)==1)
    {
        printf("%s\t",stu2.name);
        printf("%d\t",stu2.roll);
        printf("%f\t",stu2.percentage);
    }
    fclose(fptr);
}
```

Program: Write a program to understand the fseek() function

```
#include<stdio.h>
#include<stdlib.h>
struct student
{
    char name[20];
    int roll;
    float percentage;
}stu1;

int main()
{
    FILE *fptr;
    int n;

    fptr=fopen("student.dat","rb");
    if(fptr==NULL)
    {
        printf("Error in opening file\n");
        exit(1);
    }
    printf("Enter The Record Number to be read : ");
    scanf("%d",&n);
    fseek(fptr,(n-1)*sizeof(stu1),0);    //Skip n-1 record
    fread(&stu1,sizeof(stu1),1,fptr);    //Read the nth Record
    printf("Name\tRollNo\tPercentage\n");
    printf("%s\t",stu1.name);
```

```
printf("%d\t",stu1.roll);  
printf("%f\t",stu1.percentage);  
fclose(fptr);  
}
```


Program: Write a program to understand the use of ftell().

```
#include<stdio.h>
#include<stdlib.h>
struct student
{
    char name[20];
    int roll;
    float percentages;
}stu1;
int main()
{
    FILE *fptr;
    fptr=fopen("student.dat","rb");
    if(fptr==NULL)
    {
        printf("Error in opening File\n");
        exit(1);
    }
    printf("Position pointer in the begning : %ld\n",ftell(fptr));
    while(fread(&stu1,sizeof(stu1),1,fptr)==1)
    {
        printf("Position Pointer : %ld\n",ftell(fptr));
        printf("%s\t",stu1.name);
        printf("%d\t",stu1.roll);
        printf("%f\t",stu1.percentages);
    }
```

```
printf("Size of File in Bytes is : %ld\n",ftell(fptr));  
fclose(fptr);  
}
```

Program: Write a program to understand the use of rewind().

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    FILE *fptr;
    fptr=fopen("student.dat","rb");
    if(fptr==NULL)
    {
        printf("Error in opening File\n");
        exit(1);
    }
    printf("Position Pointer : %ld\n",ftell(fptr));
    fseek(fptr,0,2);
    printf("posotion Pointer : %ld\n",ftell(fptr));
    rewind(fptr);
    printf("Position Pointer : %ld\n",ftell(fptr));
    fclose(fptr);
}
```

Program: Write a program to Open and Close a text file.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    FILE *fptr;
```

```
    char file_name[]="file.txt";
```

```
    if((fptr=fopen(file_name,"r"))==NULL)
```

```
    {
```

```
        printf("Cannot Open : %s\n",file_name);
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("FILE Open Sucessfully\n");
```

```
        printf("Ready To Close The FILE\n");
```

```
        fclose(fptr);
```

```
    }
```

```
}
```

Program: Write a program to read and write one character at a time

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    FILE *fptr;
```

```
    char ch;
```

```
    /*Writing Text into File*/
```

```
    fptr=fopen("text.txt","w");
```

```
    printf("Enter The Text : ");
```

```
    /* Press ctrl+d to stop reading character*/
```

```
    while((ch=getchar())!=EOF)
```

```
        fputc(ch,fptr);
```

```
    fclose(fptr);
```

```
    /* Reading text from file and printing on to the screen*/
```

```
    if((fptr=fopen("text.txt","r"))==NULL)
```

```
        printf("This FILE dosen't exit\n");
```

```
    else
```

```
    {
```

```
        printf("Data in FILE is : \n");
```

```
        while((ch=fgetc(fptr))!=EOF)
```

```
            printf("%c",ch);
```

```
    }
```

```
    fclose(fptr);
```

```
}
```

Program: Write a program to print sum of first 10 perfect number from given starting number.

```
#include<stdio.h>

int main()
{
    unsigned long long number;
    printf("Enter any Number : ");
    scanf("%llu",&number);
    printf("The Perfect Numbers are : ");
    int total=0;
    while(total<10)
    {
        unsigned long long sum=1;
        unsigned long long divisor=2;
        while(divisor<=number/2)
        {
            if(number%divisor==0)
            {
                sum+=divisor;
            }
            divisor++;
        }
        if(sum==number)
        {
            printf("%llu\n",number);
            total++;
        }
        number+=1;
    }
}
```

}

}

Program: Write a program to print the binary of floating-point number.

```
#include<stdio.h>
```

```
union data
```

```
{
```

```
    int i;
```

```
    float f;
```

```
};
```

```
int main()
```

```
{
```

```
    union data v;
```

```
    int bit;
```

```
    printf("Enter The data : ");
```

```
    scanf("%f",&v.f);
```

```
    printf("The Binary is : ");
```

```
    for(bit=31;bit>=0;bit--)
```

```
    {
```

```
        printf("%d",(v.i>>bit)&1);
```

```
    }
```

```
    printf("\n");
```

```
}
```


Program: Write a function to reverse double linked list.

```
#include<header.h>
```

```
void reverse_display(struct person *ptr)
```

```
{
```

```
    if(ptr==NULL)
```

```
    {
```

```
        printf("List is Empty\n");
```

```
    }
```

```
    else
```

```
    {
```

```
        for(;ptr->link;ptr=ptr->link); //reach to the last node
```

```
        while(ptr) // it iterates from last node to initial value of head which is  
fully null
```

```
        {
```

```
            printf("Age is : %d\n",ptr->age);
```

```
            printf("Name is : %s",ptr->name);
```

```
            ptr=ptr->prev; // after printing 1st node goto last before node
```

```
        }
```

```
    }
```

```
}
```

Program: Write a program to print longest series of 1's in given integer set bits.

```
#include<stdio.h>

int main()
{
    int bin,data,cnt=0,flag=0,bit;
    printf("Enter The Data : ");
    scanf("%d",&data);
    for(bit=31;bit>=0;bit--)
    {
        bin=(data>>bit)&1;
        if(bin==1)
        {
            cnt++;
        }
        if(cnt==2)
        {
            flag++;
            cnt=0;
        }
        if(bin==1)
        {
            continue;
        }
    }
    for(bit=31;bit>=0;bit--)
    {
        printf("%d",(data>>bit)&1);
```

```
        if(bit%8==0)
            printf(" ");
    }
    printf("\n");
    printf("Count is : %d\n",flag);
}
```

Program: Implement user defined mem-move function.

```
#include<stdio.h>
```

```
#include<string.h>
```

```
void user_memmove(void*,const void*,int);
```

```
int main()
```

```
{
```

```
    char str_1[100];
```

```
    char str_2[100];
```

```
    char *ptr;
```

```
    int n;
```

```
    printf("Enter The String : ");
```

```
    scanf("%s",str_1);
```

```
    printf("Enter The Posn Where You Wants To Copy : ");
```

```
    scanf("%d",&n);
```

```
    user_memmove(str_1+n,str_2,strlen(str_1));
```

```
    printf("%s",str_1);
```

```
    printf("\n");
```

```
}
```

```
void user_memmove(void *str_1,const void *str_2,int n)
{
    int i;
    char temp[100];
    for(i=0;i<n;i++)
    {
        temp[i]=((char*)str_2)[i];
    }
    for(i=0;i<n;i++)
    {
        ((char*)str_1)[i]=temp[i];
    }
}
```

Program: Write a program to print the 1st 10 prime numbers.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int min,max,i,j,cnt=0;
```

```
    for(i=2;i<=30;i++)
```

```
    {
```

```
        int flag=0;
```

```
        for(j=2;j<i;j++)
```

```
        {
```

```
            if(i%j==0)
```

```
            {
```

```
                flag=1;
```

```
                break;
```

```
            }
```

```
        }
```

```
        if(flag==0)
```

```
        {
```

```
            printf("%d\n",i);
```

```
            cnt++;
```

```
        }
```

```
        if(cnt==10)
```

```
            break;
```

```
    }
```

```
}
```

Program: User defined strncat function.

```
#include<stdio.h>
#include<string.h>
char *user_strncat(char*,char*,int);
int main()
{
    int n;
    char str_1[100];
    char str_2[200];
    printf("Enter The String - 1 ");
    scanf("%s",str_1);
    printf("Enter The String -2 ");
    scanf("%s",str_2);
    printf("How Many Character You Want to Concate of Str - 2 : ");
    scanf("%d",&n);
    user_strncat(str_1,str_2,n);
    printf("The String str - 1 is : %s\n",str_1);
}
```

```

char *user_strncat(char *str_1,char *str_2,int n)
{
    int i,j;
    for(i=0;str_1[i];i++)
        for(j=0;j<n;i++,j++)
        {
            str_1[i]=str_2[j];
        }
    str_1[i]='\0';
    return str_1;
}

```

User strcat Function:

```

char user_strcat(char *dest,const char *src)
{
    char *temp=dest;
    while(*dest)
        dest++;
    while((dest++=*src++)!='\0');
    return temp;
}

```


Program: Write a program to search a word in a FILE and replace it with its reverse equivalent.

```
#include<stdio.h>

#include<string.h>

#include<stdlib.h>

int main(int argc,char **argv)
{
    FILE *fp;
    int size,len,i=0,j=0,flag=0;
    char temp,*buf=NULL,*ptr=NULL,*copy=NULL;
    len=strlen(argv[1]);
    copy=malloc(len);
    strcpy(copy,argv[1]);
    //Reversing The Word
    j=len-1;
    while(i<j)
    {
        temp=copy[i];
        copy[i]=copy[j];
        copy[j]=temp;
        i++;
        j--;
    }
    fp=fopen(argv[2],"r");
    if(fp==NULL)
    {
        printf("ERROR : File Not Available\n");
        exit(0);
    }
}
```

```

}

fseek(fp,0,2);
size=ftell(fp);
fseek(fp,0,0);
// Reading file data
buf=malloc(size);
fread(buf,sizeof(char),size,fp);
fclose(fp);
//Searching the word replacing the with its reverse
for(ptr=buf;ptr=strstr(ptr,argv[1]);ptr+=len)
{
    strncpy(ptr,copy,len);
    flag=1;
}
if(flag)
{
    printf("%s",buf);
}
else
{
    printf("Not a Found\n");
}
}

```

Program: Write a program to print the non-repeated digits in the number.

```
#include<stdio.h>
```

```
int repeatDigits(int,int);
```

```
int main()
```

```
{
```

```
    int r,data,n;
```

```
    printf("Enter the Data : ");
```

```
    scanf("%d",&data);
```

```
    if(data==0)
```

```
        printf("0");
```

```
    n=data;
```

```
    while(data)
```

```
    {
```

```
        r=data%10;
```

```
        if(repeatDigits(n,r))
```

```
            printf("%d ",r);
```

```
        data=data/10;
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
int repeatDigits(int data,int r)
```

```
{
```

```
    int r1,count=0;
```

```
    while(data)
```

```
    {
```

```
        r1=data%10;
```

```
        if(r==r1)
```

```
            count++;
```

```
        data/=10;
```

```
    }
```

```
    if(count==1)
```

```
        return 1;
```

```
    else
```

```
        return 0;
```

```
}
```

Program: Write a program to delete a particular line in the file.

```
#include<stdio.h>

int main()
{
    FILE *file_pointer_1;
    FILE *file_pointer_2;
    char file_name[40];
    char ch;
    int delete_line;
    int temp_data=0;

    printf("Enter The File Name : ");
    scanf("%s",file_name);

    //Open File In Read Mode
    file_pointer_1=fopen(file_name,"r");
    ch=getc(file_pointer_1);
    while(ch!=EOF)
    {
        printf("%c",ch);
        ch=getc(file_pointer_1);
    }

    // Rewind
    rewind(file_pointer_1);
    printf("Enter The Line Number to Be Deleted : ");
    scanf("%d",&delete_line);
```

//Open New File In Write Mode

```
file_pointer_2=fopen("copy.c","w");
```

```
ch='A';
```

```
while(ch!=EOF)
```

```
{
```

```
    ch=getc(file_pointer_1);
```

//Except The Line To Be Deleted

```
if(temp_data!=delete_line)
```

```
{
```

```
    //Copy All Line in File copy.c
```

```
    putc(ch,file_pointer_2);
```

```
}
```

```
if(ch=='\n')
```

```
{
```

```
    temp_data++;
```

```
}
```

```
}
```

```
fclose(file_pointer_1);
```

```
fclose(file_pointer_2);
```

```
remove(file_name);
```

//Rename the file copy.c to original name

```
rename("copy.c",file_name);
```

```
printf("\nThe Contents of File After Being Modified are as follows: ");
```

```
file_pointer_1=fopen(file_name,"r");
```

```
ch=getc(file_pointer_1);
```

```
while(ch!=EOF)
```

```
{  
    printf("%c",ch);  
    ch=getc(file_pointer_1);  
}  
fclose(file_pointer_1);  
return 0;  
}
```

Program: Write a program to find the factorial of a number by recursive method.

```
#include<stdio.h>
```

```
int fact(int);
```

```
int main()
```

```
{
```

```
    int data;
```

```
    printf("Enter The Data : ");
```

```
    scanf("%d",&data);
```

```
    printf("Factorial Of %d is %d\n",data,fact(data));
```

```
}
```

```
int fact(int data)
```

```
{
```

```
    if(data==0)
```

```
        return(1);
```

```
    else
```

```
        return(data*fact(data-1));
```

```
}
```


Program: Write a program to check whether the entered number is Fibonacci or not.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int a,b,c,next,num;
```

```
    printf("Enter The Number : ");
```

```
    scanf("%d",&num);
```

```
    if((num==0)|| (num==1))
```

```
        printf("%d is Fibonacci Term\n",num);
```

```
    else
```

```
    {
```

```
        a=0;
```

```
        b=1;
```

```
        c=a+b;
```

```
        while(c<num)
```

```
        {
```

```
            a=b;
```

```
            b=c;
```

```
            c=a+b;
```

```
        }
```

```
    if(c==num)
```

```
        printf("%d is a Fibonaaci Term\n",num);
```

```
    else
```

```
        printf("%d is not Fibonaaci Term\n",num);
```

```
    }
```

```
}
```

LMS Questions

Program: Write a program to print the type (category) uppercase/lowercase/Numeric character /special character by using conditional operator only.

```
#include<stdio.h>

int main()
{
    char ch;
    printf("Enter The character : ");
    scanf("%c",&ch);
    ((ch>='A')&&(ch<='Z'))?printf("Uppercase Character\n"):
    ((ch>='a')&&(ch<='z'))?printf("Lowercase Charcater\n"):
    ((ch>='0')&&(ch<='9'))?printf("Digit\n"):
    printf("Special Character\n");
}
```

Program: Write a program to convert the supplied character from one case to another case.

```
#include<stdio.h>
#include<ctype.h>
int main()
{
    char ch;
    puts("Enter The Character : ");
    ch=getchar();
    if(isalpha(ch)){
        ch=ch^32;
        printf("%c",ch);}
    else
        printf("Error : Enter the alphabhate");
    return 0;
}
```

Program: Write a program to find sum of digits in a given number.

```
#include<stdio.h>

int main()
{
    int rem;
    int data;
    int sum=0;
    printf("Enter The data : ");
    scanf("%d",&data);
    if(data>=0)
    {
        while(data)
        {
            rem=data%10;
            sum=sum+rem;
            data=data/10;
        }
        printf("%d",sum);
    }
    else
    {
        while(data)
        {
            rem=data%10;
            sum=sum+rem;
            data=data/10;
        }
    }
}
```

```
    printf("%d",-sum);}
}
```

Program: Write whether the given integer is in ascending order or descending order or not in sorted order.

```
#include<stdio.h>
```

```
int testAsc(int);
```

```
int testDec(int);
```

```
int main()
```

```
{
```

```
    int num;
```

```
    int data;
```

```
    printf("Enter The Number : ");
```

```
    scanf("%d",&num);
```

```
    if(num<0)
```

```
        data=-num;
```

```
    else
```

```
        data=num;
```

```
    if(testAsc(data))
```

```
        printf("Entered Number is in ascending order\n");
```

```
    else if(testDec(data))
```

```
        printf("Entered Number is in Descending Order\n");
```

```
    else
```

```
        printf("Number not sorted\n");
```

```
}
```

int testAsc(int data)

```
{  
    int preDig=9;  
    int curDig;  
    while(data)  
    {  
        curDig=data%10;  
        if(curDig>preDig)  
            return 0;  
        preDig=curDig;  
        data/=10;  
    }  
    return 1;  
}
```

int testDec(int data)

```
{  
    int preDig=0;  
    int curDig;  
    while(data)  
    {  
        curDig=data%10;  
        if(curDig<preDig)  
            return 0;  
        preDig=curDig;  
        data/=10;  
    }  
    return 1; }  
}
```

Program: Write a program to print the status of a given prime number.

```
#include<stdio.h>
#include<math.h>
int main()
{
    int num;
    int squire;
    int i,k;
    int data;
    int count=0;
    printf("Enter The Number : ");
    scanf("%d",&num);
    if(num%2==0)
        printf("Entered Number is Not Prime\n");
    else
    {
        squire=sqrt(num);
        for(i=2;i<=squire;i++)
        {
            if(num%i==0)
            {
                printf("Entered Number is Not Prime\n");
                data=num;
                break;
            }
        }
        if(data==num);
```

```
else
{
    if(num<9)
        printf("Single Digit Prime\n");
    else
    {
        while(num)
        {
            k=num%10;
            count++;
            num/=10;
        }
        if(count%2)
            printf("Odd Digit Prime\n");
        else
            printf("Even Digit Prime\n");
    }
}
}
```


Program: Write a program to print non-repeated digit in a number.

```
#include<stdio.h>
```

```
int repeatDigits(int,int);
```

```
int main()
```

```
{
```

```
    int r,data,n;
```

```
    printf("Enter the Data : ");
```

```
    scanf("%d",&data);
```

```
    if(data==0)
```

```
        printf("0");
```

```
    n=data;
```

```
    while(data)
```

```
{
```

```
    r=data%10;
```

```
    if(repeatDigits(n,r))
```

```
        printf("%d ",r);
```

```
    data=data/10;
```

```
}
```

```
    printf("\n");
```

```
}
```

```
int repeatDigits(int data,int r)
```

```
{
```

```
    int r1,count=0;
```

```
    while(data){
```

```
        r1=data%10;
```

```
        if(r==r1)
```

```
            count++;
```

```
        data/=10;
```

```
    }
```

```
    if(count==1)
```

```
        return 1;
```

```
    else
```

```
        return 0;
```

```
}
```

Program: Write a program to find second highest digit in input data

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int data;
```

```
    int temp,rem;
```

```
    int snd_high_digt=0,high_digt=0;
```

```
    printf("Enter The Data : ");
```

```
    scanf("%d",&data);
```

```
    if(data<0)
```

```
        data=-data;
```

```
    temp=data;
```

```
    if(data<=9)
```

```
        printf("You Supplied Single Digit Only\n");
```

```
    else
```

```
    {
```

```
        while(data)
```

```
        {
```

```
            rem=data%10;
```

```
            if(rem>high_digt)
```

```
                rem^=high_digt^=rem^=high_digt;
```

```
            if((rem>snd_high_digt)&&(rem<high_digt))
```

```
                rem^=snd_high_digt^=rem^=snd_high_digt;
```

```
            data/=10;
```

```
        }
```

```
    }
```

```
    if(temp>9)
```

```
if(snd_high_digt==0)
    printf("All Digits are Same\n");
else
    printf("Second Highest Digit is : %d\n",snd_high_digt);
}
```

Program: Write a program to print the leap year.

```
#include<stdio.h>

int main()
{
    int n;
    int min,max;
    int count=0;
    printf("Enter The Min Year : ");
    scanf("%d",&min);
    printf("Enter The Max Year : ");
    scanf("%d",&max);
    if(min%4!=0)
        printf("Birth Year is Incorrect\n");
    else if(min>max)
        printf("ERROR : Min is greater than Max\n");
    else
    {
        for(;min<=max;min++)
        {
            n=min;
            if((n%4==0)&&(n%100!=0)||((n%400==0)))
                count++;
        }
        printf("Count is : %d\n",count);
    }
}
```

Program: Write a program to print prime numbers in the range where the prime number should contain entered digit.

```
#include<stdio.h>
```

```
int testPrime(int);
```

```
int primeDigit(int,int);
```

```
int main()
```

```
{
```

```
    int min,max;
```

```
    int digit;
```

```
    printf("Enter The Mix Digit : ");
```

```
    scanf("%d",&min);
```

```
    printf("Enter The Max Digit : ");
```

```
    scanf("%d",&max);
```

```
    printf("Enter The Digit : ");
```

```
    scanf("%d",&digit);
```

```
    for(;min<=max;min++)
```

```
    {
```

```
        if(testPrime(min))
```

```
        if(primeDigit(min,digit))
```

```
        {
```

```
            printf("Prime Digits are :%d\n",min);
```

```
        }
```

```
    }
```

```
    if(digit%2==0)
```

```
        printf("No Number present is this range with entered digit\n");
```

```
}
```

```
int primeDigit(int data, int digit)
```

```
{  
    while(data)  
    {  
        if(digit%2==0)  
            return 0;  
        if(data%10==digit)  
            return 1;  
        data/=10;  
    }  
    return 0;  
}
```

```
int testPrime(int min)
```

```
{  
    for(int i=2;i<min;i++)  
    {  
        if(min%i==0)  
            return 0;  
    }  
    return 1;  
}
```

Program: Write a program to check entered data is palindrome or not.

```
#include<stdio.h>
```

```
int revDigits(int);
```

```
int main()
```

```
{
```

```
    int data;
```

```
    printf("Enter The Data : ");
```

```
    scanf("%d",&data);
```

```
    if(revDigits(data)==data)
```

```
        printf("Enter Number is Palindrome\n");
```

```
    else
```

```
        printf("Enter Number is Not Palindrome\n");
```

```
}
```

```
int revDigits(int data)
```

```
{
```

```
    int rev=0;
```

```
    while(data)
```

```
    {
```

```
        rev=rev*10+data%10;
```

```
        data/=10;
```

```
    }
```

```
    return rev;
```

```
}
```


Program: Write a program to count pair of set bits in a given number.

```
#include<stdio.h>
```

```
int setBitPairs(int);
```

```
int main()
```

```
{
```

```
    int data;
```

```
    printf("Enter The Data : ");
```

```
    scanf("%d",&data);
```

```
    printf("Set Bit Pairs are : %d\n",setBitPairs(data));
```

```
}
```

```
int setBitPairs(int data)
```

```
{
```

```
    int cnt=0,bit=31;
```

```
    while(bit>0)
```

```
    {
```

```
        if((((data>>bit)&1)==1)&&(((data>>--bit)&1)==1))
```

```
        {
```

```
            cnt++;
```

```
            --bit;
```

```
        }
```

```
        else
```

```
            --bit;
```

```
    }
```

```
    return cnt;
```

```
}
```

Program: Write a program to hide a word with stars (*) in a given string.

```
#include<string.h>
#include<stdio.h>

int main()
{
    char str[50];
    char str_1[50];
    char temp[50]=" ";
    printf("Enter The String 1 : ");
    scanf("%s",str);
    strcpy(str+strlen(str),temp);
    printf("Enter The words to hide : ");
    scanf("%s",str_1);
    strcpy(str_1+strlen(str_1),temp);
    char *ptr;

    while(ptr=strstr(str,str_1))
    {
        memset(ptr,'*',strlen(str_1)-1);
        ptr=(ptr+strlen(str_1)-1);
    }
    printf("%s\n",str);
}
```

Program: Write a program to reverse the word which contains 2 or more than 2 vowels in a supplied string.

```
#include<stdio.h>
#include<string.h>
#include<ctype.h>
int main()
{
    char str[50];
    scanf("%[^\n]s",str);
    char ptr[20]="aeiou";
    char ptr1[20]="AEIOU";
    char temp[50]="";
    char str1[50];
    int i,j,n,k,c,c1,c2;
    n=strlen(str)-1;
    for(j=0,c1=0;j<=n;j++)
    if(isupper(str[j]))
    {
        for(k=0;k<strlen(ptr1);k++)
        if(str[j]==ptr1[k])
            c1++;
    }
    for(i=0;i<n;i++){
        for(j=0;str[i]!=' ';&&i<=n;j++,i++)
            str1[j]=str[i];
        str1[j]='\0';
        if(c1>=2)
        {
```

```

    for(j=0,c=0;j<strlen(str1);j++)
    for(k=0;k<strlen(ptr1);k++)
    {
        if(str1[j]==ptr1[k])
            c++;
    }
    if(c>=1)
        rev(str1);
}
else
{
    for(j=0,c=0,c2=0;j<strlen(str1);j++)
    for(k=0;k<strlen(ptr);k++)
    {
        if(str1[j]==ptr[k])
            c++;
        if(str1[j]==ptr1[k])
            c2++;
        if(str1[j]=="")
            str1[j]='\0';
    }
    if(c>=2||c2>=1)
        rev(str1);
}
strcat(temp,str1);
strcat(temp," ");

```

```
}  
if(c1>=2||c1>=1)  
printf("%s",temp);  
else  
{  
    if(temp[strlen(temp)-1]==' ')  
        temp[strlen(temp)-1]='\0';  
    printf("ouput=%s",temp);  
}  
}
```

void rev(char *ptr)

```
{  
    int i,j,n;  
    char temp;  
    n=strlen(ptr)-1;  
    for(j=n,i=0;i<j;j--,i++)  
    {  
        temp=ptr[i];  
        ptr[i]=ptr[j];  
        ptr[j]=temp;  
    }  
}
```

Program: In Given an unsorted integer array, place all zero to the end of the array without changing the sequence of nonzero.

```
#include<stdio.h>
```

```
int sort_data(int a[],int n);
```

```
int main()
```

```
{
```

```
    int a[1000];
```

```
    int i,n,j,k;
```

```
    scanf("%d",&n);
```

```
    if(n>100)
```

```
        printf("Memory overflow");
```

```
    else
```

```
    {
```

```
        for(i=0;i<n;i++)
```

```
            scanf("%d",&a[i]);
```

```
        sort_data(a,n);
```

```
        for(i=0;i<n;i++)
```

```
            printf("%3d",a[i]);
```

```
    }
```

```
}
```

```
int sort_data(int a[],int n)
```

```
{
```

```
    int i,j,k;
```

```
    for(i=0;i<n;i++)
```

```
        for(k=0;k<n;k++)
```

```
            if(a[i]==0)
```

```
{
```

```
    for(j=i;j<n-1;j++)
```

```
        a[j]=a[j+1];
```

```
        a[j]=0;
```

```
}
```

```
    return a;
```

```
}
```

Program: Write a program to find the GDC and LCM of Two integers.

```
#include<stdio.h>

int main()
{
    int num_1;
    int num_2;
    int gcd;
    int lcm;

    printf("Enter Number 1 : ");
    scanf("%d",&num_1);
    printf("Enter Number 2 : ");
    scanf("%d",&num_2);

    for(int i=1;i<=num_1&& i<=num_2;i++)
    {
        if(num_1%i==0 && num_2%i==0)
            gcd=i;
    }

    lcm=(num_1*num_2)/gcd;
    printf("GCD is : %d\n",gcd);
    printf("LCM is : %d\n",lcm);
}
```


Program: Write a program to print all the LEADERS in the array, an Element is a leader if it is greater than all the elements to its tight side.

For: Example, int the array {16,17,4,3,5,2}, leaders are 17,5, and 2.

```
#include<stdio.h>

int main()
{
    int a[20];
    int i,j,n;
    printf("Enter The No. To be input : ");
    scanf("%d",&n);
    if(n<0)
    {
        printf("Invalid Choice\n");
        return 0;
    }
    for(i=0;i<n;i++)
    {
        printf("Enter The Element of array : ");
        scanf("%d",&a[i]);
    }
    for(i=0;i<n;i++)
    {
        for(j=i+1;j<n;j++)
        {
            if(a[i]<a[j])
                break;
        }
        if(j==n)
```

```
        printf("LADDR is : %d\n",a[i]);  
    }  
}
```

Program: Write a program to convert temperature form to F or vice versa.

1. For Fahrenheit to Celsius

2. For Celsius to Fahrenheit

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    printf("1 - For Fahrenheit to Celsius\n");
```

```
    printf("2 - For Celsius To Fahrenheit\n");
```

```
    int choice;
```

```
    float fahr,celc;
```

```
    printf("Enter Your Choice : ");
```

```
    scanf("%d",&choice);
```

```
    switch(choice)
```

```
    {
```

```
        case 1: printf("Enter the Data : " );
```

```
            scanf("%f",&celc);
```

```
            fahr=(celc*9/5)+32;
```

```
            printf("Farenite Data is : %f\n",fahr);
```

```
            break;
```

```
        case 2: printf("Enter The Data : ");
```

```
            scanf("%f",&fahr);
```

```
            celc=(fahr-32)*5/9;
```

```
            printf("Centrigrate Data is : %f\n",celc);
```

```
            break;
```

```
    }
```

```
}
```

Program: Write a program to implant user-defended atio function.

```
#include<stdio.h>
#include<ctype.h>
#include<string.h>
int main()
{
    char str[20];
    int i;
    printf("Enter The String : ");
    scanf("%s",str);
    if(isalpha(str[0]))
    {
        printf("0");
        return 0;
    }
    for(i=0;i<strlen(str);i++)
        if((str[i]>='0'&&str[i]<='9')||(str[i]=='+'||str[i]=='-'))
            printf("%c",str[i]);
        else if(isalpha(str[i]))
            return 0;
}
```

Program: Write a program to print the binary equivalent of float.

```
#include<stdio.h>

union data
{
    int i;
    float f;
};

int main()
{
    union data v;
    int bit;
    printf("Enter The data : ");
    scanf("%f",&v.f);
    printf("The Binary is : ");
    for(bit=31;bit>=0;bit--)
    {
        printf("%d",(v.i>>bit)&1);
    }
    printf("\n");
}
```

Program: Write a program to print binary data of double, and give space for every 8 bits and also it has to work for exponential input.

```
#include<stdio.h>

union data
{
    long long int i;
    double d;
};

int main()
{
    union data v;
    printf("Enter The Data : ");
    scanf("%lf",&v.d);
    int bit;
    printf("Double Equavalnt Of Binary is : ");
    for(bit=63;bit>=0;bit--)
    {
        printf("%lld",(v.i>>bit)&1);
        if(bit%8==0)
            printf(" ");
    }
    printf("\n");
}
```

Program: Write a program to convert little endian to big endian.

```
#include<stdio.h>

int main()
{
    int bit;
    int data;
    int big=0;
    printf("Enter The Data : ");
    scanf("%d",&data);
    printf("Little Endian : ");
    for(bit=31;bit>=0;bit--)
    {
        printf("%d",(data>>bit)&1);
        if(bit%8==0)
            printf(" ");
    }
    printf("\n");
    printf("Big Endian  : ");
    for(bit=7;big<=3;bit--)
    {
        printf("%d",(data>>bit)&1);
        if(bit%8==0)
            printf(" ");
        if(bit==0)
        {
            data=data>8;
            bit=8;
        }
    }
}
```

```
        big++;  
    }  
}  
printf("\n");  
}
```


Program: Write a program to print and count palindrome words in given string.

```
#include<stdio.h>

#include<string.h>

int testPalindrome(char*);

int main()
{
    char str[50];
    char str_1[50];
    int cnt=0;
    printf("Enter The String : ");
    scanf("%s",str);
    int len;
    int j;
    len=strlen(str);
    for(int i=0;i<len-1;i++)
    {
        for(j=0,i<len;str[i]!=' ';&&str[i]!='\0';i++,j++)
        {
            str_1[j]=str[i];
        }
        str_1[i]='\0';
        if(testPalindrome(str_1))
        {
            printf("Palindrome is : %s\n",str_1);
            cnt++;
        }
    }
    printf("Palindrome Count is : %d",cnt);
}
```

```
    }  
    printf("\n");  
}
```

```
int testPalindrome(char* str_1)  
{  
    int len;  
    int j;  
    len=strlen(str_1);  
    for(int i=0,j=len-1;i<j;i++,j--)  
    {  
        if(str_1[i]!=str_1[j])  
            return 0;  
    }  
    return 1;  
}
```

Program: Write a program to find highest and lowest elements in an array.

```
#include<stdio.h>

int main()
{
    int arr[20];
    int len,num;
    int i;
    int high=0,low=0;

    printf("Enter The data : ");
    scanf("%d",&num);
    if(num<=0)
    {
        printf("Invalid Size\n");
        return 0;
    }
    for(i=0;i<num;i++)
    {
        printf("Enter The Numbers [%d] : ",i);
        scanf("%d",&arr[i]);
    }
    high=low=arr[0];
    for(i=1;i<num;i++)
    {
        if(high<arr[i])
            high=arr[i];
        if(low>arr[i])
```

```
        low=arr[i];
    }
    if(high==low)
        printf("All Elements are Same\n");
    else
        printf("Highest :%d\nLowest : %d\n",high,low);
}
```

Program: Write a program to convert first character of every word into uppercase.

```
#include<string.h>
#include<ctype.h>
int main()
{
    char str[50];
    printf("Enter The String : ");
    scanf("%[^\n]s",str);
    int len,i,j=0;
    len=strlen(str);
    for(i=0;i<len;i++)
    {
        if(isupper(str[j]))
        {
            str[i]=str[j];
            break;
        }
        else
            if(isalpha(str[i]))
            {
                str[i]=str[i]^32;
                break;
            }
            else
                if(isalpha(str[j]))
                {
                    str[i]=str[j];
```

```
        }  
    }  
    printf("Final String : ");  
    for(i=0;i<len;i++)  
        printf("%c",str[i]);  
    printf("\n");  
}
```

Program: Write a program to convert all small vowels letters into the capital letter from a given string.

```
#include<stdio.h>

#include<string.h>

int main()
{
    char str[20];
    printf("Enter The String : ");
    scanf("%[^\n]s",str);
    int len,i;
    len=strlen(str);
    printf("Converted String : ");
    for(i=0;i<len;i++)
    {
        if(str[i]=='a' || str[i]=='e' || str[i]=='i' || str[i]=='o' || str[i]=='u')
        {
            str[i]=str[i]-32;
        }
        printf("%c",str[i]);
    }
    printf("\n");
}
```

Program: Write a program to display duplicate element in an array. Array size must be >20.

```
#include<stdio.h>

int main()
{
    int a[20];
    int i,j,k;
    int cnt=0;
    int cnt_1;
    int num;
    printf("Enter The Number : ");
    scanf("%d",&num);
    if(num<=0)
    {
        printf("Invalid Size\n");
        return 0;
    }
    for(i=0;i<num;i++)
    {
        printf("Enter The Numbers %d : ",i);
        scanf("%d",&a[i]);
    }
    for(i=0;i<num;i++)
        for(j=i+1;j<num;j++)
            if(a[i]==a[j])
            {
                for(k=0;k<j;k++)
                {
```



```
        if(a[i]==a[k])
            cnt++;
    }
    if(cnt==1)
    {
        printf("Duplicate Element is : ");
        printf("%d",a[i]);
cnt_1++;
    }
}

if(cnt_1==0)
{
    printf("No Duplicate Element Found\n");
}

printf("\n");
}
```

Program: Write a program to print prime numbers in a given range the sum of prime number should be equal to be 2.

```
#include<stdio.h>
```

```
int testPrime(int);
```

```
int sumPrime(int);
```

```
int main()
```

```
{
```

```
    int min,max;
```

```
    printf("Enter The Min : ");
```

```
    scanf("%d",&min);
```

```
    printf("Enter The Max : ");
```

```
    scanf("%d",&max);
```

```
    if((min==0)&&(max==1))
```

```
    {
```

```
        printf("Entered Invalid Range\n");
```

```
        return 0;
```

```
    }
```

```
    for(;min<=max;min++)
```

```
    {
```

```
        if(testPrime(min))
```

```
        if(sumPrime(min)==2)
```

```
            printf("%d, ",min);
```

```
    }
```

```
}
```

```
int testPrime(int data)
```

```
{  
    int i;  
    for(i=2;i<data;i++)  
    {  
        if(data%i==0)  
            return 0;  
    }  
    return 1;  
}
```

```
int sumPrime(int data)
```

```
{  
    int r,sum=0,n;  
    n=data;  
    while(data)  
    {  
        r=data%10;  
        sum=sum+r;  
        data/=10;  
    }  
    if(sum==2)  
    {  
        printf("%d",n);  
        return 0;  
    }  
    else if(sum>2)
```

```
{  
    int sum1=0;  
    while(sum  
    {  
        sum1=sum1+(sum%10);  
        sum/=10;  
    }  
    return sum1;  
}  
}
```

Program: Write a program to build strong password.

```
#include<stdio.h>
#include<ctype.h>
int main()
{
    char str[10];
    scanf("%[^\n]s",str);
    int i,c1=0,c2=0,c3=0,c4=0,c=0;
    for(i=0;str[i];i++)
    {
        if(isupper(str[i]))
            c1++;
        else if(islower(str[i]))
            c2++;
        else if(isdigit(str[i]))
            c3++;
        else if(str[i]!='\n')
            c4++;
        c++;
    }
    if(c1==0&& c2==0&& c3==0&& c4==0)
        printf("password with min 6 characters should contain special
char,numbers,upper and lower case");
    else if(c1==6)
        printf("use lower case,numbers and special");
    else if(c2==6)
        printf("use upper case numbers and special");
    else if(c3==6)
```

```
printf("use upper case lower and special");
else if(c4==6)
printf("use upper case,lower case and numbers");
else if(c1==1&&c2==1&&c3==1&&c4==0)
printf("3 more use special");
else if(c1==1&&c2==1&&c3==0&&c4==1)
printf("3 more use numbers");
else if(c1==1&&c2==0&&c3==1&&c4==1)
printf("3 more use lower case");
else if(c1==0&&c2==1&&c3==1&&c4==1)
printf("3 more use upper case");
else if(c1==1 &&c2==0 &&c3==0 &&c4==0)
printf("5 more use lower case,numbers and special");
else if(c2==1&&c1==0&&c3==0&&c4==0)
printf("5 more use upper case numbers and special");
else if(c3==1&&c1==0&&c2==0&&c4==0)
printf("5 more use upper case lower and special");
else if(c4==1&&c1==0&&c2==0&&c3==0)
printf("5 more use upper case,lower case and numbers");
else if(c1==1&&c4==1&&c2==0&&c3==0)
printf("4 more use lower case and numbers");
else if(c2==1&&c4==1&&c1==0&&c3==0)
printf("4 more use upper case and numbers");
else if(c3==1&&c4==1&&c1==0&&c2==0)
printf("4 more use upper and lower case");
else if(c1==1&&c3==1&&c2==0&&c4==0)
printf("4 more use lower case and special");
```

```
else if(c1>=1&&c2>=1&&c3>=1&&c4>=1&&c>=7)
printf("passed");
}
```

Program: Write a program to print the highest even length of a word in given string.

```
#include<stdio.h>
#include<string.h>
int main()
{
    int cnt=0;
    int len=0;
    int n,i;
    char str[50];
    printf("Enter The String : ");
    scanf("%[^\n]s",str);
    n=strlen(str);
    for(i=0;i<n-1;i++)
    {
        if(str[i]!=' ' && str[i]!='\0' && str[i]!="")
        {
            cnt++;
            continue;
        }
        printf("Length Is : %d\n",cnt);
        return 0;
    }
    printf("Lenght is : %d",cnt=0);
    printf("\n");
}
```


Program: Write a program to reverse the numeric string inside a main string.

```
#include<stdio.h>
#include<ctype.h>
#include<string.h>
#include<stdlib.h>
int main()
{
    char *str=NULL;
    str=getstring();
    int i,n,x=0;
    n=strlen(str);
    for(i=0;i<n;i++)
    {
        if((isdigit(str[i]) && str[i+1]==' ') || (isdigit(str[i-1]) && i==n-1))
            rev(i,x,str);
        if(str[i]==' ')
            x=i+1;
        if(str[i]=='!')
        {
            printf("Invalid String");return 0;}
    }
    puts(str);
}
```

void* getstring()

```
{
    int i=0;
    char *p=NULL;
    do
    {
        p=realloc(p,i+1);
        p[i]=getchar();
    }while(p[i++]!='\n');
    p[i-1]='\0';
    return p;
}
```

void rev(int s,int x,char *str)

```
{
    int i,j;
    char temp;
    for(i=x,j=s;i<j;i++,j--)
    {
        temp=str[i];
        str[i]=str[j];
        str[j]=temp;
    }
}
```

Program: Write a program to replace all 0's with 1 in a given integer.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int n,data,a,b;
```

```
    scanf("%d",&data);
```

```
    if(data==0)
```

```
        printf("1");
```

```
    else
```

```
        if(rev(data)==data)
```

```
        {
```

```
            printf("No Zeros In Input");
```

```
        }
```

```
    else if(data<0)
```

```
    {
```

```
        a=-data;
```

```
        b=rev(a);
```

```
        printf("%d",-b);
```

```
    }
```

```
    else
```

```
        printf("%d",rev(data));
```

```
}
```

```
int rev(int data)
{
    int r,rev1=0,n;
    while(data)
    {
        r=(data%10);
        if(r==0)
            r=1;
        rev1=(rev1*10)+r;
        data/=10;
    }
    n=rev1;
    int r1,reve=0;
    while(n)
    {
        r1=n%10;
        reve=(reve*10)+r1;
        n/=10;
    }
    return reve;
}
```

Program: Write a program to print non-repeated digit in a number.

```
#include<stdio.h>
```

```
int repeatDigits(int,int);
```

```
int main()
```

```
{
```

```
    int data;
```

```
    int rem;
```

```
    int num;
```

```
    printf("Enter The Data : ");
```

```
    scanf("%d",&data);
```

```
    if(data==0)
```

```
        printf("0");
```

```
    num=data;
```

```
    while(data)
```

```
{
```

```
    rem=data%10;
```

```
    if(repeatDigits(num,rem))
```

```
        printf("%d ",rem);
```

```
    data=data/10;
```

```
}
```

```
    printf("\n");
```

```
}
```

```
int repeatDigits(int data, int rem)
```

```
{
```

```
    int rem_1;
```

```
    int count=0;
```

```
    while(data)
```

```
{
```

```
        rem_1=data%10;
```

```
        if(rem==rem_1)
```

```
            count++;
```

```
            data/=10;
```

```
}
```

```
if(count==1)
```

```
    return 1;
```

```
else
```

```
    return 0;
```

```
}
```

Program: Write a program to print all palindrome numbers in given range.

```
#include<stdio.h>
```

```
int testPalindrome(int);
```

```
int main()
```

```
{
```

```
    int min;
```

```
    int max;
```

```
    int i;
```

```
    int n;
```

```
    printf("Enter The Min Value : ");
```

```
    scanf("%d",&min);
```

```
    printf("Enter The Max Value : ");
```

```
    scanf("%d",&max);
```

```
    for(i=min;min<=max;min++)
```

```
    {
```

```
        n=testPalindrome(min);
```

```
        if(i==n)
```

```
            printf("%d ",i);
```

```
    }
```

```
}
```

```
int testPalindrome(int min)  
{  
    int rev=0;  
    while(min)  
    {  
        rev=(rev*10)+(min%10);  
        min/=10;  
    }  
    return rev;  
}
```


Program: Write a program to reverse the word which contains 2 or more than 2 vowels in a supplied string.

```
#include<stdio.h>
#include<string.h>
#include<ctype.h>
int main()
{
    char str[50];
    printf("Enter The String : ");
    scanf("%[^\n]s",str);
    char ptr[20]="aeiou";
    char ptr1[20]="AEIOU";
    char temp[50]="";
    char str1[50];
    int i,j,n,k,c,c1,c2;
    n=strlen(str)-1;
    for(j=0,c1=0;j<=n;j++)
        if(isupper(str[j]))
        {
            for(k=0;k<strlen(ptr1);k++)
                if(str[j]==ptr1[k])
                    c1++;
        }
    for(i=0;i<n;i++)
    {
        for(j=0;str[i]!=' '&&i<=n;j++,i++)
            str1[j]=str[i];
        str1[j]='\0';
    }
}
```

```

if(c1>=2)
{
    for(j=0,c=0;j<strlen(str1);j++)
        for(k=0;k<strlen(ptr1);k++)
        {
            if(str1[j]==ptr1[k])
                c++;
        }
    if(c>=1)
        rev(str1);
}
else
{
    for(j=0,c=0,c2=0;j<strlen(str1);j++)
        for(k=0;k<strlen(ptr);k++)
        {
            if(str1[j]==ptr[k])
                c++;
            if(str1[j]==ptr1[k])
                c2++;
            if(str1[j]=="")
                str1[j]='\0';
        }
    if(c>=2||c2>=1)
        rev(str1);
}
strcat(temp,str1);

```

```

        strcat(temp, " ");
    }
    if(c1>=2||c1>=1)
        printf("%s",temp);
    else
    {
        if(temp[strlen(temp)-1]==' ')
            temp[strlen(temp)-1]='\0';
        printf("Reversed String : %s",temp);
    }
    printf("\n");
}

```

void rev(char *ptr)

```

{
    int i,j,n;
    char temp;
    n=strlen(ptr)-1;
    for(j=n,i=0;i<j;j--,i++)
    {
        temp=ptr[i];
        ptr[i]=ptr[j];
        ptr[j]=temp;
    }
}

```

Program: In Given an unsorted integer array, place all zero to the end of the array without changing the sequence of nonzero.

```
#include<stdio.h>
```

```
int sort(int a[],int n);
```

```
int main()
```

```
{
```

```
    int a[1000];
```

```
    int i,n,j,k;
```

```
    scanf("%d",&n);
```

```
    if(n>100)
```

```
        printf("Memory Overflow");
```

```
    else
```

```
    {
```

```
        for(i=0;i<n;i++)
```

```
            scanf("%d",&a[i]);
```

```
        sort(a,n);
```

```
        for(i=0;i<n;i++)
```

```
            printf("%3d",a[i]);
```

```
    }
```

```
}
```

```
int sort(int a[],int n)
{
    int i,j,k;
    for(i=0;i<n;i++)
    for(k=0;k<n;k++)
        if(a[i]==0){
            for(j=i;j<n-1;j++)
                a[j]=a[j+1];
            a[j]=0;
        }
    return a;
}
```

Program: Write a program to delete the particular word in FILE.

```
#include <stdio.h>
#include <string.h>
#define MAX 256
int main()
{
    FILE *fp1, *fp2;
    char word[MAX], filename[MAX];
    char str[MAX], temp[] = "temp.txt", *ptr, *tmp;

    /* Get The Input File From The User */
    printf("Enter your input file name:");
    fgets(filename, MAX, stdin);
    filename[strlen(filename) - 1] = '\0';

    /* Get the word to Delete From The User */
    printf("Enter your input word:");
    scanf("%s", word);

    /* Open Input File In Read Mode */
    fp1 = fopen(filename, "r");
```

/* Error Handling */

```
if (!fp1)
{
    printf("Unable to open the input file!!\n");
    return 0;
}
```

/* Open Temporary File in Write Mode */

```
fp2 = fopen(temp, "w");
```

/* Error Handling */

```
if (!fp2)
{
    printf("Unable to open temporary file!!\n");
    return 0;
}
```

/* Delete The Given Word From the File */

```
while (!feof(fp1))
{
    strcpy(str, "\0");
    /* Read line by line From the Input File */
    fgets(str, MAX, fp1);
```

/*Check whether the Word to Delete is Present in The Current Scanned Line */

if (strstr(str, word))

{

tmp = str;

while(ptr = strstr(tmp, word))

{

/*Letters Present Before The Word To Be Deleted

***/**

while (tmp != ptr) {

fputc(*tmp, fp2);

tmp++;

}

/* Skip The Word To Be Deleted */

ptr = ptr + strlen(word);

tmp = ptr;

}

/* Characters Present After The Word To Be Deleted */

while (*tmp != '\0')

{

fputc(*tmp, fp2);

tmp++;

}

}


```
        else
        {
            /*Current Scanned Line doesn't have The Word that Need
To Be Deleted*/

            fputs(str, fp2);
        }
    }

/* Close The Opened Files */
fclose(fp1);
fclose(fp2);

/* Remove The Input File */
remove(filename);

/* Rename Temporary File Name To Input File Name */
rename(temp, filename);
return 0;
}
```

Program: Write a program to reverse the word in the FILE.

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
int main(int argc, char **argv)
{
    FILE *fp;
    int size,len,i=0,j=0,flag=0;
    char temp,*buf=NULL,*ptr=NULL,*copy=NULL;

    len=strlen(argv[1]);
    copy=malloc(len);
    strcpy(copy,argv[1]);

    // reversing the word
    j=len-1;
    while(i<j)
    {
        temp=copy[i];
        copy[i]=copy[j];
        copy[j]=temp;

        i++;
        j--;
    }
}
```

```

fp=fopen(argv[2],"r");
if(fp==NULL)
{
    printf("ERROR: File is not available...\n");
    exit(0);
}

fseek(fp,0,2);
size=ftell(fp);
fseek(fp,0,0); //rewind(fp;

//reading file data
buf=malloc(size);
fread(buf,sizeof(char),size,fp);
fclose(fp);

//searching word & replacing with its reverse
for(ptr=buf; ptr=strstr(ptr,argv[1]); ptr+=len)
{
    strncpy(ptr,copy,len);
    flag=1;
}
if(flag)
printf("%s",buf);
else
printf("Not a single word available in that particular file as you
given\n");
}

```

Program: Write a program to replace the substring in main string.

```
#include<stdio.h>
```

```
#include<string.h>
```

```
void replaceSubstring(char [],char[],char[]);
```

```
int main()
```

```
{
```

```
    char string[100];
```

```
    char sub[100];
```

```
    char new_str[100];
```

```
    printf("Enter a string: ");
```

```
    gets(string);
```

```
    printf("Enter the substring: ");
```

```
    gets(sub);
```

```
    printf("Enter the new substring: ");
```

```
    gets(new_str);
```

```
    replaceSubstring(string,sub,new_str);
```

```
    printf("The string after replacing : %s\n",string);
```

```
}
```

```
void replaceSubstring(char string[],char sub[],char new_str[])
```

```
{
```

```
    int stringLen;
```

```
    int subLen;
```

```
    int newLen;
```

```
    int i=0,j,k;
```

```
    int flag=0;
```

```
    int start;
```

```
    int end;
```

```
    stringLen=strlen(string);
```

```
    subLen=strlen(sub);
```

```
    newLen=strlen(new_str);
```

```
    for(i=0;i<stringLen;i++)
```

```
    {
```

```
        flag=0;
```

```
        start=i;
```

```
        for(j=0;string[i]==sub[j];j++,i++)
```

```
            if(j==subLen-1)
```

```
                flag=1;
```

```
        end=i;
```

```
        if(flag==0)
```

```
            i-=j;
```

```
        else
```

```
        {
```

```
            for(j=start;j<end;j++)
```

```
            {
```

```
                for(k=start;k<stringLen;k++)
```

```
        string[k]=string[k+1];
    stringLen--;
    i--;
}
for(j=start;j<start+newLen;j++)
{
    for(k=stringLen;k>=j;k--)
        string[k+1]=string[k];
    string[j]=new_str[j-start];
    stringLen++;
    i++;
}
}
}
```

Program: Write a program to delete a particular line in FILE.

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#include<unistd.h>
int main(int argc,char **argv)
{
    char str[100];
    char(*p)[100]=NULL;
    int linenum,cnt=0,i;
    FILE *fp;
    linenum=atoi(argv[1]);
    fp=fopen(argv[2],"r");
    if(fp==NULL)
    {
        printf("Error: Unable to Open The File\n");
        return 0;
    }
    while(fgets(str,100,fp)!=NULL)
    {
        p=realloc(p,(cnt+1)*sizeof(*p));
        strcpy(p[cnt],str);
        cnt++;
    }
    fclose(fp);
```

```

printf("Total Lines are : % d \n",cnt);
printf("Defore Delete Line : \n");
for(i=0;i<cnt;i++)
    printf("%s",p[i]);
if((linenum>cnt)||(linenum<0))
{
    printf("Error : Sorry This Line Number is Not Here\n");
    return 0;
}
linenum--;
memmove(p+linenum,p+linenum+1,(cnt-linenum)*sizeof(*p));
cnt--;
p=realloc(p,cnt*sizeof(*p));
printf("After Delete Line : \n" );
for(i=0;i<cnt;i++)
    printf("%s",p[i]);
fp=fopen(argv[2],"w");
for(i=0;i<cnt;i++)
    fputs(p[i],fp);
fclose(fp);
}

```


Program: Write a program to delete particular digit in entered data.

```
#include<stdio.h>
```

```
int power(int,int);
```

```
int check_digit(int,int);
```

```
int main()
```

```
{
```

```
    int number,d,temp;
```

```
    printf("Enter The data : ");
```

```
    scanf("%d",&number);
```

```
    printf("Enter The Number You Want to be Delete : ");
```

```
    scanf("%d",&d);
```

```
    temp=number;
```

```
    number=check_digit(number,d);
```

```
    if(temp==number)
```

```
        printf("Data not Found");
```

```
    printf("Data After Delelte The Number : %d\n",number);
```

```
}
```

```
int power(int x, int i)
```

```
{
```

```
    int y=1;
```

```
    if(i==0)
```

```
        return 1;
```

```
    while(i)
```

```
    {
```

```
        y=10*y;
```

```
        i--;
```

```
    }
```

```
    return y;  
}
```

```
int check_digit(int num,int d)
```

```
{  
    int rem,sum=0,i=0,x=10;  
    while(num>0)  
    {  
        rem=num%10;  
        if(rem!=d)  
        {  
            sum=sum+rem*power(10,i);  
            i++;  
        }  
        num=num/10;  
    }  
    return sum;  
}
```

Program: Write a structure to add the data in single linked list.

```
struct Student * Add(struct Student *ptr)
{
    struct Student *newnode=NULL,*temp=NULL;
    /*** creation of the node ***/
    newnode=calloc(1,sizeof(struct Student ));

    if(newnode==NULL)
    {
        printf("node not created\n");
    }
    else
    {
        /*** initialise the node ***/
        printf("enter the roll\n");
        scanf("%d",&newnode->roll);
        printf("enter the name\n");
        scanf("%s",newnode->name);
        printf("enter the marks\n");
        scanf("%f",&newnode->marks);
        //newnode->link=NULL to wriiten if we use
        //malloc func for creation of node
        if(ptr==NULL)// List is empty
        {
            ptr=newnode; // making newnode as first node
```

```
    }  
    else  
    {  
        // if list is already existing , traverse up to last node and link  
newnode to last node  
        for(temp=ptr;temp->link;temp=temp->link);  
  
        temp->link=newnode; // linking newnode to last node  
    }  
}  
return ptr;  
}
```

Program: Write a structure to print/display the data in single linked list.

void Display(struct Student *ptr)

```
{  
    if(ptr==NULL)  
    {  
        printf("List is empty\n");  
        return;  
    }  
    while(ptr)  
    {  
        printf("%d %s %f\n",ptr->roll,ptr->name,ptr->marks);  
        ptr=ptr->link;  
    }  
}
```

Program: Write a structure to add data in sorted order in single linked list.

struct Student * Add(struct Student *ptr)

```
{  
    struct Student *newnode=NULL,*temp=NULL,*prev=NULL;  
    /** creation of the node ***/  
    newnode=calloc(1,sizeof(struct Student ));  
  
    if(newnode==NULL)  
    {  
        printf("Node Not Created\n");  
    }  
    else  
    {  
        /** initialise the node ***/  
        printf("Enter The roll : ");  
        scanf("%d",&newnode->roll);  
        printf("Enter The Name : ");  
        scanf("%s",newnode->name);  
        printf("Enter The Marks : ");  
        scanf("%f",&newnode->marks);  
  
        /* if list is empty or newnode roll is smaller than first node roll ,  
adding newnode as first node***/  
        if(ptr==NULL || newnode->roll < ptr->roll)  
        {  
            newnode->link=ptr;  
            ptr=newnode;  
        }  
        else
```

```
        {      /** if list is empty and newnode roll is greater than first  
node roll,then traverse inthe list untill we reach end of the list or until we  
come across a value which is greater than newnode roll *****/
```

```
            prev=ptr;
```

```
            temp=ptr->link;
```

```
            while(temp && newnode->roll > temp->roll)
```

```
            {
```

```
                prev=temp;
```

```
                temp=temp->link;
```

```
            }
```

```
            prev->link=newnode;    //newnode address is assigned to prev
```

```
link
```

```
            newnode->link=temp;
```

```
// temp is assigned to newnode link;
```

```
        /** we are inserting newnode between the prev and temp  
nodes*****/
```

```
        }
```

```
    }
```

```
    return ptr;    // returning first node address
```

```
}
```

Program: Write a program to save the data in single linked list.

void Save(struct Student *ptr)

```
{  
    FILE *fp;  
    if(ptr==NULL)  
    {  
        printf("List is empty\n");  
        return;  
    }  
    fp=fopen("Stu","w");  
    if(fp==NULL)  
    {  
        printf("file not opened\n");  
        return;  
    }  
    while(ptr)  
    {  
        fwrite(ptr,1,sizeof(struct Student)-sizeof(struct Student *),fp);  
        ptr=ptr->link;  
    }  
    fclose(fp);  
}
```


Program: Write a program to delete first data in single linked list.

```
struct Student * Del_first(struct Student *ptr)  
{  
    struct Student *temp=NULL;  
    if(ptr==NULL)  
    {  
        printf("List is empty\n");  
        return ptr;  
    }  
    temp=ptr;           // first node address assigned to temp  
    ptr=ptr->link;       // making second node as first  
    free(temp);          // deleting first node  
    temp=NULL;  
    return ptr;  
}
```

Program: Write a program to delete all data in single linked list.

struct Student * Del_All(struct Student *ptr)

```
{    struct Student *temp=NULL;
    while(ptr)
    {
        temp=ptr;
        ptr=ptr->link;
        free(temp);
        temp=NULL;
    }
    return ptr;
}
```

Program: Write a program to delete last data in single linked list.

```
struct Student * Del_last(struct Student *ptr)
{
    struct Student *temp=NULL,*prev=NULL;
    if(ptr==NULL)
    {
        printf("list is empty\n");
        return ptr;
    }
    temp=ptr;
    while(temp->link)
    {
        prev=temp;
        temp=temp->link;
    }
    free(temp);
    temp=NULL;
    if(prev)
        prev->link=NULL;
    else
        ptr=NULL;
    return ptr;
}
```

Program: Write a program to delete roll no data in single linked list.

```
struct Student *Del_roll(struct Student *ptr)
{
    struct Student *temp=NULL,*prev=NULL;
    int data;
    printf("Enter The Data To Be Deleted : ");
    scanf("%d",&data);
    if(ptr==NULL)
    {
        printf("List Is Empty\n");
        return ptr;
    }
    else if(data==ptr->roll)          // matching with first node data
    {
        temp=ptr;
        ptr=ptr->link;
        free(temp);
        temp=NULL;
    }
    else
    {
        temp=ptr->link;
        prev=ptr;
        // traversing until we find the data or we reached the end of the
list
        while(temp && temp->roll!=data)
        {
            prev=temp;
```

```

        temp=temp->link;
    }
    if(temp==NULL) // if we reach end of the list
    {
        printf("%d is not found\n",data);
    }
    else
    {
// if data found, linking prev node with next node of temp
        prev->link=temp->link;
        free(temp);        // deleting temp
        temp=NULL;
    }
}

return ptr;        // returning first node address
}

```

Program: Write a program to reverse the data in single linked list.

```
struct Student *Reverse(struct Student *ptr)
{
    struct Student *prev=NULL,*cur=NULL,*next=NULL;
    if(ptr==NULL)           // list is empty
    {
        printf("SLL is empty\n");
//        return ptr;
    }
    else if(ptr->link==NULL)
    {
        printf("SLL is having only one node\n");
    }
    else
    {
        next=ptr;
        while(next)
        {
            prev=cur;
            cur=next;
            next=next->link;
            cur->link=prev;    // linking with prevnode
        }
        ptr=cur;
    }
    return ptr;
}
```

```
}
```

Program: Write a program to add the data in double linked list.

```
struct Person * Add_beg(struct Person *ptr)
```

```
{
```

```
    struct Person *newnode=NULL;
```

```
    newnode=calloc(1,sizeof(struct Person));
```

```
    if(newnode==NULL)
```

```
    {
```

```
        printf("Node Not Created\n");
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("Enter The Age : ");
```

```
        scanf("%d",&newnode->age);
```

```
        printf("Enter The Name : ");
```

```
        scanf("%s",newnode->name);
```

```
        newnode->link=ptr;           // forward linking
```

```
        if(ptr)
```

```
            ptr->prev=newnode;       // backward linking
```

```
        ptr=newnode;
```

```
    }
```

```
    return ptr;
```

```
}
```

Program: Write a program to delete first in double linked list.

```
struct Person * Del_first(struct Person *ptr)
{
    struct Person *temp=NULL;
    if(ptr==NULL)
    {
        printf("DLL is Empty\n");
    }
    else
    {
        temp=ptr;           // assigning first node to temp
        ptr=ptr->link;       // moving ptr to next node
        if(ptr)
            ptr->prev=NULL;   // making ptr->prev to NULL
        free(temp); // deleting first node
        temp=NULL;
    }
    return ptr;
}
```


Program: Write a program to delete all data in double linked list.

```
struct Person * Del_data(struct Person *ptr)
```

```
{  
    struct Person *temp=NULL;  
    int data;  
    printf("Enter The Data To be Deleted : ");  
    scanf("%d",&data);  
    if(ptr==NULL)  
    {  
        printf("DLL is Empty\n");  
    }  
    else if(data==ptr->age)  
    {  
        /*temp=ptr;  
        ptr=ptr->link;  
        if(ptr)  
            ptr->prev=NULL;  
        free(temp);  
        temp=NULL;*/  
        ptr=Del_first(ptr);  
    }  
    else  
    {  
        temp=ptr;  
        while(temp && temp->age !=data)  
            temp=temp->link;  
        if(temp==NULL)
```

```
        {
            printf("%d is not Found\n",data);
        }
    else
    {
        temp->prev->link=temp->link;
        if(temp->link)
            temp->link->prev=temp->prev;
        free(temp);
        temp=NULL;
    }

}

return ptr;
}
```

Program: Write a program to delete last data in double linked list.

```
struct Person * Dellast(struct Person *ptr)
{
    struct Person *temp=NULL;
    if(ptr==NULL)
    {
        printf("DLL is Empty\n");
    }
    else
    {
        for(temp=ptr;temp->link;temp=temp->link);
        if(temp->prev)
            temp->prev->link=NULL;
        else
            ptr=NULL;
        free(temp);
        temp=NULL;
    }
    return ptr;
}
```

Program: Write a program to display the data in double linked list.

void Display(struct Person *ptr)

```
{
    if(ptr==NULL)
    {
        printf("DLL is Empty\n");
    }
    else
    {
        while(ptr)
        {
            Printf(Entered Age is : %d\n",ptr->age);
            printf("Entered Name Is : %s\n",ptr->name);
            ptr=ptr->link;
        }
    }
}
```

Program: Write a program to reverse the data in double linked list.

```
struct Person *Reverse(struct Person *ptr)
{
    struct Person *temp=NULL,*temp1=NULL;
    if(ptr==NULL)
    {
        printf("DLL is Empty\n");
    }
    else if(ptr->link==NULL)
    {
        printf("DLL is having One Node\n");
    }
    else
    {
        temp=ptr;
        while(temp)
        {
            temp1=temp->prev;
            temp->prev=temp->link;
            temp->link=temp1;
            temp=temp->prev;      // moving to next node
        }
        ptr=temp1->prev;
    }
    return ptr;
}
```

Program: Write a program to display the reverse data in double linked list.

```
void RDisplay(struct Person* ptr)
{
    if(ptr==NULL)
    {
        printf("DLL is Empty\n");
    }
    else
    {
        for(;ptr->link;ptr=ptr->link);

        while(ptr)
        {
            printf("%d %s\n",ptr->age,ptr->name);
            ptr=ptr->prev;    // backward traversing
        }
    }
}
```

Program: Write a program to add data in circular linked list.

```
struct Employee * Add(struct Employee *ptr)
{
    struct Employee *newnode=NULL,*temp=NULL;

    newnode=calloc(1,sizeof(struct Employee));
    if(newnode==NULL)
    {
        printf("Node not Created\n");
    }
    else
    {
        printf("Enter The Employ ID : ");
        scanf("%d",&newnode->empid);
        printf("Enter The Name : ");
        scanf("%s",newnode->name);
        if(ptr==NULL)
        {
            // list is empty
            ptr=newnode;
            newnode->link=ptr;
            // newnode->link=newnode
        }
    }
}
```

```

else
{
    for(temp=ptr;temp->link!=ptr;temp=temp->link);

    temp->link=newnode;
    newnode->link=ptr;
    /**
    newnode->link=temp->link
    temp->link=newnode
    *****/
}
}

return ptr;
}

```


Program: Write a program to delete data in circular linked list.

struct Employee * Del(struct Employee *ptr)

{

int data;

struct Employee *temp=NULL,*temp1=NULL;

printf("Enter The Data To Be Deleted : ");

scanf("%d",&data);

if(ptr==NULL)

{

printf("CLL is empty\n");

}

else if(data==ptr->empid)

{

//matching with first node data

temp=ptr;

ptr=ptr->link;

if(ptr==temp) **// only single node**

ptr=NULL;

else

{

// making second node or next node address to ptr

for(temp1=temp;temp1->link!=temp;temp1=temp1->link);

temp1->link=ptr;

}

// lastnode link should be updated with latest first node address

```
    free(temp);
    temp=NULL;
}
else
{
    temp=ptr;
    temp1=ptr->link;
    while(temp1!=ptr && data != temp1->empid)
    {
        temp=temp1;
        temp1=temp1->link;
    }
    if(temp1==ptr)
    {
        printf("&d is not Found\n",data);
    }
    else
    {
        temp->link=temp1->link;
        free(temp1);
        temp1=NULL;
    }
}
return ptr;
}
```

Program: Write a program to print the data in circular linked list.

void Display(struct Employee *ptr)

```
{
    struct Employee *temp=NULL;
    if(ptr==NULL)
    {
        printf("List is Empty\n");
    }
    else
    {
        temp=ptr;
        do
        {
            printf("Employ Id : %d %s\n",temp->empid);
            printf("Employ Name is : %s,"temp->name);
            temp=temp->link;
        }while(temp!=ptr);
    }
}
```

Assignment-1

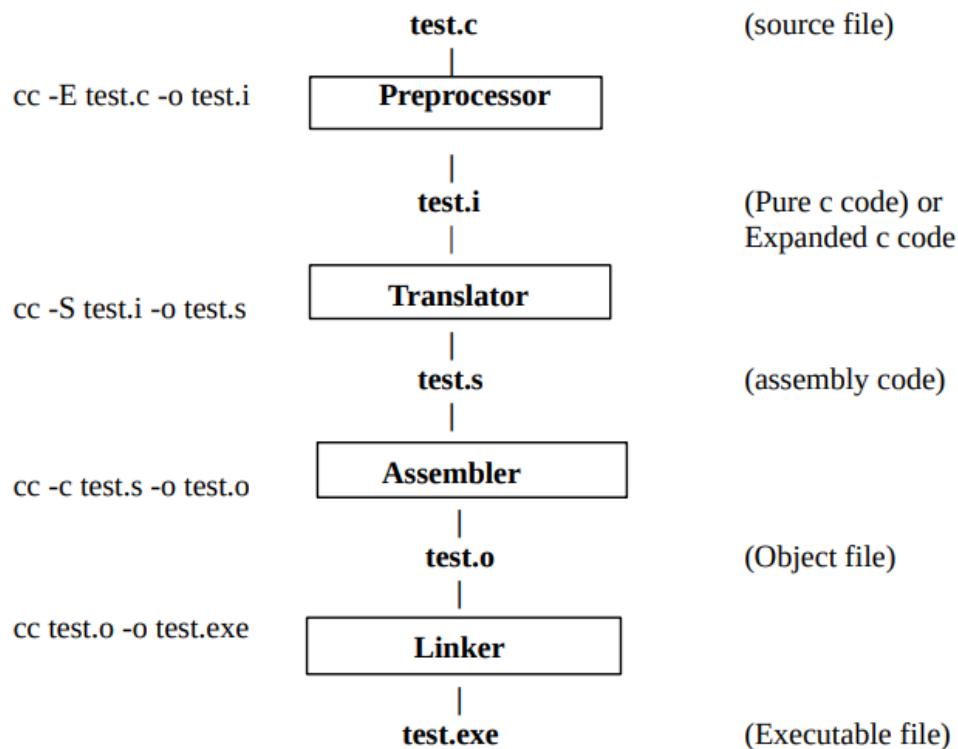
<Question_1> What is compiler? What are the compilation stages?

In computing, a compiler is a computer program that translates computer code written in one programming language (the source language) into another language (the target language). The name "compiler" is primarily used for programs that translate source code from a high-level programming language to a lower-level language (e.g., assembly language, object code, or machine code) to create an executable program.

There are four phases for a C program to become an executable:

- Pre-processing
- Translating
- Assembling
- Linking

Compilation stages :



Pre-processing:

- Header files are replaced by its contents by pre-processor (in place of #include statement, files contain will be replaced and #include statements will be removed)
- comments are removed
- macro replacement is done by pre-processor
- conditional compilation

output of pre-processor is called pure c code (expanded c code). Because it will be having only c statement, pre-processor directive and comments are removed.

NOTE - Header files contain declaration of library function but function body is present in

library file of that function.

NOTE - Header files are written at the top of the program because it contains declaration of library function which are used in our program, and function must be declared before they are called.

NOTE - It is not necessary to include header files, but then will have to define all the function we use in our program by ourself.

Translator:

- Check for syntax error
- if any error is not found, then converts source code into assembly code.

Assembler:

- convert assembly code into binary code.

Linker:

➤ Links calling function to called function (called function mapping)

- creates `_start` function, which called main function in our program and `_start` also contain proper exit procedure.

NOTE - Giving intermediate filename extension like. i. o. s is necessary, as they will use by compiler as input. As GCC is not extension independent, it doesn't accept input files

with any other extension than it should be having.

NOTE - In GCC function declaration is not strict (undeclared function is not an error in gcc).

ISO-C has made it a strict rule that any function used in the program must be declared. No declaration of function results in error.

<Question_2> Is header file inclusion is must? Explain.

Header files contain declaration of library function but function body is present in library file of that function. Header files are written at the top of the program because it contains declaration of library function which are used in our program, and function must be declared before they are called. It is not necessary to include header files, but then will have to define all the function we use in our program by ourself.

<Question_3> Write the types of errors. Give example.

Error is an illegal operation performed by the user which result in abnormal working of program.

- 1) Compile Time Error.
- 2) Run Time Error.

1) Compile Time Error:

Prepressing Error: invalid header files name.

invalid comments (nested comments).

Translating Error: syntax error.

Assembling Error: not present (translator converts source file to assembly file, so if there is no error in translating stages, its output assembly code will be error free. That assembly code is used by assembler to generate object code. So, assembly is always error free).

Linking Error: no main () defined.

function called but not defined.

2) Run Time Error:

- Segmentation fault (due to trying to access or modify that portion of memory to which our application doesn't have permission to)
- floating point exception (dividing by 0).

<Question_4> Object files (test.o) and executable file (test.exe) are binary files. What is the difference between?

Object File (test.o)	Executable File (test.exe)
1) The Object files is that is generated after compiling the source code.	1) In executable file is a file that is generated after linking a set of object file together using linker.
2) Test.o is machine friendly file (understandable).	2) Test.exe is OS friendly i.e (operating system under stable file).

<Question_5> If function main is not written in a C program, then what happens? How to create a valid C application without main, explain.

We can write c program without using main () function. To do so, we need to use #define pre-processor directive. The C pre-processor is a microprocessor that is used by compiler to transform your code before compilation. It is called micro pre-processor because it allows us to add macros.

<Question_6> What are tokens in C? Explain with example.

Tokens are the smallest elements of a program, which are meaningful to the compiler.

The following are the types of tokens: Keywords, Identifiers, Constant, Strings, Operators, etc.

<Question_7> 7. What is function declaration, definition, call? demonstate with example.

- 1) Function declaration tells the compiler about a function name and how to call function.
- 2) A function is a group of statement that together perform a task. Function definition provides a actual body of function.
- 3) While creating a c function, you give a definition of what the function has to do. To user a function, you will have to call that function to perform the defined task.
- 4) When a program calls a function, the program control is transferred to the called function.


```

#include<stdio.h>

int check_max(int,int);           //Function Deceleration

int main()
{
    int a=100;
    int b=200;
    int ret;
    ret=check_max(a,b);           // Calling Function
    printf("Max Value is : %d\n",ret);
}

int check_max(int a,int b)       //Called Function
{
    int result;
    if(a>b)
        result=a;
    else
        result=b;               // Function Body
    return result;
}

```

<Question_8> Compare between putchar, puts and printf.

putchar()	puts ()	printf()
1) putchar() prints only character.	1) Puts print string only (character string).	1) It is most commonly used function, which prints all type of data.

<Question_9> Why C language is preferred in Embedded system?

- 1) C is middle level language which has simplicity of high level and power of low-level language.
- 2) Executable binary compiled in C compiler are compact (smaller) in size than executable (binary) compiled in C++ and java compilers.
- 3) Platform independent. i.e., java needs JRE, python need PyCharm but c does not need any platform.

<Question_10> List the features of C programming language.

- Procedural Language.
- Fast and Efficient.
- Modularity.
- Statically Type.
- General-Purpose Language.
- Rich set of built-in Operators.
- Libraries with rich Functions.
- Middle-Level Language.
- Portability.
- Easy to Extend.

Assignment-2

<Question_1> Convert the following binary numbers to a decimal value:

a) $(011011)_2 = ()_{10}$

b) $(101001)_2 = ()_{10}$

Solution:

a) $(011011)_2 = (27)_{10}$

b) $(101001)_2 = (41)_{10}$

<Question_2> Convert the following decimal numbers to binary:

a) $(257)_{10} = ()_2$

b) $(2048)_{10} = ()_2$

c) $(87)_{10} = ?$

d) $(-1)_{10} = ?$

e) $(-45)_{10} = ()$

Solution:

a) $(257)_{10} = (100000001)_2$

b) $(2048)_{10} = (100000000000)_2$

c) $(87)_{10} = (01100111)_2$

d) $(-1)_{10} = (11111111)_2$

e) $(-45)_{10} = (010011)_2$

<Question_3> What will be the equivalent in hexadecimal for the given binary data:

a) 11101101101

b) 0100010010111

Solution:

a) $(0011\ 1011\ 01101)_2 = (76D)_{16}$

b) $(0100010010111)_2 = (897)_{16}$

<Question_4> Convert the following hexadecimal numbers to a binary number:

a). 0XABCD

b). 0X0F1

Solution:

a) $(0xABCD)_{16} = (1010\ 1011\ 1100\ 1101)_2$

b) $(0x0F1)_{16} = (0000\ 1111\ 0001)_2$

<Question_5> Convert the following octal number to a binary number:

a) 0111

b) 0777

c)074

Solution:

a) $(0111)_8 = (001001001)_2$

b) $(0777)_8 = (111111111)_2$

c) $(074)_8 = (111100)_2$

<Question_6> Convert the following binary number to an octal number:

a) 100111010

b) 010000001

Solution:

a) $(100111010)_2 = (100\ 111\ 010)_8$

b) $(010000001)_2 = (010\ 000\ 001)_8$

<Question_7> Convert the following hexadecimal number to an octal number:

a) 0X1BC

b) 0XFFFF

Solution:

a) $(0x1BC) = (0001\ 1011\ 1100)_8$

b) $(0xFFFF) = (1111\ 1111\ 1111)_8$

<Question_8> If the variable is declared as given below, then what will be the exact binary presentation in memory, when the program is being executed?

a) char ch = '6';

b) char ch = 'a';

c) char ch = '*';

d) char ch = '1';

e) char ch = 98;

f) char ch = 127;

g) char ch = 128;

h) unsigned char u = 127;

i) unsigned char u = 128;

j) unsigned char u = 255;

k) unsigned char u = 256;

Solution:

a) $\text{char ch} = '6' = (\text{ASCII} - 54) = (110110)_2$

b) $\text{char ch} = 'a' = (\text{ASCII} - 97) = (1100001)_2$

c) $\text{char ch} = '*' = (\text{ASCII} - 42) = (101010)_2$

d) $\text{char ch} = '1' = (\text{ASCII} - 1) = (00000001)_2$

e) $\text{char ch} = 98 = (\text{ASCII} - 98) = (1100010)_2$

f) $\text{char ch} = 127 = (01111111)_2$

g) $\text{char ch} = 128 = (1000\ 0000)_2$

h) $\text{unsigned char u} = 127 = (0111\ 1111)_2$

i) $\text{unsigned char u} = 128 = (1000\ 0000)_2$

j) unsigned char u=255 = $(0111\ 1111)_2$

k) unsigned char u=256 = $(0000\ 0000)_2$

<Question_9> If the following integer variables are declared, then what will be exact picture of memory occupied.

- a) short int s=50;
- b) short int s=-5;
- c) short int s=32767;
- d) short int s=32768;

Solution:

- a) short int s=50 = $(110010)_2$
- b) short int s=-5 = $(011)_2$
- c) short int s=32767 = $(0111\ 1111\ 1111\ 1111)_2$
- d) short int s=32768 = $(1000\ 0000\ 0000)_2$

<Question_10> If the integer variables are declared as given below, then predict the binary presentation in memory.

- a) int i='1';
- b) int j=520;
- c) int k=65536;
- d) int l=-2;
- e) m=-50;

Solution:

- a) int i='1' = (ASCII – 49) = $(110001)_2$
- b) int j=520 = $(01000001000)_2$
- c) int k=65536 = $(0001\ 0000\ 0000\ 0000\ 0000)_2$
- d) int l=-2 = $(11111111111111111111111111111110)_2$
- e) int m=-50 = $(111111111111111111111111111101110)_2$

Assignment-3

<Question_1> What is binary equivalent of

- i) char ch='5';
- ii) short int s = -520;
- iii) int i = -2;
- iv) float f=45.6;
- v) double d= -45.6;

Solution:

- i) char ch='5' = (ASCII – 52) = (110100)₂
- ii) short int s = -520; = (1111 1101 1111 1000)₂
- iii) int i = -2 = (1111 1111 1111 1111 1111 1111 1111 1110)₂
- iv) float f=45.6 = (010000100011011001111001100110)₂
- v) double d= -45.6 = (110000000100)₂

<Question_2> What is the range of char, short int, int, long int, float.

Solution:

Data Type	Range	Size
Signed char	-128 to 127	1 byte
Unsigned char	0 to 255	1 byte
Signed short int	-32768 to 32767	2 byte
Unsigned short int	0 to 65535	2 byte
Signed int	-2g to 2g (-2,147,483,648 to 2,147,483,647)	4 byte
Unsigned int	0 to 4g (0 to 4,294,967,295)	4 byte
Signed long int	9223372036854775808 to 9223372036854775807	8 byte (4 for 32 bit)
Unsigned long int	0 to 18446744073709551615	8 byte
Float	1.2E-38 to 3.4E+38	4 byte
Double	2.3E-308 to 1.7E+308	8 byte
Long Double	3.4E-4932 to 1.1E+4932	10 byte

<Question_3>

```
main() {  
    unsigned char a;  
    a = 0xFF + 1;  
    printf("%u\n", a);  
}
```

Solution:

test.c: In function ‘main’:

test.c:5:4: warning: unsigned conversion from ‘int’ to ‘unsigned char’ changes value from ‘256’ to ‘0’ [-Woverflow]

```
5 | a=0xFF+1;  
  |      ^~~~
```

<Question_4>

```
main()  
{  
    int x=10, y=20, z=5, i;  
    i =x<y<z;  
    printf("%d\n", i);  
}
```

Solution:

a=1

<Question_5>

```
#include<stdio.h>

int main()
{
    unsigned int x=022;
    unsigned int y=0x22;
    unsigned int z=022;
    printf(“%d\n”,x+y+z);
}
```

Solution:

Output=70

<Question_6>

```
#include<stdio.h>

int main()
{
    int a=123;
    int b=456;
    a^=b^=a^=b;
    printf(“%d %d”,a,b);
}
```

Solution:

a=456

b=123

<Question_7> List of number of unary operators.

Solution:

- a) Indirection operator (*)
- b) Address-of operator (&)
- c) Unary plus operator (+)
- d) Unary negation operator (-)
- e) Logical negation operator (!)
- f) One's complement operator (~)
- g) Prefix increment operator (++)
- h) Prefix decrement operator (--)
- i) sizeof operator

<Question_8>

```
#include<stdio.h>

int main()
{
    int a=10,b;
    b=a>=5?100:200;
    printf("b=%d\n",b);
}
```

Solution:

b=100

<Question_9>

```
#include<stdio.h>

int main()
{
    printf("a=%d\n", -1<<4);
    printf("b=%d\n", -1 >>2);
    printf("c=%x\n", 1<<10);
    printf("d=%x\n", 0xFF<<4);
    printf("e=%o\n", 034<<2);
}
```

Solution:

```
a=-16
b=-1
c=400
d=ff0
e=160
```

<Question_10>

```
#include<stdio.h>

int main()
{
    printf("a=%d\n", 1<<4>>4);
    printf("b=%d\n", 1<<31>>30);
}
```

Solution:

```
a=1
b=-2
```

<Question_11>

```
#include<stdio.h>

int main()
{
    double f=33.3;
    int x;
    x=(f>33.3)+(f==33.3);
    printf("x=%d\n",x);
}
```

Solution:

x=1

<Question_12>

```
#include<stdio.h>

int main()
{
    unsigned int res;
    res=(64 >>(2+1-2))&(~(1<<2));
    printf("res=%d\n",res);
}
```

Solution:

res=32

<Question_13>

```
#include<stdio.h>

int main()
{
    int z,x=5,y=-10,a=4,b=2;
    z=x++ - --y * --b/a;
    printf("res=%d\n",z);
}
```

Solution:

res=7

<Question_14>

```
#include<stdio.h>

int main()
{
    printf("a=%lu\n",sizeof('\n'));
    printf("b=%lu\n",sizeof("5"));
    printf("c=%lu\n",sizeof(5.0));
    printf("d=%lu\n",sizeof(23.4f));
    printf("e=%lu\n",sizeof("23.4f"));
}
```

Solution:

a=4

b=2

c=8

d=4

e=6

<Question_15>

```
#include<stdio.h>

int main()
{
    printf("a=%d\n",(0x11>11)&&(11>011));
    printf("b=%d\n",sizeof(-1)>10);
    printf("c=%d\n",23.0>23);
}
```

Solution:

```
a=1
b=0
c=0
```

<Question_16>

```
#include<stdio.h>

int main()
{
    int a=-1,b=0, c=5;
    char ch='a';
    printf("a=%d\n", (ch>=97)&&(ch<=122));
    printf("b=%d\n", (a==0) || (b==0) || (c==0));
    printf("c=%d\n", (a<b)&&(b<c));
}
```

Solution:

```
a=1
b=1
c=1
```

<Question_17>

```
#include<stdio.h>

int main()
{
    printf("a=%d\n",-1&3456);
    printf("b=%d\n", -1| 456);
    printf("c=%d\n", -1 ^ 789);
    printf("d=%d\n", 0&789);
    printf("e=%d\n",0&234);
}
```

Solution:

a=3456

b=-1

c=-790

d=0

e=0

<Question_18>

```
#include<stdio.h>

int main()
{
    int bitMask=1;
    printf("a=%d\n", 50 & bitMask);
    bitMask=32;
    printf("b=%d\n",50 & bitMask);
    printf("c=%d\n",60 & bitMask);
    printf("d=%d\n",70 & bitMask);
    bitMask=512;
    printf("e=%d\n",525&bitMask);
}
```

Solution:

a=0

b=32

c=32

d=0

e=512

<Question_19>

```
#include<stdio.h>

int main()
{
    int var=50,bit=3;
    printf("a=%d\n",var&(1<<3));
    printf("b=%d\n",var&(1<<4));
    printf("c=%d\n",var&(1<<5));
}
```

Solution:

a=0
b=16
c=32

<Question_20>

```
#include<stdio.h>

int main()
{
    int a=2,b=4,c=5;
    a+=b*=c-=10;
    printf("a=%d b=%d c=%d\n",a,b,c);
}
```

Solution:

a=-18 b=-20 c=-5

Assignment-4

<Question_1> Write a program to print the type(category) of user supplied char from stdin. The categories are Numeric/Uppercase/lowercase/special character.

Solution:

```
#include<stdio.h>
#include<ctype.h>
int main()
{
    char ch;
    puts("Enter a char:");
    ch=getchar();
    isupper(ch)?printf("ch = %c is Uppercase Character\n",ch):
    islower(ch)?printf("ch = %c is Loweercase Character\n",ch):
    isdigit(ch)?printf("ch = %c is Numeric character\n",ch):
    printf("ch = %c is Special character\n",ch);
}
```

<Question_2> Write a program to Convert temperature from o C to o F or vice versa. Your program should show the output as given below. / a.out

1.for centigrade to Fahrenheit

2. for Fahrenheit to centigrade

Enter your choice: 1

Enter temperature in Celsius: 37

37 o C = 98.6 o F

Solution:

```
#include<stdio.h>

int main()
{
    int choice;
    float data,cels,fahr;
    printf("Choice 1: To convert Data Celcius to Fahrenheit.\n");
    printf("Choice 2: To convert Data Fahrenheit to Temp.\n");
    INPUT: printf("Enter Your Choice: ");
    scanf("%d",&choice);
    switch(choice)
    {
        case 1 :
            printf("Enter Data in celsius : ");
            scanf("%f",&data);
            fahr=(data*9/5) + 32;
            printf("Data in fahrengait is %f \n:",fahr);
            break;
```

case 2 :

```
printf("Enter Data in fahrengheit: ");  
scanf("%f",&data);  
cels=((data-32)*5)/9;  
printf("Data in celcius is %f\n:",cels);  
break;
```

default:

```
printf("Invalid Choice\n");  
goto INPUT;
```

```
}
```

```
}
```

<Question_3> Write a program to find the highest of three integers.

Note: If two are equal and higher than the third or appropriate result should print on screen. all the three variables are equal.

Solution:

```
#include<stdio.h>

int main()
{
    int data,rev=0,high=0;
    int n;
    printf("Enter the data: ");
    scanf("%d",&data);
    n=data;
    while(n!=0)
    {
        rev=n%10;
        if(rev>high)
            high=rev;
        n=n/10;
    }
    printf("Highest Number is: %d is %d\n",data,high);
}
```

<Question_4> Write a program to test the status of a bit in a supplied integer.

NOTE: user should supply the integer data and bit-position to test.

Solution:

```
#include<stdio.h>

int main()
{
    int data,pos;
    printf("Enter the number : ");
    scanf("%d",&data);
    printf("Enter the position : ");
    scanf("%d",&pos);
    ((data&(1<<pos))==0)?printf("%d bit is clear\n",pos):
        printf("%d bit is set\n",pos);
}
```

<Question_5> Write a program to set/clear/toggle the supplied bit in a supplied integer.

Solution:

```
#include<stdio.h>

int printoutput(int);
int setbit(int,int);
int clearbit(int,int);
int togglebit(int,int);
void printMENU();

int main()
{
    int data,bit;
    char choice;
    printf("Enter The data : ");
    scanf("%d",&data);

    while(1)
    {
        printMENU();
        printf("Enter Your Choice : ");
        scanf("%c",&choice);
        printf("Enter the Bit : ");
        scanf("%d",&bit);
```

```
switch(choice)
{
    case 's' :
        printoutput(setbit(data,bit));
        break;

    case 'c' :
        printoutput(clearbit(data,bit));
        break;

    case 't' :
        printoutput(togglebit(data,bit));
        break;

    default:
        printf("Invalid Choice\n");
        break;
}
}
```


int printoutput(int data)

```
{
    printf("\t Data in Decimal : %d\n",data);
    printf("\t Data in Binary : ");
    for(int i=31;i>0;i++)
    {
        printf("%d",(data>>i)&1);
        if(i%8==0)
            printf(" ");
    }
}
```

int togglebit(int data, int bit)

```
{
    data=data^(1<<bit);
    return data;
}
```

int clearbit (int data, int bit)

```
{
    data=data&~(1<<bit);
    return data;
}
```

```
int setbit(int data, int bit)
```

```
{  
    data=data|(1<<bit);  
    return data;  
}
```

```
void printMENU()
```

```
{  
    printf("\nMENU\n\n");  
    printf("s:\tSet the bit\n");  
    printf("c:\tClear the bit\n");  
    printf("t:\tToggle the bit\n");  
}
```

<Question_6> Write a program to swap the two integers.

i)without using third(temporary) variable

ii) you can use temporary variable.

Solution:

i)without using third(temporary) variable

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int a;
```

```
    int b;
```

```
    printf("Enter 1st number : ");
```

```
    scanf("%d",&a);
```

```
    printf("Enter 2nd number : ");
```

```
    scanf("%d",&b);
```

```
    a^=b^=a^=b;
```

```
    printf("a=%d b=%d\n",a,b);
```

```
}
```

ii) you can use temporary variable.

```
#include<stdio.h>

int main()
{
    int a;
    int b;
    int temp;
    printf("Enter 1st number : ");
    scanf("%d",&a);
    printf("Enter 2nd number : ");
    scanf("%d",&b);
    temp=a;
    a=b;
    b=temp;
    printf("a=%d b=%d\n",a,b);
}
```

<Question_7> Convert the case of a supplied alphabet. If user supplied lowercase, then convert to uppercase and vice-versa.

Solution:

```
#include<stdio.h>

int main()
{
    char ch;
    puts("Enter the Character : ");
    ch=getchar();
    ((ch>=65) && (ch<=90)) ? (ch+=32) : (ch-=32);
    printf("ch = %c\n",ch);
}
```

<Question_8> List the unary operators. Also mention the syntax.

Solution:

- | | | |
|-------------------------------|---|---------------------------|
| 1) Unary '+' | - | c=a+b; |
| 2) Unary '-' | - | c=a-b; |
| 3) Increment operator '++' | - | c=a++; |
| 4) Decrement operator '--' | - | c=a--; |
| 5) Address of operator '&' | - | printf("%d",&a); |
| 6) sizeof operator 'sizeof()' | - | sizeof(int/float/double); |
| 7) Dereferencing operator '*' | - | |
| 8) Logical NOT '!' | - | c!=a; |
| 9) Bitwise not '~' | - | c=~b; |

<Question_9> List the bitwise operators with syntax.

Solution:

- | | | |
|-------|----------------------|---------------------------------|
| 1) & | - Bitwise AND | output – operand 1 & operand 2 |
| 2) | - Bitwise OR | output – operand 1 operand 2 |
| 3) ~ | - Bitwise Compliment | output – ~ operand 1 |
| 4) << | - Left Shift | output – operand 1 << operand 2 |
| 5) >> | - Right Shift | output – operand 1 >> operand 2 |
| 6) ^ | - Bitwise Ex-OR | output – operand 1 ^ operand 2 |

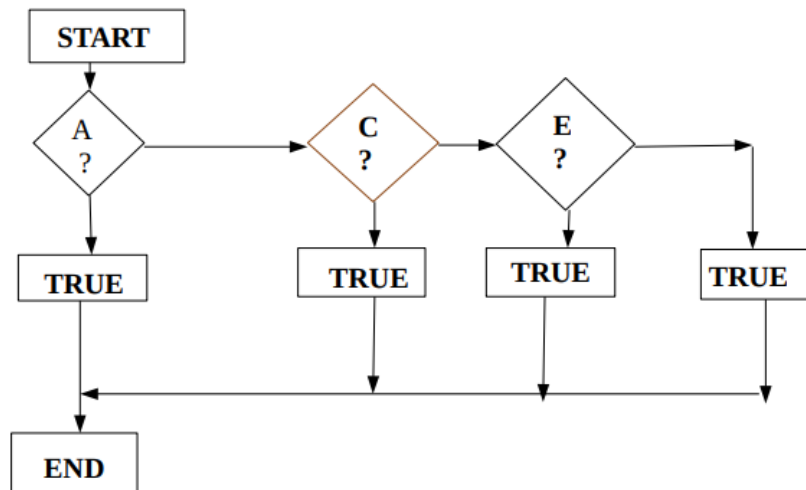
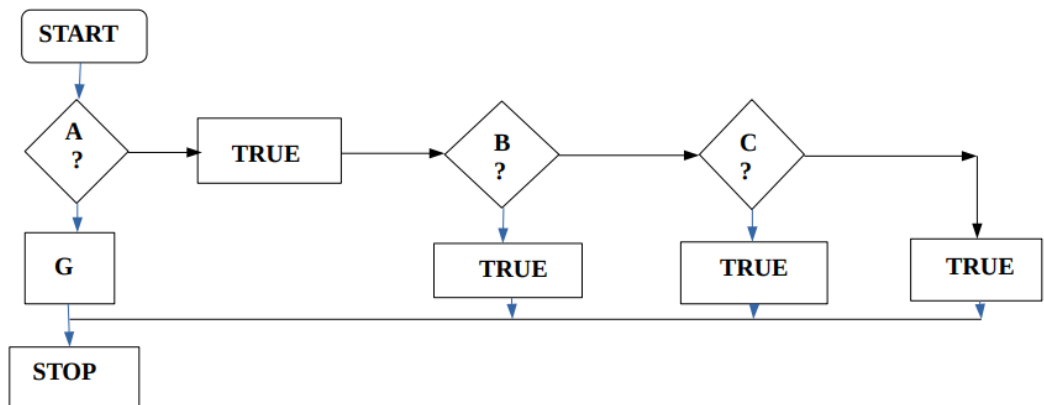
<Question_10> Draw flow diagram of the following expressions.

i) $A?B:C?D:E:F:G;$

ii) $A?B:C?D:E?F:G;$

iii) $A?B?C:D?E:F:G;$

Solution:



Assignment-5

<Question_1> Draw flow diagram “to Count the number of digits” in a supplied integer. Write a C program based on your flow diagram.

Solution:

```
#include<stdio.h>

int main()
{
    int data,i,count=0;
    printf("Enter data :");
    scanf("%d",&data);
    if(data==0)
        printf("count=1\n");
    for(i=data;i;i=i/10)
        count++;
    printf("count = %d\n",count);
}
```


<Question_2> Draw flow diagram “to find the Highest digit” in a supplied integer. Write a C program for the same.

Solution:

```
#include<stdio.h>

int main()
{
    int data, temp = 0, large = 0;
    printf("Enter A Number:\t");
    scanf("%d", &data);
    for(;data > 0;data = data/10)
    {
        temp = data % 10;
        if(temp > large)
        {
            large = temp;
        }
    }
    printf("Largest Digit in the Integer:%d\n", large);
    return 0;
}
```

<Question_3> Write a program to find the 2nd highest digit in a supplied integer. Note: Use a single loop.

Solution:

```
#include<stdio.h>

int main ()
{
    int data,rem,largest=0,Sec_large=0;
    printf("Enter the Number :");
    scanf("%d",&data);
    while(data>0)
    {
        rem=data%10;
        if(largest<rem)
        {
            Sec_large=largest;
            largest=rem;
        }
        else if(rem>=Sec_large)
            Sec_large=rem;
        data=data/10;
    }
    printf("The Second Largest Digit is %d\n", Sec_large);
    return 0;
}
```

<Question_4> Write a program to verify the given integer is palindrome or not.

Solution:

```
#include<stdio.h>

int revDigits(int);

int main()
{
    int data;
    printf("Enter data to test palindrome: ");
    scanf("%d",&data);

    if(revDigits(data)==data)
        printf("Palindrome\n");
    else
        printf("Not Palindrome\n");
}

int revDigits(int data)
{
    int rev = 0;
    while(data)
    {
        rev=rev*10+data%10;
        data/=10;
    }
    return rev;
}
```

<Question_5> W.A.P to calculate sum-of-Odd-digits in an integer.

Solution:

```
#include<stdio.h>

int sumofodddigits(int);

int main()
{
    int data;
    printf("Enter a Number: ");
    scanf("%d",&data);
    printf("Sum of Odd Digits: %d\n",sumofodddigits(data));
}

int sumofodddigits(int data)
{
    int r,sum=0;
    while(data>0)
    {
        r=data%10;
        data=data/10;
        if (r%2==1)
            sum=sum+r;
    }
    return sum;
}
```

<Question_6> W.A.P to verify the digits in an integer are in descending order or not.

Solution:

```
#include<stdio.h>

int main()
{
    int data,preDig,curDig,temp=0;
    printf("Enter the data: ");
    scanf("%d",&data);
    preDig=data%10;
    data=data/10;
    while(data>0)
    {
        curDig=data%10;
        if(curDig<preDig)
        {
            temp=1;
            break;
        }
        preDig=curDig;
        data=data/10;
    }
    if(temp==0)
        printf("Decresing\n");
    else
        printf("Not Decreasing\n");
}
```

<Question_7> Verify the digits in a supplied integer are sorted or not.

Solution:

```
#include<stdio.h>

int testDec(int);
int testAsc(int);

int main()
{
    int data;
    int v1,v2,rem=0;
    printf("Enter The Data : ");
    scanf("%d",&data);
    if(testAsc(data) || testDec(data))
        printf("Sorted\n");
    else
        printf("Not Sorted\n");
}
```

```
int testAsc(int data)
{
    int preDig,curDig,temp=0;
    curDig=data%10;
    data=data/10;
    while(data>0)
    {
        preDig=data%10;
        if(preDig>curDig)
        {
            temp=0;
            break;
        }
        curDig=preDig;
        data=data/10;
    }
    if(data==0)    return 1;
    else          return 0;
}
```

```
int testDec(int data)
{
    int preDig,curDig,temp=0;
    curDig=data%10;
    data=data/10;
    while(data>0)
    {
        preDig=data%10;
        if(preDig<curDig)
        {
            temp=0;
            break;
        }
        curDig=preDig;
        data=data/10;
    }
    if(data==0)    return 1;
    else          return 0;
}
```


Assignment-6

<Question_1>

```
#include<stdio.h>

int main()
{
    int p;
    for(p=1;p<10;p--;p=p+2)
        printf("Hello");
        printf("Hi");
}
```

Solution:

Infinite loop of Hello

<Question_2>

```
#include<stdio.h>

int main()
{
    int i;
    for (i=5; ++i; i -=3)
        printf("%d \n",i);
}
```

Solution:

6
4
2

<Question_3>

```
#include<stdio.h>

void main()
{
    int i=0,j=0;
    for(i=0;i<5;i++)
    {
        for(j=0;j<4;j++)
        {
            if(i>1)
                continue;
            printf(" Hi \n");
        }
    }
}
```

Solution:

```
Hi
Hi
Hi
Hi
Hi
Hi
Hi
Hi
```

<Question_4>

```
#include<stdio.h>

int main()
{
    int a=0,i=0,b;
    for(i=0;i<5;i++)
    {
        printf("in loop: i=%d\n",i);
        a++;
        if(i==3)
            break;
    }
}
```

Solution:

in loop: i=0

in loop: i=1

in loop: i=2

in loop: i=3

<Question_5>

```
#include<stdio.h>

int main()
{
    int i = 0;
    for (i = 0; i < 5; i++)
        if (i < 4)
        {
            printf("Hello\n");
            break;
        }
}
```

Solution:

Hello

<Question_6>

```
#include<stdio.h>

int main()
{
    int i=0;
    for(i=0; i<20; i++)
    {
        switch(i)
        {
            case 0: i+=5;
            case 1: i+=2;
            case 5: i+=5;
            default: i+=4;
            break;
        }
        printf("%d\n", i);
    }
    return 0;
}
```

Solution:

16

21

<Question_7>

```
#include<stdio.h>

int main()
{
    int n;
    for(n = 7; n!=0; n--)
        printf("n = %d\n", n - -);
    return 0;
}
```

Solution:

Infinite Loop of counting numbers

<Question_8>

```
#include<stdio.h>

int main()
{
    unsigned int i=10;
    while(i-- >= 0)
        printf("i=%u \n",i);
    return 0;
}
```

Solution:

Infinite loop

<Question_9>

```
#include<stdio.h>

int main()
{
    int i = 1;
    do
    {
        printf("%d\n", i);
        i++;
        if (i < 15)
            continue;
    } while (0);
    return 0;
}
```

Solution:

1

<Question_10>

```
#include<stdio.h>

int main()
{
    int i, j, sum = 0,n;
    for(i = 1;i<=n;i++)
    for(j=i;j<=i;j++)
    sum=sum+j;
    printf("%d\n",sum);
}
```

Solution:

0

<Question_11>

```
#include<stdio.h>

int main()
{
    int i = 0;
    do
    {
        i++;
        if (i == 2)
            continue;
        printf("In while loop ");
    } while (i < 2);
    printf("%d\n", i);
}
```

Solution:

In while loop 2

<Question_12>

```
#include<stdio.h>

int main()
{
    int i = 0, j = 0;
    for (i; i < 2; i++)
    {
        for (j = 0; j < 3; j++)
        {
            printf("inner loop\n");
            break;
        }
        printf("outer loop\n");
    }
    printf("after loop\n");
}
```

Solution:

inner loop
outer loop
inner loop
outer loop
after loop

<Question_13>

```
#include<stdio.h>

int main()
{
    int i = 0;
    char c = 'a';
    while (i < 2)
    {
        i++;
        switch (c)
        {
            case 'a':
                printf("%c ", c);
                break;
                break;
        }
    }
    printf("after loop\n");
}
```

Solution:

a a after loop

<Question_14>

```
#include<stdio.h>

void main()
{
    int var=1;
    switch(var<<2+var)
    {
        default:printf("Chennai\n");
        case 4: printf("Hyderabad\n");
        case 5: printf("Vector\n");
        case 8: printf("Bangalore\n");
    }
}
```

Solution:

Bangalore

<Question_15>

```
#include<stdio.h>

int main()
{
    int x = 5, y = 2 ;
    char op = '*' ;
    switch ( op )
    {
        default : x += 1 ;
        case '+' : x += y ;
        case '-' : x -= y ;
    }
    printf("%d",x);
}
```

Solution:

6

<Question_16>

```
#include<stdio.h>

int main()
{
    int n=2, sum=1;
    switch(n)
    {
        case 2:sum=sum+2;
                printf("sum=%d\n",sum);
        case 3:sum*=2;
                printf("sum=%d\n",sum);
                break;
        default : sum=0;
                printf("sum=%d\n",sum);
    }
}
```

Solution:

```
sum=3
sum=6
```

<Question_17>

```
#include<stdio.h>

int main()
{
    int i=9;
    switch(i)
    {
        printf("hello");
        case 9 : printf("abc");
        break;
        default : printf("def");
    }
}
```

Solution:

abc

<Question_18>

```
#include<stdio.h>

int main()
{
    int i = 65;
    switch(i)
    {
        case 65:
            printf("Integer 65");
            break;
        case 'A':
            printf("Char 65");
            break;
        default:
            printf("Bye");
    }
}
```

Solution:

test.c: In function ‘main’:

test.c:10:1: error: duplicate case value

10 | case 'A':

| ^~~~

test.c:7:1: note: previously used here

7 | case 65:

<Question_19>

```
#include<stdio.h>

int main()
{
    int i = 0;
    while(i < 3, i = 0, i < 5)
    {
        printf("%d ",i++);
    }
}
```

Solution:

Infinite loop with 0

<Question_20>

```
#include<stdio.h>

int main()
{
    int x=011,i;
    for(i=0;i<x;i+=3)
    {
        printf("Start ");
        continue;
        printf("End");
    }
    return 0;
}
```

Solution:

Start Start Star

Assignment-7

<Question_1> Write a program to reverse bits of a given integer number.

Solution:

```
#include<stdio.h>

int main()
{
    int data;
    int bit;
    int i,j;
    printf("Enter The data : ");
    scanf("%d",&data);
    for(bit=31;bit>=0;bit--)
        printf("%d",(data>>bit)&1);
    printf("\n");
    for(i=31,j=0;i>j;i--,j++)
    {
        if(((data>>i)&1) != ((data>>j)&1))
        {
            data=data^(1<<i);
            data=data^(1<<j);
        }
    }
    for(bit=31;bit>=0;bit--)
        printf("%d",(data>>bit)&1);
    printf("\n");
}
```

<Question_2> Write a program to count pair of set bits in a given number.

Solution:

```
#include<stdio.h>

int setBitPairs(int);

int main()
{
    int data;
    printf("Enter The Data : ");
    scanf("%d",&data);
    printf("Set Bit Pairs are : %d\n",setBitPairs(data));
}

int setBitPairs(int data)
{
    int cnt=0,bit=31;
    while(bit>0)
    {
        if((((data>>bit)&1)==1)&&(((data>>--bit)&1)==1))
        {
            cnt++;
            --bit;
        }
        else
            --bit;
    }
    return cnt;
}
```

<Question_3> Write a program to print longest series of 1's in given integer set bits.

Solution:

```
#include<stdio.h>

int main()
{
    int bin,data,cnt=0,flag=0,bit;
    printf("Enter The Data : ");
    scanf("%d",&data);
    for(bit=31;bit>=0;bit--)
    {
        bin=(data>>bit)&1;
        if(bin==1)
        {
            cnt++;
        }
        if(cnt==2)
        {
            flag++;
            cnt=0;
        }
        if(bin==1)
        {
            continue;
        }
    }
    for(bit=31;bit>=0;bit--)
    {
```

```
        printf("%d",(data>>bit)&1);
        if(bit%8==0)
            printf(" ");
    }
    printf("\n");
    printf("Count is : %d\n",flag);
}
```

<Question_4> Write a program to make arrangement to list the numbers the sum of digits, reduce to single digit is 9.

Solution:

```
#include<stdio.h>

int main()
{
    int min,max;
    int data;
    int sum=0;
    int temp;
    printf("Enter The Min Digit : ");
    scanf("%d",&min);
    printf("Enter The Mix Digit : ");
    scanf("%d",&max);
    for(data=min;data<=max;data++)
    {
        temp=data;
        while(temp)
        {
            sum=sum+temp%10;
            temp/=10;
        }
        if(sum>9)
        {
            temp=sum;
            sum=0;
            while(temp)
            {
```

```
        sum=sum+temp%10;
        temp/=10;
    }
}
if(sum==9)
    printf("%d, ",data);
}
printf("\n");
}
```

<Question_5> In the above program make arrangement, to list only numbers which are ascending-order-digits.

Solution:

```
#include<stdio.h>

int testDigits(int);
int testAsc(int);

int main()
{
    int min,max;
    int data;
    printf("Enter The Min Value : ");
    scanf("%d",&min);
    printf("Enter The Max Value : ");
    scanf("%d",&max);
    for(data=min;data<=max;data++)
        if(testDigits(data)==9)
            if(testAsc(data))
                printf("%d, ",data);
}
```



```
int testDigits(int data)  
{  
    int sum=0;  
    while(data)  
    {  
        sum=sum+data%10;  
        data/=10;  
    }  
    return sum;  
}
```

```
int testAsc(int data)  
{  
    int curDig;  
    int preDig=9;  
    while(data)  
    {  
        curDig=data%10;  
        if(curDig>preDig)  
            return 0;  
        preDig=curDig;  
        data/=10;  
    }  
    if(data==0)  
        return 1;  
    else  
        return 0;  
}
```

<Question_6> Write a program to print the factorial of a given integer.

Solution:

```
#include<stdio.h>

int main()
{
    int data,i,fact=1;
    printf("Enter the data: ");
    scanf("%d",&data);
    for(i=data;i-->0)
        fact*=i;
    printf("factorial = %d\n",fact);
}
```

<Question_7> Write a program to print the Fibonacci series up to a given count.

Solution:

```
#include<stdio.h>

int main()
{
    a=0,b=1,c,max;
    printf("Enter the max number: ");
    scanf("%d",&max);
    printf("0, 1, ");
    for(c=a+b;(c=a+b)<=max;a=b,b=c)
        printf("%d, ",c);
}
```

<Question_8> Write a program to test the supplied integer is prime or not.

Solution:

```
#include<stdio.h>
#include<math.h>
int main()
{
    int data,s,i;
    printf("Enter The Integre Number :");
    scanf("%d",&data);
    s=sqrt(data);
    for(i=2;i<=s;i++)
    {
        if(data%i==0)
            break;
    }
    if(i==(s+1))
        printf("Prime");
    else
        printf("Not Prime");
}
```

<Question_9> List the prime numbers within a supplied range.

Solution:

```
#include<stdio.h>

int testPrime(int);

int main()
{
    int data;
    int min,max;
    printf("Enter min number : ");
    scanf("%d",&min);
    printf("Enter max number : ");
    scanf("%d",&max);
    for(data=min;data<=max;data++)
    {
        if(testPrime(data)==1)
            printf("%d",data);
    }
}

int testPrime(int data)
{
    int i;
    for(i=2;i<data;i++)
    {
        if(data%i==0)
            return 0;
    }
    return 1;
}
```

<Question_10>In the above program make arrangement to list those prime numbers, which are sorted(digit-wise), ascending or descending, both valid.

Solution:

```
#include<stdio.h>

int testPrime(int);
int testDec(int);
int testAsc(int);
int testDigits(int,int);

int main()
{
    int data;
    int max,min;
    printf("Enter min data : ");
    scanf("%d",&min);
    printf("Enter max data : ");
    scanf("%d",&max);
    for(data=min;data<=max;data++)
    {
        if(testPrime(data)==1)
            if(testDigits(data,5))
                if(testAsc(data)||testDec(data))
                    printf("%d",data);
    }
}
```

int testPrime(int data)

```
{  
    int i;  
    for(i=data;i<data;i++)  
    {  
        if(data%i==0)  
            return 0;  
        return 1;  
    }  
}
```

int testAsc(int data)

```
{  
    int preDig,curDig;  
    preDig=9;  
    while(data)  
    {  
        curDig=data%10;  
        if(curDig>preDig)  
        {  
            return 0;  
        }  
        preDig=curDig;  
        data/=10;  
    }  
    return 1;  
}
```

```
int testDec(int data)
{
    int preDig,curDig;
    preDig=0;
    while(data)
    {
        if(curDig<preDig)
        {
            return 0;
        }
        data/=10;
    }
    return 1;
}
```

```
int testDigits(int data, int digit)
{
    while(data)
    {
        if(data%10==digit)
            return data;
        data/=10;
    }
    return 0;
}
```

Assignment-8

<Question_1>

```
#include<stdio.h>

int main()
{
    int i,j,flag=1;
    for(i=1;i<=5;i++,printf("\n"))
    {
        if(i%2==0)
            flag=0;
        else
            flag=1;
        for(j=1;j<=i;j++)
        {
            printf("%d ",flag);
            flag=!flag;
        }
    }
}
```

Output:

```
1
0 1
1 0 1
0 1 0 1
1 0 1 0 1
```


<Question_2>

```
#include<stdio.h>

int main()
{
    int i,j;
    for(i=1;i<=5;i++,printf("\n"))
        for(j=1;j<=5;j++)
            (j<i) ? printf(" ") : printf("* ");
}
```

Output:

```
* * * * *
* * * *
* * *
* *
*
```

3)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int a,b,B,n;
```

```
    int f;
```

```
    char ch;
```

```
    printf("Enter The N : ");
```

```
    scanf("%d",&n);
```

```
    for(a=0;a<n;a++,printf("\n"))
```

```
    {
```

```
        f=1;
```

```
        ch=65;
```

```
        for(b=-(n-1);b<n;b++)
```

```
        {
```

```
            if(b<0)
```

```
                B=-b;
```

```
            else
```

```
                B=b;
```

```
            if(B<=a)
```

```
            {
```

```
                if(f==1)
```

```
                {
```

```
                    printf("%c",ch++);
```

```
                    f=0;
```

```
                }
```

```
            else
```

```
        {
            printf("*");
            f=1;
        }
    }
else
{
    printf(" ");
}
}
}
```

Output:

Enter The N: 5

A

A*B

A*B*C

A*B*C*D

A*B*C*D*E

5)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int a,b,c,n,f=1;
```

```
    char ch=65;
```

```
    printf("Enter The Number N : ");
```

```
    scanf("%d",&n);
```

```
    for(a=1;a<=n;a++,printf("\n"))
```

```
    {
```

```
        f=1;
```

```
        for(b=1;b<=n;b++)
```

```
        {
```

```
            if(b<=(n-a))
```

```
                printf(" ");
```

```
            else if(f==1)
```

```
            {
```

```
                printf("%c",ch);
```

```
                f=0;
```

```
            }
```

```
            else if(b==n)
```

```
            {
```

```
                printf("*%c",ch);
```

```
            }
```

```
        else
```

```
        {
```

```
            printf("***");
```

```
        }
```

```
}  
    ch++;  
}  
}
```

Output:

Enter The Number N : 5

A

B*B

C***C

D*****D

E*****E

6)

```
#include<stdio.h>

int main()
{
    int a,b,n;
    printf("Enter The N : ");
    scanf("%d",&n);
    for(a=n;a>0;a--,printf("\n"))
        for(b=1;b<=a;b++)
        {
            if(a%2==0)
                printf("*");
            else
                printf("%d",a);
        }
}
```

Output:

Enter The N : 5

55555

333

**

1

7)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int i,j;
```

```
    for(i=1;i<=5;i++,printf("\n"))
```

```
        for(j=5;j>=i;j--)
```

```
            printf("* ");
```

```
    for(i=1;i<=4;i++,printf("\n"))
```

```
        for(j=1;j<=i+1;j++)
```

```
            printf("* ");
```

```
}
```

```
* * * * *
```

```
* * * *
```

```
* * *
```

```
* *
```

```
*
```

```
* *
```

```
* * *
```

```
* * * *
```

```
* * * * *
```

8)

```
#include<stdio.h>

int main()
{
    int i,j;
    for(i=1;i<=5;i++,printf("\n"))
        for(j=1;j<=5;j++)
            (j<i)?printf(" "):printf("* ");
    for(i=1;i<=4;i++,printf("\n"))
        for(j=5;j>=1;j--)
            (j>i+1)?printf(" "):printf("* ");
}
```

Output:

```
* * * * *
* * * *
* * *
* *
*
* *
* * *
* * * *
* * * * *
```


Assignment-9

<Question_1> Define function in C. List the types of function. What are the benefits of using more functions in a C program?

A function is a block of statements that performs a specific task.

Types of functions:

- 1) Predefined standard library functions
- 2) User Defined functions

Why we need functions in C:

Functions are used because of following reasons –

- a) To improve the readability of code.
- b) Improves the reusability of the code, same function can be used in any program rather than writing the same code from scratch.
- c) Debugging of the code would be easier if you use functions, as errors are easy to be traced.
- d) Reduces the size of the code, duplicate set of statements are replaced by function calls.

<Question_2> What is actual and formal argument?

Actual arguments: The arguments which are mentioned in the function call is known as the actual argument. For example:

function (12, 23); *//here 12 and 23 are actual arguments.*

Actual arguments can be constant, variables, expressions etc.

function (a, b); *// here actual arguments are variable*

function (a + b, b + a); *// here actual arguments are expression*

Formal Arguments: Arguments which are mentioned in the definition of the function is called formal arguments. Formal arguments are very similar to local variables inside the function. Just like local variables, formal arguments are destroyed when the function ends.

```
int factorial (int n)  
{  
    // write logic here  
}
```

Here n is the formal argument. Things to remember about actual and formal arguments.

<Question_3> Explain memory partitions in an executable and in process.

Before executing of program start, stack allocated for its by operating system.
Stack size depends on operating system.

Stack	LIFO (Last in First Out Stack) contains: 1) Function return address 2) Formal Arguments 3) Local Variable
Heap	Heap used for dynamic memory allocation. It is not allocated its free section of memory.
a.out	Contains 2 section 1) Text Section: Text section contains binary equivalent of code contains all user defined function bodies. 2) Data Section: contains global and local variable.

Whenever the function called stack frame is created in stack section. Stack frame for the function contains local variable defined function, formal arguments, and function return address.

<Question_4> Can a function return more than one data? If needed to return more than one data, how to arrange?

In C, we cannot return multiple values from a function directly.

We can return more than one values from a function by using the method called “call by address”, or “call by reference”. In the invoker function, we will use two variables to store the results, and the function will take pointer type data. So, we have to pass the address of the data.

<Question_5> What is the role of stack in function call?

- 1) When program execution starts, main stack frame is created. Local variable, formal arguments and return address are present in stack from, when function is called from main function stack frame is created and return address is pushed in stack shown.
- 2) In same way when the function is called, their stack frame is created, when a function is collapse, its stack frame is also collapse, that means, local variable present int that function, formal argument also collapses
- 3) When function is called again, its stack frame is created again, meaning local variable formal arguments will also be created again.

<Question_6> What is recursion?

The function called itself call recursion, or recursive function.

<Question_7> Compare recursion with loop? Give example where loop is better than recursion. Also give example where recursion is better than loop.

Recursion	Loops
1) A method of calling a function itself within the same function.	1) A control structure that allows executing a block of code repeatedly within the program.
2) Execution is slower.	2) Execution is faster.
3) Stack is used to store the local variable when function is called.	4) Stack is not used.
4) If there is no termination condition, then it can be infinite loop.	4) If the condition never become false, it will be an infinite loop.
5) More Readable.	6) Less Readable.
7) Space complexity is higher.	8) Space complicity is lower.

<Question_8> Write a program to calculate sum of digits of a supplied integer. Create function named int sumOfDigits(int), to return the sum of digits of a supplied integer.

i) calculate using loop

ii) calculate using recursion.

```
#include<stdio.h>
```

```
int sumofDigits(int);
```

```
int main()
```

```
{
```

```
    int data;
```

```
    printf("Enter The data : ");
```

```
    scanf("%d",&data);
```

```
    printf("Sum of Digit of %d is %d\n",data,sumofDigits(data));
```

```
}
```

```
int sumofDigits(int data)
```

```
{
```

```
    int sum=0;
```

```
    int rem;
```

```
    while(data>0)
```

```
    {
```

```
        rem=data%10;
```

```
        sum=sum+rem;
```

```
        data/=10;
```

```
    }
```

```
    return sum;
```

```
}
```

ii) calculate using recursion.

```
#include<stdio.h>

int sumDigits(int);

int main()
{
    int data;
    printf("Enter The Data : ");
    scanf("%d",&data);
    printf("Sum of Digit in %d is %d\n",data,sumDigits(data));
}

int sumDigits(int data)
{
    if(data/10==0)
        return data;
    return (data%10+sumDigits(data/10));
}
```

<Question_9> Calculate factorial of a supplied integer. Create function named fact of prototype “unsigned long int fact (unsigned int);”

i)using loop

ii)using recursion.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int data;
```

```
    int fact=1;
```

```
    printf("Enter The Data : ");
```

```
    scanf("%d",&data);
```

```
    for(int i=data;i;i--)
```

```
        fact*=i;
```

```
    printf("Factorial is : %d\n",fact);
```

```
}
```

ii)using recursion.

```
#include<stdio.h>
```

```
int fact(int);
```

```
int main()
```

```
{
```

```
    int data;
```

```
    printf("Enter The Data : ");
```

```
    scanf("%d",&data);
```

```
    printf("Factorial Of %d is %d\n",data,fact(data));
```

```
}
```

```
int fact(int data)
```

```
{
```

```
    if(data==0)
```

```
        return(1);
```

```
    else
```

```
        return(data*fact(data-1));
```

```
}
```


<Question_10> Print Fibonacci series up to a specified value. Use function named fib of prototype int fib(int); The function fib should return Fibonacci value associated with supplied number. For example: fib (1) is 0 fib (2) is 1 fib (5) is 3

```
#include<stdio.h>

int main()
{
    int max;
    int a=0,b=1;
    int c;
    printf("Enter The Max Number : ");
    scanf("%d",&max);
    printf("0, 1, ");
    for(c=a+b;(c=a+b)<=max;a=b,b=c)
        printf("%d, ",c);
    printf("\n");
}
```

```
#include<stdio.h>
```

```
int fib(int);
```

```
int main()
```

```
{
```

```
    int data,i;
```

```
    printf("Enter The Data : ");
```

```
    scanf("%d",&data);
```

```
    for(i=0;i<data;i++)
```

```
        printf("%d, ",fib(i));
```

```
    printf("\n");
```

```
}
```

```
int fib(int data)
```

```
{
```

```
    if(data==0 || data==1)
```

```
        return(1);
```

```
    else
```

```
        return(fib(data-1)+fib(data-2));
```

```
}
```

Assignment-10

Predict the output of the following code snippets.

<Question_1>

```
#include<stdio.h>

int f(int);

int main()
{
    int i=3,val;
    val=f(i+=2)+f(i=1)+f(i++);
    printf("Val=%d i=%d\n",val,i);
    return 0;
}

int f(int num)
{
    return num*5;
}
```

Solution:

Val=35 i=2

<Question_2>

```
#include<stdio.h>

int func(int i)
{
    if(i%2)return 0;
    else return 1;
}

int main()
{
    int i=3;
    i=func(i);
    i=func(i);
    printf("%d\n",i);
}


int counter(int i)
{
    static int count=0;
    count=count + i;
    return(count);
}


int main()
{
    int i, j;
    for(i=0;i<=5;i++)
        j=counter(i);
    printf("%d", j);
```

}

Solution:

test.c:22:5: error: redefinition of ‘main’

22 | int main()

| ^~~~

test.c:7:5: note: previous definition of ‘main’ was here

7 | int main()

| ^~~~

<Question_3>

```
#include<stdio.h>

int func(int);

int main ()
{
    int x,y;
    y=5;
    x=func(y++);
    (x==5)?printf(" True\n "):printf(" False\n " );
}

int func(int z)
{
    if(z==6)
        return 5;
    else
        return 6;
}
```

Solution:

False

<Question_4>

```
#include<stdio.h>

int f(int);

int main()
{
    int x;
    x = 3;
    f(x);
    printf("MAIN\n");
}

int f(int n)
{
    printf("F\n");
    if (n != 0)
        f(n-1);
}
```

Solution:

```
F
F
F
F
MAIN
```

<Question_5>

```
#include<stdio.h>

int find(int);

int main()
{
    int a=5;
    a=find(a+=find(a++));
    printf("%d\n",a);
}

int find(int a)
{
    return(a++);
}
```

Solution:

11

<Question_6>

```
#include<stdio.h>

void fn(int,int);

int main()
{
    int a=5;
    printf("in main: %d %d\n",a++,++a);
    fn(a,a++);
}

void fn(int a,int b)
{
    printf("fn: %d ,%d\n",a,b);
}
```

Solution:

in main: 6 7

fn: 8 ,7

<Question_7>

```
#include<stdio.h>

int fn(int v)
{
    if(v==1 || v==0)
        return 1;
    if(v%2==0)
        return fn(v/2)+2;
    else
        return fn(v-1)+3;
}

int main()
{
    printf("%d\n",fn(7));
}
```

Solution:

11

<Question_8>

```
#include<stdio.h>

int x=5;

void print()
{
    printf("%d\n",x--);
}

int main()
{
    print();
}
```

Solution:

5

<Question_9>

```
#include<stdio.h>

int main()
{
    int n=10;
    int func(int);
    printf("%d\n",func(n));
}

int func(int n)
{
    if(n>0)
        return(n+func(n-2));
    else return 0;
}
```

Solution:

30

<Question_10>

```
#include<stdio.h>
```

```
int cap(int);
```

```
int main()
```

```
{
```

```
    int n;n= cap(6);
```

```
    printf("%d\n",n);
```

```
}
```

```
int cap(int n)
```

```
{
```

```
    if(n<=1) return 1;
```

```
    else return(cap(n-3)+cap(n-1));
```

```
}
```

Solution:

9

<Question_11>

```
#include<stdio.h>

int main()
{
    int num = _a_123(4);
    printf("%d", --num);
    return 0;
}

int _a_123(int num)
{
    return(num++);
}
```

Solution:

3

<Question_12>

```
#include<stdio.h>

int function(int, int);

int main()
{
    int a = 25, b = 24, c;
    printf("%d ", function(a + 2, b + 3));
}

int function(int x, int y)
{
    return (x - (x == y));
}
```

Solution:

26

C Assessment HE6

(By Shashank & Varlakxmi Ma'am.)

Q1. Output of following code.

Void main()

```
{
    Unsigned char I = 65,j;
    For(j=0;j<8;j++)
    {
        Printf(“%d”, i&(0x80>>j) ? 1 :0);
    }
}
```

Answer-> 01000001

```

Explanation -> for j=0 ---> 65 & (0x80>>0) -> 01000001      (65)
                                & 10000000      (128)
                                00000000      (0) -> i.e. false so 0 will print 1st

For j=1 --> 65 & (0x80>>1) -> 01000001      (65)
10000000>>1 ----->      01000000      (64)
                                01000000      (64)---> i.e. True so it will print 1

For j=2 --> 65 & (0x80>>2) -> 01000001      (65)
10000000>>2 ----->      00100000      (32)
                                00000000      (0) ---> i.e. False so it will print 0

For j=3 --> 65 & (0x80>>3) -> 01000001      (65)
10000000>>3 ----->      00010000      (16)
                                00000000      (0) ---> i.e False so it will print 0

For j=4 --> 65 & (0x80>>4) -> 01000001      (65)
10000000>>4 ----->      01001000      (8)
                                00000000      (0) ---> i.e. False so it will print 0

```


For j=5 --→ 65 & (0x80>>5) -→	01000001	(65)
10000000>>5 -----→	01000100	(4)
	00000000	(0) ---→ i.e. False so it will print 0
For j=6 --→ 65 & (0x80>>6) -→	01000001	(65)
10000000>>6 -----→	00000010	(2)
	00000000	(0) ---→ i.e. False so it will print 0
For j=7 --→ 65 & (0x80>>7) -→	01000001	(65)
10000000>>7 -----→	00000001	(1)
	00000001	(1) ---→ i.e. True so it will print 1

Q2. Output of Following Code?

```
int main()
{
    unsigned char x = 0x14;
    x = (x>>4 | x<<4);
    printf(“%c %d %x %o”,x,x,x,x);
}
```

Answer -> A 65 41 101

i.e 0x14 is 20 in decimal ---→	0001 0100	(20)
20>>4	0100 0001	(65)
20<<4	0100 0001	(65)
(or)	0100 0001	(65) so it will print 65 in Ascii Dec Hex & Octal.

Q3. Output for Following Code?

```
int main()
{
    char a = 'A';
    Int x = 41;
    Float b = 2.3;
    Void *ptr;
    ptr = &a;
    printf("%c",*ptr);
    ptr = &x;
    printf("%d",*ptr);
    ptr = &b;
    printf("%f",*ptr);
}
```

Answer -> Any Garbage Value of Char Integer & Float

Explanation -> As void *ptr; did not assigned to NULL, so it will initially have any Garbage Value.

Q4. Output of Code?

```
int main()
{
    int x=10,y=20,z;
    z = (x++ || --y);
    printf("%d %d %d",x,y,z);
}
```

Answer --> 11 20 1

Explanation ---> x++ || --y i.e. 10 || 20 (--y Will not checked) but or will only check left to right if left value is true output is true i.e. 1

So z = 1, x = 11, y = 20

Q5. Predict the output?

```
int main()
{
    float a=5.0,b=5.0;
    int c;
    c = a|b;
    printf("%d\n",c);
}
```

Answer--→ Compiler Error

Explanation --→ we cannot perform bit wise operation on float & double.

Q6. Output of code?

```
#include<stdio.h>
int main()
{
    int i;
    for(i=0;i++<10;);
    printf("%d\n",i);
}
```

Answer -→ 11

Explanation --→ for loop will execute till $i < 10$ so when it break I will be 10 & we use $i++$ so 11 will be final output.

Q7. Predict the output?

```
#include<stdio.h>

union t
{
    unsigned char x;
    unsigned b0:1;
    unsigned b1:1;
    unsigned b2:1;
    unsigned b3:1;
    unsigned b4:1;
    unsigned b5:1;
    unsigned b6:1;
    unsigned b7:1;
};

int main()
{
    union t v={0x34};
    unsigned temp;

    temp = v.b7; v.b7=v.b0; v.b0=temp;
    temp = v.b6; v.b6=v.b1; v.b1=temp;
    temp = v.b5; v.b5=v.b2; v.b2=temp;
    temp = v.b4; v.b4=v.b3; v.b3=temp;
    printf("%x",v.x);
}
```

Answer --→ 34

Explanation ---→ All assignment operation has been performed on bit field so no change happens there.

Q8. Output of code?

```
#include<stdio.h>

int main()
{
    printf("%d %d\n",sizeof('a'),sizeof("ab"));
}
```

Answer -→ 4 3

Explanation → **sizeof** the character literal is char. In **C** the type of character literal is integer (int). So in **C** the **sizeof('a')** is 4 for 32bit architecture,

- It is also looks like char constant i.e ascii value of 'a' is integer i.e 4 byte.

Sizeof("ab") --→ a b '\0' i.e 1+1+1 == 3

Q9.output of code?

```
#include<stdio.h>

int main()
{
    int arr[] = {10,20,30,40,50};
    printf("%d\n",&arr[4]-&arr[1]);
}
```

Or---→

```
#include<stdio.h>

int main()
{
    char str[]="vector";
```

```
printf("%d\n",&str[4]-&str[1]);
}
```

Answer --→ 3

Explanation --→

0 th	1 st	2 nd	3 rd	4 th
10 1004	20 1008	30 1012	40 1016	50 1020

&arr[4] --→ 1020

&arr[1] --→ 1004

1016 i.e. 3rd

Or.

Str+4 – (str+1)

~~Str+4-str-1~~

4-1 = 3.

Q10. Output of Below Code?

```
int main()
{
    int a=356;
    char *p;
    p = &a;           // if p=a; then Segment fault.
    printf("%d\n",*p);
}
```

Answer --→ 100

Explanation → 356 --→ 0000 0000 0000 0000 0000 0001 0110 0100

Char *p will store only 1 byte i.e. MSB-----→ 0110 0100 (100)

Q11. Output Prediction?

```
#include<stdio.h>

struct var{
    int x;
    char a[100];
};

int main()
{
    struct var v;
    v.x = 10;
    v.a = "abcd";
    printf("%d\n",v.x);
    printf("%s\n",v.a);
}
```

Answer -→ Compiler Error.

Explanation ---→ Assign with = operator not possible we have to use strcpy() or strcat().

Q12. Predict the output?

```
int main()
{
    float a=5.0,b=5.0;
    int c;
```

```
    c = a&b;    // if any of bit wise operator is used then error.

    printf("%d\n",c);
}
```

Answer→Compiler Error

Explanation →we cannot perform bit wise operation on float & double.

Q13. Predict Output?

```
#include<stdio.h>

int main()
{
    int i=5;
    if(i=3)
        printf("Hello");
    else
        printf("hai");
}
```

Answer → Hello

If (i=3) here we used assignment operator so it will always be true.

So it prints “Hello”.

Q14. Output of code?

```
#include<stdio.h>

int main()
{
    int i=0;
    while(i++<10);
    printf("%d",i);
}
```


Answer --→ 11

While(i++<10)----→ it will execute this till <10

i.e while(0<10) (i=1)--→

while(1<10) (i=2)

while(2<10) (i=3)

while(3<10) (i=4)

while(4<10) (i=5)

while(5<10) (i=6)

while(6<10) (i=7)

while(7<10) (i=8)

while(8<10) (i=9)

while(9<10) (i=10)

while(10<10) Loop break here, but I will increment so, i=11

Q15 Output of code?

```
#include<stdio.h>
```

```
struct st
```

```
{
```

```
    int data;
```

```
    char *ptr;
```

```
};
```

```
int main()
```

```
{
```

```
    char s1[] = "abcd";
```

```
    struct st V;
```

```
    V.data = 10;
```

```
    strcpy(V.ptr,s1);
```

```
    printf("%d %s",V.data,V.ptr);
```

```
}
```

Answer ---→ 10 abcd

Q16 Output predict?

```
#include<stdio.h>

struct st
{
    int data=20;
    char str[20];
};

int main()
{
    struct st V;
    printf("%d %s",V.data,V.ptr);
}
```

Answer --→ Compiler Error.

Explanation ---→

- We cannot assign any value inside the structure block,
- Structure is data type which is used to assign every time new value so we cannot fix or make it assign to any value.

Q17. Compilation Command to each stage?

Answer→

1. Preprocessor –

- Gcc -E <filename.c>
Eg. Gcc -E hello.c
- 2. Translator –
Gcc -s <filename.c>
Eg. Gcc -s hello.c
- 3. Assembler –
Gcc -c <filename.c>
Eg. Gcc -c hello.c
- 4. Linker --
Gcc <filename.c>
Eg. Gcc hello.c

Q18. Application of signed char?

Answer--→

- We can use it as in 8051 need to store baud rate some time its positive or negative. So to store that we need signed char.

Q19. Difference between sizeof & strlen ?

Answer ---→

Parameter	Sizeof	Strlen
Type	Unary Operator	Pre-Defined Function
Data Type Supported	Sizeof gives actual size of any type of data (allocated) in bytes (including the null values)	Get the actual length of char of array/str
Evolution Time	sizeof() is a compile-time expression giving you the size of a type or a variable's type.	Strlen on the other hand, gives you the length of a C-style NULL-terminated string.

Q20. Difference between memcpy & strcpy?

Answer--→

- memcpy() function is use to copy a specified number of bytes from one memory to another. Whereas, strcpy() function is use to copy the contents of one string into another string.
- memcpy() function acts on memory rather than value. Whereas, strcpy() function acts on value rather than memory.

Q21. Preprocessor Stages?

Answer---→

- it is the 1st stage in compilation stages.
- It performs preprocessing of the High Level Language(HLL).
- ❖ Task of Preprocessor --→
 1. Comment Removing.
 2. File Inclusion
 3. Macro Expansion / replacement.

Q22. Void Pointer & Application?

Answer--→

- ✓ A void pointer is a pointer that has no associated data type with it. A void pointer can hold address of any type and can be typecasted to any type.
- **Use**
 - Malloc() and calloc() return void * type and this allows these functions to be used to allocate memory of any data type (just because of void *).
 - Void pointers in C are used to implement generic functions
I.e. eg. Comparator in qsort().
- **Limitation**
 - Pointer arithmetic cannot be performed in a void pointer.

Q23. Features of storage class?

Answer --→

- A storage class represents the visibility and a location of a variable. It tells from what part of code we can access a variable.
- **A storage class is used to describe the following things:**
 - ✓ The variable scope.
 - ✓ The location where the variable will be stored.
 - ✓ The initialized value of a variable.
 - ✓ A lifetime of a variable.
 - ✓ Who can access a variable?
- **Type of Storage Class->**

1. Auto
2. Extern
3. Static
4. Global
5. Register

Q24. When to use for loop & While Loop?

Answer ->

❖ **For Loop--**

- Use a for loop to iterate over an array.
- Use a for loop when you know the loop should execute n times.

❖ **While Loop-**

- Use a while loop for reading a file into a variable.
- Use a while loop when asking for user input.
- Use a while loop when the increment value is nonstandard.

Q25. Difference Between While & Do-While Loop?

Answer ->

While	Do-While
Condition is checked first then statement(s) is executed.	Statement(s) is executed at least once, thereafter condition is checked.
It might occur statement(s) is executed zero times, If condition is false.	At least once the statement(s) is executed
No semicolon at the end of while. while(condition)	Semicolon at the end of while. while(condition);

While loop is entry controlled loop. i.e. while(condition) { statement(s); }	Do-while loop is exit controlled loop. i.e. do { statement(s); }while(condition);
If there is a single statement, brackets are not required.	Brackets are always required
Variable in condition is initialized before the execution of loop.	Variable may be initialized before or within the loop.

Q26. Difference between Internal Linkage & External Linkage?

Answer →

❖ Internal Linkage→

- In this type the variable/identifier scope can be used within the file.
- We cannot use the variable/Identifier of other file inside current file.
- Implemented by “static” keyword followed by identifier.

❖ External Linkage→

- In this type the variable/identifier scope can be used within the file as well as inside other / outer file.
- Implemented by “extern” keyword followed by identifier.

Q27. Difference between SLL & DLL?

Answer->

SLL	DLL
It content single Self-Referential structure Pointer. I.e. struct st *next.	It content 2 Self-Referential structure Pointer. i.e. struct st *next,*prev.
1 st node connected to next node	In this 1 st node connected to next node & prev must be connected 1 st node.
Reverse, Search, Delete sometimes become complicated	Reverse, Search, Delete becomes easier
We have to maintain or use temp pointer (*temp) in almost all function	There is no need to maintain temp pointer (*temp).
I content Data & Address of Next Node	It Content Address of prev node, Data, & Address of next node

Q28. Range & size of Data Type?

Answer ->

Data Type	Range	Size
Signed char	-128 to 127	1 byte
Unsigned char	0 to 255	1 byte
Signed short int	-32768 to 32767	2 byte
Unsigned short int	0 to 65535	2 byte
Signed int	-2g to 2g (-2,147,483,648 to 2,147,483,647)	4 byte
Unsigned int	0 to 4g (0 to 4,294,967,295)	4 byte
Signed long int	9223372036854775808 to 9223372036854775807	8 byte (4 for 32 bit)
Unsigned long int	0 to 18446744073709551615	8 byte
Float	1.2E-38 to 3.4E+38	4 byte
Double	2.3E-308 to 1.7E+308	8 byte
Long Double	3.4E-4932 to 1.1E+4932	10 byte

Q29. Structure vs union

Answer ->

Parameter	Structure	Union
Definition	A structure is a user-defined data type available in C that allows to combining data items of different kinds.	A union is a special data type available in C that allows storing different data types in the same memory location
Keyword	Keyword “struct” is used to define structure.	Keyword “union” is used to define structure.
Size	Is the sum of sizeof of all structure members	Is the sizeof largest member on union.
Memory allocation	Each member in structure is assigned with unique storage area.	All member of union get shared memory.
Dependency	There is no dependant member in structure as each member get unique memory	Union members are dependent on each other, as it get shared memory.
Accessing member	Individual members can be accessed at a time	Only one members accessed at a time

Q30. Uses of structure over union

Answer->

For Structure-

- To store different **types** of data at one place i.e. can be accessed using a **single** variable name.
- To implement almost all ADTs (data structures) like :
 - a. lists
 - b. queue
 - c. stack
 - d. trees
 - e. graphs
- You can create a **database** using an **array** of **struct**
- In fact you can perform **all operations on structures** which you do on any primitive data type.

For union –

- ✓ used to implement pseudo-polymorphism in C,
- ✓ Useful feature is the bit modifier.
- ✓ Used as space optimizer.

Q31 Difference between “/” and “%” and their uses. “%” is used using what data types.

Answer ->

❖ For “%”

- % is a modulo operator , It give remainder of two numbers on division as result and discard quotient and gives an integer
- Eg. $11\%4=3$
- $11/4 \Rightarrow 11-(4*2) \Rightarrow 11-8 = 3$, remainder is 3 as it is less than 4.
- Application ->
 1. It is used as format specifier I.e. (%c/d/x/o/s/f etc)
 2. In so many code like prime number, getting reminder of division.
- It can be used with char,int but it cannot be used with **Float & double**

❖ For “/”

- “/” is a division operator, it give quotient of two numbers on division as result and gives a floating point number.
- Eg. $11/4=2.75$
- $11/4 \Rightarrow 11-(4*2) \Rightarrow 11-8 = 3 \Rightarrow 3/4= 0.75$. so quotient is 2.75
- On integer type casting Result will be 2 and on double or float result will be 2.75.

- Application--→
- 1. To get last digit of integer.
- 2. In arithmetic Division.
- 3. Used in commenting

Q32. Void pointer, prototype, declaration, example program.

Answer -→

- ✓ A void pointer is a pointer that has no associated data type with it. A void pointer can hold address of any type and can be typecasted to any type.
- **Prototype –**
 Void ptr_name;
 Eg. Void *ptr;
- **Example –**
 int a=20;
 char b = 'x';
 void *ptr = &a;
 ptr = &b;

Q33. Why do we use void pointer when we can explicitly type cast a point even when it is declared using another data type. For example (consider an int pointer we can use this pointer somewhere else in the program by type casting it to float. So what is the use of a void pointer when we have this type of facility available in C?)

Answer--→

Void pointer --→

- ❖ It is a pointer which can hold the address of any data type.
- ❖ Speciality of these pointer is it does not take/occupy any kind of memory until we didn't assigned it.
- ❖ Above point make this a special & different whereas in other hand other pointer take / occupy memory when it declared.

I.e. For Normal Pointer---→

```
Int a=10;
Int *ptr;    //It takes 4byte memory as it is of int type.
```

For Void Pointer-----→

```
Int a=10;
```

```
Void *ptr;           //Do not take / occupy memory.  
Ptr = &a;           //now it take 4byte for int. & implicitly typecasted to int.
```

Q34. Enum uses.

Answer→

- When there is integer constant is used in your program and you want assign some name to that constant enum are used.

Q35. Var pointer, Void pointer, dangling pointer, null pointer: prototypes, difference and uses?

Answer->

Var Pointer	Void Pointer	Dangling Pointer	NULL Pointer
It is normal pointer.	It is a pointer which can hold address of any data type.	It is a case where pointer can use unallocated memory location & perform unexpected behaviour.	It is a pointer where we can avoid the problem of dangling pointer.
Used in almost all application in c where we use call by reference concept.	Can be Used as alternate of general pointer.		To avoid dangling or wild behaviour of pointer.
We might require to maintain type casting of pointer for appropriate situation.	No need to type cast it in gcc as it implicitly type cast it. Some time it requires to typecast.		

Prototype- Data_type *ptr_name;	Prototype- Void *ptr_name;	Prototype- Int a=20; Int *ptr=&a; Free(ptr); Here now ptr content some garbage value & act unexpected called dangling.	Prototype- Int a=20; Int *ptr = &a; Free(ptr); Ptr = NULL; Now this is NULL pointer.
------------------------------------	-------------------------------	---	--

Q36. Difference between Malloc and Realloc?

Answer →

Malloc	Realloc
This function allocates a size byte of memory.	Realloc is used to change the size of memory block on the heap.
Initially after allocation it will contain garbage value.	Initially if there is existing value then till will be copied & allocate rest to NULL.
It cannot be used like realloc.	If pointer passed to realloc is null, then it will behave exactly like malloc. If the size passed is zero, and ptr is not NULL then the call is equivalent to free.
Prototype- (type *) malloc(sizeof(type));	Prototype- (datatype *) realloc (ptr_name,size_t);

Q37. Difference between static allocation and dynamic allocation along with some practical examples.

Answer ---→

Static Allocation	Dynamic Allocation
variables get allocated permanently	Variables get allocated only if your program unit gets active.
Static Memory Allocation is done before program execution.	Dynamics Memory Allocation is done during program execution.
It uses stack for managing the static allocation of memory	It uses heap for managing the dynamic allocation of memory
It is less efficient	It is more efficient
there is no memory re-usability	there is memory re-usability and memory can be freed when not required
Once the memory is allocated, the memory size cannot change.	When memory is allocated the memory size can be changed.

we cannot reuse the unused memory	This allows reusing the memory
Execution is faster than dynamic memory allocation.	Execution is slower than static memory allocation.
In this allocated memory remains from start to end of the program.	In this allocated memory can be released at any time during the program.

✓ **Practical Example for Static Allocation-**

It is basically used for “ARRAY”.

Eg. `Int arr[5];` //Here size of array declared statically.

✓ **Practical Example for Dynamic Allocation-**

It is basically used for “Link-List”, “Tree”, ”Circular Queue.”, etc

Eg. `Struct st *newnode = NULL;`

`Newnode = calloc(1,sizeof(struct st));` //Here newnode will get allocated Dynamically.
We Can Also Use `malloc`, `realloc`.

Q38. Type of storage classes available and their uses. Which storage class is used for a faster programming and y?

Answer --→

- Storage Classes are used to describe the features of a variable/function. These features basically include the scope, visibility and life-time.
- There are 4 storage class.

Auto	Extern	Static	Register
This is the default storage class for all the variables.	Extern storage class simply tells us that the variable is defined elsewhere and not	Static variables have a property of preserving their value even after they are out of their scope!	The only difference between auto & Register is that the compiler tries to store these variables in the register of the microprocessor if a free register is available.

	within the same block where it is used		
Can be only accessed within the block/function they have been declared and not outside them.	they can be accessed between two different files which are part of a large program in multiple files.	Global static variables can be accessed anywhere in the program.	Can be only accessed within the block/function they have been declared and not outside them.
new memory is allocated because they are not re-declared	new memory is allocated because they are not re-declared	no new memory is allocated because they are not re-declared	new memory is allocated because they are not re-declared
They are assigned a garbage value by default whenever they are declared.	They are assigned a Zero Value by default whenever they are declared.	They are assigned a zero value by default whenever they are declared.	They are assigned a garbage value by default whenever they are declared.
Used In very programming. (knowingly or unknowingly)	Whenever we need to change the variable in overall program		
	Extern variable is nothing but a global variable initialized with a legal value where it is declared in order to be used elsewhere.		If a free register is not available, these are then stored in the memory only.
Medium Speed	Medium Speed	Good Speed	Faster in speed

- **Register** storage class is faster than all storage class.
- Because it uses Register which used inside the microprocessor for faster operations.
- There are very few Register available inside microprocessor, so we need to use this very carefully.

Q39. Difference between malloc and calloc?

Answer -->

Malloc	Calloc
This function allocates a size byte of memory.	Calloc is used to change the size of memory block on the heap.
Initially after allocation it will contain garbage value.	Initially after allocation it will content NULL.
It cannot be used like calloc	It cannot be used as malloc.
Prototype- (type *) malloc(sizeof(type));	Prototype- (data_type *) calloc (num,size_t);

Q40. Difference between an array and pointer?

Answer =>

Parameter	Array	Pointer
Definition	Collection of same data element with contiguous memory allocation	It are address variable which holds the address of any variable.
Sizeof operator	It prints the array size. i.e. no of element * sizeof(datatype)	It print the size of ptr i.e. 4/8 depends on os.
Assignment	Array cannot store address of variable	Pointer stores the address of variable.
Iteration	array elements can be navigated using indexes using [], i.e. arr[0],arr[1],etc	Pointer can give access to array elements by using pointer arithmetic. i.e, array[2] == *(p+2)

Q41.Difference between if else and conditional compilation. (#if#else#endif)

Answer→

❖ If- Else –

- It is base on true false condition.
- It is more readable than Conditional compilation equation.
- Syntax
If(true)
{
 Statement;
}
Else
{
 Statement;
}
- Efficient.
- It take more space in programming compared to conditional compilation.
- Execution is slow compared to conditional compilation.
-

❖ Conditional compilation—

- It is also based on true-false condition.
- It is in equation form, so complex than if-else.
- Syntax.

Condition ? statement1 : statement2;

I.e. Condition ? (if true) execute statement1 : else execute statement2;

- More Effective & Efficient.
- It takes less size as it is a equation only.
- Execution is faster than if-else.

Q42. Difference between r+ & w+ mode of file Handling.

Answer→

❖ r+ Mode—

- read/write mode.
- Open a text file for update.
- neither deletes the content nor create a new file if it doesn't exist.
- Data stored at initially beginning of file 1st line.

❖ w+ Mode –

- read/write mode.
- Open a text file for update.
- First truncating the file to zero length if it exists or creating the file if it does not exist.
- Data stored at initially beginning of file 1st line.
- Deletes the content of the file and creates it if it doesn't exist.

Q43. Difference between fprintf vs fwrite.

Answer →

❖ Fprintf ---

- It refer to normal later format (human readable format). i.e. Decimal/char/float/double.
- It is a predefined function.
- Fprintf will copy content of str pointer into file pointed by file pointer.
- Prototype
Int fprintf(FILE *stream, const char *format, ...);
Stream – This is the pointer to a FILE object that identifies the stream.
Format – Fornat specifier (%c/d/x/o/f etc), size(how many byte to be stored), str_pointer(copy content of str into file)
- If successful, the total number of characters written is returned otherwise, a negative number is returned (-1).

❖ Fwrite ---

- It refers to binary format. Data stores into Binary or machine language format.
- It is also a predefined function.
- Fprintf will copy content of str pointer into file pointed by file pointer in binary format.
- Prototype
Size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);

▪ Parameters

- Ptr – This is the pointer to the array of elements to be written.
- Size – This is the size in bytes of each element to be written.
- Nmemb – This is the number of elements, each one with a size of size bytes.
- Stream – This is the pointer to a FILE object that specifies an output stream.
 - This function returns the total number of elements successfully returned as a size_t object, which is an integral data type. If this number differs from the nmemb parameter, it will show an error.

Q44. Difference between break & return.

Answer→

❖ Break —

- It will break the currently running block or loop, & come out of it.
- Break are used mostly inside loops & switch case.
- Break name itself gives idea about its behaviour.

❖ Return ---

- It will terminate all execution of currently running block or function, & go return back to where function call is happens.
- It mostly used with function.
- Function must be declares its return type.
- If return type is not present then return will not work, it throws an error.

Q45. Difference between object code & executable code.

Answer→

❖ Object Code —

- Object Code is the code in which no extra library file are linked.
- It is less in size compared to Executable code.
- It can be taken after Assembler stage.
i.e. gcc -c filename.c //we will get (.o) file.
- It is in binary format.
- It can be used for debugging using objdump.

❖ Executable Code –

- Code is the code in which extra library file are linked.
- It is large in size compared to Object code.
- It can be taken after Linker stage.
i.e. gcc filename.c //we will get (.out or .exe) file.
- It is in binary format.
- It can be used for debugging using objdump & gdb.

Q46. What is DMA?

Answer-→

- Dynamic Memory Allocation is the term refer to give memory to an variable or pointer.
- It totally work on heap area.
- In this method variable or pointer get memory block to store the assigned value to that variable.
- There are 4 type of DMA.
 1. Malloc --- allocate memory to variable & assign garbage by default.
 2. Calloc ---- allocate memory to variable & assign zero by default.
 3. Realloc --- allocate or update the memory size of variable. It can be increased or decreased.
 4. Free --- de allocate the allocated memory.

Q47. What is constant pointer? & its uses?

Answer-→

Q48. What are the header files?

Answer-→

- Headerfile are those file which content the predefined definitions of various functions.
- It is default present inside the computer in predefined directory.
- It can be included from current directory too.
- There are lots of headerfile arr present in gcc.
- This inclusion of headerfile take place at pre-processor stage.
- Syntax for inclusion of headerfile.
 - **#include<headerfile.h>** // it get included form predefined directory.
 - **#include"headerfile.h"** //it included forn current directory

Q49. Array to Pointer & Pointer to Array? Its Example?

Answer-→

- ❖ **Array to Pointer ---**
 - It is a array of Pointer variable.

- It consist of pointer which holds address of variable.
- Collection of same type of pointer variable.
- Syntax-

```
Int arr[5] = {ptr1, ptr2 , ptr3, ptr4, ptr5};
```

 Array arr[5] content ptr.

❖ **Pointer to Array ---**

- it is a pointer to array.
- It hold the address of array, which content the variable.
- Pointer can be char, int, double, float.
- Syntax.

```
Int arr[5] = {1,3,4,5,6};
```

```
Int *ptr = arr;
```

Q50. Prototyoes

Answer→

❖ **Printf()**

- ✓ **Int printf(char *format, arg1, arg2, ...);**
- ✓ Printf converts, formats, and prints its arguments on the standard output. It returns the number of characters printed.

❖ **Scanf()**

- ✓ **Int scanf(const char *format, ...);**
- ✓ Read from stdin.
- ✓ The function returns the total number of items successfully matched, which can be less than the number requested. If the input stream is exhausted or reading from it otherwise fails before any items are matched, EOF is returned.

❖ **Strlen()**

- ✓ **Int strlen(const char *str);**
- ✓ The strlen() function calculates the length of a given string. The strlen() function is defined in string.h header file. It doesn't count null character '\0'.
- ✓ **Return:** This function returns the length of string passed.
- ✓ It is defined in the `string.h` header file.

❖ **Strcpy()**

- ✓ **Char* strcpy(char* destination, const char* source);**

- ✓ Strcpy() is a standard library function in C/C++ and is used to copy one string to another. In C it is present in string.h header file and in C++ it is present in cstring header file.
- ✓ **Return Value:** After copying the source string to the destination string, the strcpy() function returns a pointer to the destination string.
- ✓ It is defined in the `string.h` header file.

❖ Strncpy()

- ✓ **Char *strncpy(char *dest, const char *src, size_t n);**
- ✓ At most n bytes of src are copied. If there is no NULL character among the first n character of src, the string placed in dest will not be NULL-terminated. If the length of src is less than n, strncpy() writes additional NULL character to dest to ensure that a total of n character are written.
- ✓ **Return Value:** It returns a pointer to the destination string.
- ✓ It is defined in the `string.h` header file.

❖ Strcat()

- ✓ **Char *strcat(char *destination, const char *source);**
- ✓ The strcat() function concatenates the destination string and the source string, and the result is stored in the destination string.
- ✓ It is defined in the string.h header file.
- ✓ **Return Value :** This function returns a pointer to the resulting string dest.

❖ Strncat()

- ✓ **Char *strncat(char *dest, const char *src, size_t n);**
- ✓ This function appends not more than n characters from the string pointed to by src to the end of the string pointed to by dest plus a terminating Null-character. The initial character of string(src) overwrites the Null-character present at the end of string(dest).
- ✓ **Return Value:** The strncat() function shall return the pointer to the string(dest).

❖ Strcmp()

- ✓ **Int strcmp(const char *leftStr, const char *rightStr);**
- ✓ Strcmp() is a built-in library function and is declared in <string.h> header file. This function takes two strings as arguments and compare these two strings lexicographically.
- ✓ This function can return three different integer values based on the comparison:
 - Zero (0): A value equal to zero when both strings are found to be identical.
 - Greater than zero (>0): A value greater than zero is returned when the first not matching character in leftStr have the greater ASCII value than the corresponding character in rightStr.

- Less than Zero (<0): A value less than zero is returned when the first not matching character in leftStr have lesser ASCII value than the corresponding character in rightStr.

❖ Strncmp()

- ✓ **Int strncmp(const char* lhs, const char* rhs, size_t count);**
- ✓ The strncmp() function in C++ compares a specified number of characters of two null terminating strings. The comparison is done lexicographically.
- ✓ It is defined in <cstring> header file.
- ✓ The strncmp() function returns a:
 - Positive value if the first differing character in lhs is greater than the corresponding character in rhs.
 - Negative value if the first differing character in lhs is less than the corresponding character in rhs.
 - 0 if the first count characters of lhs and rhs are equal.

❖Memcpy()

- ✓ **Void *memcpy(void *dest, const void * src, size_t n);**
- ✓ Copies n characters from memory area src to memory area dest.
- ✓ This function returns a pointer to destination, which is str1.

❖Memmove()

- ✓ **Void *memmove(void *str1, const void *str2, size_t n);**
- ✓ Copies n characters from str2 to str1, but for overlapping memory blocks, memmove() is a safer approach than memcpy().
- ✓ This function returns a pointer to the destination, which is str1.

❖Fprintf ()

- ✓ **Int fprintf(FILE *stream, const char *format, ...);**
- ✓ Sends formatted output to a stream.
- ✓ If successful, the total number of characters written is returned otherwise, a negative number is returned.

❖Fscanf()

- ✓ **Int fscanf(FILE *stream, const char *format, ...);**
- ✓ Reads formatted input from a stream.
- ✓ This function returns the number of input items successfully matched and assigned, which can be fewer than provided for, or even zero in the event of an early matching failure.

❖Fputs()

- ✓ **Int fputs(const char *str, FILE *stream);**
- ✓ Writes a string to the specified stream up to but not including the null character.
- ✓ This function returns a non-negative value, or else on error it returns EOF.

❖ **Fgets()**

- ✓ **Char *fgets(char *str, int n, FILE *stream);**
- ✓ Reads a line from the specified stream and stores it into the string pointed to by str. It stops when either (n-1) characters are read, the newline character is read, or the end-of-file is reached, whichever comes first.
- ✓ On success, the function returns the same str parameter. If the End-of-File is encountered and no characters have been read, the contents of str remain unchanged and a null pointer is returned.
- ✓ If an error occurs, a null pointer is returned.

❖ **Fwrite()**

- ✓ **Size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);**
- ✓ Writes data from the array pointed to, by ptr to the given stream.
- ✓ This function returns the total number of elements successfully returned as a size_t object, which is an integral data type. If this number differs from the nmemb parameter, it will show an error.

❖ **Fread()**

- ✓ **Size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);**
- ✓ Reads data from the given stream into the array pointed to, by ptr.
- ✓ The total number of elements successfully read are returned as a size_t object, which is an integral data type. If this number differs from the nmemb parameter, then either an error had occurred or the End Of File was reached.

Q51. WAP to find the min and max numbers in an array.

Answer→

```
#include<stdio.h>
```

```
Void Sort(int *p);
```

```
Int main()
```

```
{
```

```
    Int i=0,n,min;
```

```
    Int arr[5];
```

```

For(i=0;i<5;i++)
{
    Puts("Enter the numbers:");
    Scanf("%d",&arr[i]);
}
Puts("Elements of Array");
For(int j=0;j<5;j++)
    Printf("%d",arr[j]);
Printf("\n");
N=arr[0];
For(int j=1;j<5;j++)
{
    If(n<arr[j])
        N = arr[j];
}
Min=arr[0];
Int temp;
For(int j=1;j<5;j++)
{
    If(min>arr[j])
        Min = arr[j];
}
Printf("Largest no is: %d\n",n);
Printf("Smallest No is: %d\n",min);
}

```

Q52. WAP to delete a substring from a main string.

Q53. WAP to insert insertion sort?

Answer--→

```
#include<stdio.h>

void Sort(int *p);

int main()
{
    int i=0,n;
    int arr[5];
    for(i=0;i<5;i++)
    {
        puts("Enter the numbers:");
        scanf("%d",&arr[i]);
    }
    puts("Before Sort");
    for(int j=0;j<5;j++)
        printf("%d",arr[j]);
    printf("\n");
    Sort(arr);
}

void Sort(int *p)
{
    int temp;
    int j;
    for(int i=1;i<5;i++)
    {
```

```

temp=p[i];
j=i-1;
while(j>=0)
{
    if(temp<p[j])
    {
        p[j+1]=p[j];
        j--;
    }
    else
        break;
}
p[j+1]=temp;
}
puts("After Soting:");
for(int x=0;x<5;x++)
    printf("%d",p[x]);
printf("\n");
}

```

Q54. WAP to generate vowels from a string?

Q55. WAP to find the highest value from an array using 1 loop.

Answer --→

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int i=0,n;
```

```
    int arr[5];
```

```
    for(i=0;i<5;i++)          //This loop is only taken for getting array assigned if we don't  
want this for loop we can use
```

```
                                arr[] = {5,7,9,5,1}; & ignore this loop.
```

```
{
```

```
    puts("Enter the numbers:");
```

```
    scanf("%d",&arr[i]);
```

```
}
```

```
n=arr[0];
```

```
for(int j=1;j<5;j++)
```

```
{
```

```
    if(n<arr[j])
```

```
        n = arr[j];
```

```
}
```

```
printf("Largest no is: %d\n",n);
```

```
}
```

Q56. WAP to print binary equivalent of float using union.

Answer ---→

```
#include<stdio.h>
```

```
union t{
```

```
    float f;
```

```
    struct st{
```

```
        unsigned int mentisa:23; //Bit field of 23 bit for mantissa part.
```

```
        unsigned int exponant:8; //Bit Field of 8 bit for exponent part.
```

```
        unsigned int sign:1;    //Bit filed of 1 bit for signed part
```

```
    }row;
```

```
}u;
```

```
int main()
```

```
{
```

```
    union t v;
```

```
    puts("Enter Floating Real Value:");
```

```
    scanf("%f",&v.f);          //Getting Real Value Here
```

```
    printf("Binary Form is: ");
```

```
    printf("%d ",v.row.sign);    //1st Printing Signed bit of Real value
```

```
    /* For Exponent part */
```

```
    for(int i=8;i>=0;i--)        //loop for printing binary value till 8 bit
```

```
{
```

```
    if((v.row.exponant >> i) & 1) //if exponant of real value (in binary) is true then print 1
```

```
        printf("1");
```

```
        else
            printf("0");
    }

    /* For Mentisa Part */
    for(int i=23;i>=0;i--)
    {
        if((v.row.mentisa >> i) & 1)
            printf("1");
        else
            printf("0");
    }
    printf("\n");
}
```

Q57. WAP to delete all nodes of circular linked list.

Answer---→ Delete_all (For Circular Single Link List)

```
Struct st {
    Int data;
    Struct st *next;
};
Struct st *Delete_all(struct st *p)
{
    struct st *temp=NULL,*temp1=NULL;
    if(p == NULL)
    {
        puts("No Data Found");
        return 0;
    }
    else
    {
        temp1 = p;
        do{
            temp = p;
            p = p->next;
            free(temp);
            temp = NULL;
        }while(p!=temp1);
        free(p);
    }
}
```

```
    free(temp1);  
    temp1=NULL;  
    p= NULL;  
}  
return p;  
}
```